Bài giảng chi tiết về Machine Learning, Deep Learning và Logic mờ

Từ nền tảng toán học đến ANFIS và phân tích mô hình trong `run_feature_group_anfis.py`

Biên soạn cho người mới bắt đầu, nhưng giữ mức độ lập luận toán học chặt chẽ

OpenAI Codex

Ngày biên soạn: 13/03/2026

Tài liệu này được viết dưới dạng một bộ bài giảng có chủ đích giảng dạy, không phải một mục lục mở rộng. Mục tiêu là xây dựng một đường dây nhận thức liên tục: từ cách đặt bài toán học máy, đến nền tảng xác suất – tối ưu, đến các mô hình học máy cổ điển, mạng nơ-ron sâu, logic mờ, hệ suy diễn mờ Takagi–Sugeno, ANFIS, và cuối cùng là đọc, giải cấu trúc, phân tích lỗi, và đánh giá hệ quả của mô hình cụ thể trong `run_feature_group_anfis.py`.

Lời nói đầu

Tài liệu này được viết cho một đối tượng hơi khó chiều: người mới bắt đầu học *Machine Learning* (học máy), *Deep Learning* (học sâu), và logic mờ, nhưng lại muốn một bài giảng không "ru ngủ" bằng các danh sách tổng hợp ngắn gọn. Nếu ta học quá nhanh, ta dễ rơi vào một ảo tưởng nguy hiểm: nhớ tên mô hình thì tưởng như đã hiểu mô hình. Trong thực tế, điều khó nhất không nằm ở chỗ nhớ được tên "linear regression" (hồi quy tuyến tính), "transformer" (kiến trúc biến đổi), hay "ANFIS", mà nằm ở chỗ nhìn thấy mỗi mô hình đang giải một bài toán gì, đang áp đặt giả thiết gì lên dữ liệu, và đang đánh đổi điều gì để lấy điều gì.

Một bài giảng nghiêm túc về học máy nên bắt đầu từ câu hỏi rất cơ bản: khi ta nói một hệ thống "học", ta thực sự đang nói đến điều gì? Có một tập dữ liệu đầu vào, có một không gian giả thuyết, có một hàm mất mát, có một cơ chế tối ưu, và có nhiều nguồn bất định. Ở mức trừu tượng cao, rất nhiều mô hình khác nhau chỉ là những cách khác nhau để xây dựng và điều chỉnh một ánh xạ

$$f_\theta : \mathcal{X} \to \mathcal{Y}.$$

Sự khác nhau giữa hồi quy tuyến tính, cây quyết định, mạng nơ-ron sâu, hay hệ suy diễn mờ Takagi–Sugeno không nằm ở việc chúng đều "đoán" giá trị đầu ra, mà nằm ở ngôn ngữ biểu diễn, hình học của tham số, cách *regularize* (điều chuẩn), và cách con người đọc hiểu cấu trúc bên trong.

Có ba tuyến tư duy sẽ chạy song song xuyên suốt tài liệu này.

Tuyến thứ nhất là tư duy thống kê. Dữ liệu không bao giờ tự nhiên đến trong tinh khiết. Nó được tạo ra bởi một quá trình sinh dữ liệu nào đó, và ta chỉ quan sát được một mẫu hữu hạn. Từ đây xuất hiện rủi ro tổng quát hóa, phương sai của ước lượng, sai lệch chọn mô hình, và những ảo tưởng hiệu năng do *leakage* (rò rỉ thông tin) gây ra. Nếu không nắm vững tư duy thống kê, ta rất dễ bị mê hoặc bởi một giá trị R^2 đẹp, một *loss* (hàm mất mát) giảm đều, hay một biểu đồ dự báo "có vẻ đúng".

Tuyến thứ hai là tư duy tối ưu và biểu diễn. Mỗi mô hình học được đều cần một cơ chế cập nhật tham số. Với hồi quy *ridge* (hồi quy có phạt chuẩn L^2), cập nhật có thể có dạng đóng. Với *logistic regression* (hồi quy logistic), cập nhật dựa trên *gradient* (đạo hàm theo hướng tăng nhanh nhất). Với học sâu, *chain rule* (quy tắc dây chuyền) trở thành xương sống của *backpropagation* (lan truyền ngược). Với ANFIS, ta vừa có phần *premise* (tiền đề) mang

tính mờ, vừa có phần *consequent* (hệ quả) mang tính tuyến tính, nên cấu trúc học có thể pha trộn giữa *least squares* (bình phương tối thiểu) và *gradient descent* (hạ dốc theo gradient) [6]. Hiểu biểu diễn mà không hiểu tối ưu là thiếu một nửa; hiểu tối ưu mà không hiểu biểu diễn cũng vẫn thiếu nửa còn lại.

Tuyến thứ ba là tư duy diễn giải. Trong các hệ thống sử dụng cho tài chính, y tế, điều khiển công nghiệp, câu hỏi "vì sao mô hình ra kết quả này?" không phải là một câu hỏi phụ. *Fuzzy logic* (logic mờ) ra đời một phần vì nhu cầu giữ lại ngôn ngữ khái niệm của con người: thấp, cao, tăng mạnh, giảm nhẹ, xu hướng bất ổn. Học sâu, ngược lại, cực mạnh về biểu diễn và xấp xỉ phi tuyến, nhưng thường khó đọc. ANFIS là một nỗ lực nối hai truyền thống ấy: giữ cấu trúc suy diễn mờ, nhưng học tham số bằng phương pháp thích nghi [6, 9, 11].

Tài liệu này không có ý định thay thế các giáo trình kinh điển như Bishop [1], Hastie et al. [3], Goodfellow et al. [2], Murphy [8], Klir and Yuan [7], hay Ross [9]. Trái lại, nó được viết như một cây cầu nối. Nếu bạn chưa học qua những sách đó, tài liệu này cho bạn một bản đồ để đọc và tiêu hóa chúng. Nếu bạn đã học qua, nó giúp liên kết những miền kiến thức vốn thường bị dạy tách rời: thống kê học máy, học sâu, logic mờ, và *forecasting* (dự báo) cho dữ liệu chuỗi thời gian.

Bảng chú giải thuật ngữ cốt lõi

Để người đọc mới không bị đẩy vào tình trạng "thấy từ tiếng Anh thì mặc nhiên phải hiểu", tài liệu này dùng nguyên tắc sau: thuật ngữ chuyên môn sẽ được ưu tiên viết bằng tiếng Việt, đồng thời kèm thuật ngữ tiếng Anh ở lần xuất hiện đầu tiên hoặc tại những điểm cần so đối chiếu với tài liệu gốc. Dưới đây là một số thuật ngữ sẽ xuất hiện xuyên suốt:

Machine Learning	Học máy: ngành nghiên cứu các mô hình học quy luật từ dữ liệu để dự đoán, phân loại, hoặc ra quyết định.
Deep Learning	Học sâu: nhánh của học máy dùng mạng nơ-ron nhiều lớp để học biểu diễn phức tạp.
Feature	Đặc trưng: biến đầu vào dùng để mô tả đối tượng hay trạng thái của hệ thống.
Label / Target	Nhãn / biến mục tiêu: đại lượng mà mô hình cần dự đoán.
Loss	Hàm mất mát: đại lượng đo mức sai giữa dự đoán và giá trị thật.
Optimizer	Bộ tối ưu: thuật toán dùng để cập nhật tham số nhằm giảm hàm mất mát.
Gradient	Vector đạo hàm biểu thị hướng tăng nhanh nhất của hàm số.
Backpropagation	Lan truyền ngược: kỹ thuật tính gradient hiệu quả trong mạng nơ-ron.

Overfitting	Quá khớp: mô hình học quá sát dữ liệu huấn luyện nên tổng quát hóa kém.
Regularization	Điều chuẩn: cách ép mô hình bớt phức tạp hoặc bớt nhạy để giảm quá khớp.
Validation set	Tập thẩm định: dữ liệu dùng để chọn siêu tham số hoặc dừng sớm.
Test set	Tập kiểm tra: dữ liệu chỉ dùng để đánh giá cuối cùng.
Leakage	Rò rỉ thông tin: tình huống thông tin từ tương lai hoặc từ tập kiểm tra rò vào quá trình học.
Membership function	Hàm thuộc: hàm cho biết một giá trị thuộc vào một khái niệm mờ ở mức độ nào.
Fuzzy rule	Luật mờ: quy tắc dạng "nếu – thì" có dùng các khái niệm mờ.
Consequent	Phần hệ quả của luật mờ, tức vế "thì".
Premise	Phần tiền đề của luật mờ, tức vế "nếu".
Time series	Chuỗi thời gian: dãy quan sát có thứ tự theo thời gian.
Forecasting	Dự báo: dự đoán giá trị tương lai từ dữ liệu quá khứ và hiện tại.

Cách đọc tài liệu

Bạn có thể đọc từ đầu đến cuối, và đó là cách tôi khuyến khích. Tuy nhiên, nếu bạn có một mục tiêu rất cụ thể, vẫn có thể dùng tài liệu như một sổ tay có hệ thống.

Nếu bạn gặp khó ở phần toán, hãy đọc Chương 2 thật chậm. Trong rất nhiều trường hợp, cái cần không phải là "toán cao cấp" mà là sự rõ ràng trong định nghĩa. Thừa số này là *scalar* (đại lượng vô hướng) hay *vector*? Đạo hàm này theo biến nào? Xác suất này có điều kiện trên biến gì? Chỉ cần mù mờ ở những điểm đó, một chuỗi lập luận rất dễ sập.

Nếu bạn muốn hiểu học máy cổ điển trước khi vào học sâu, hãy đọc liền Chương 3 và Chương 4. Ở đó, ta thấy ý tưởng *empirical risk minimization* (tối thiểu hóa rủi ro thực nghiệm), điều chuẩn, *kernel* (hàm hạt nhân), cây quyết định, *ensemble* (mô hình tổ hợp), và *clustering* (phân cụm). Những mô hình ấy không "cũ" theo nghĩa lạc hậu. Ngược lại, chúng đặt ra những vấn đề cốt lõi mà mô hình hiện đại vẫn phải đối mặt.

Nếu bạn muốn hiểu tại sao học sâu học được những biểu diễn phức tạp, Chương 5 và Chương 6 sẽ là trọng tâm. Ở đó, ta phân tích *perceptron*, *MLP* (mạng nơ-ron nhiều lớp), lan truyền ngược, điều chuẩn, hiện tượng nổ gradient, triệt tiêu gradient, huấn luyện theo lô, và các quy tắc đánh giá.

Nếu bạn muốn tập trung vào logic mờ và ANFIS, hãy đọc Chương 8, Chương 9, Chương 10. Những chương này không chỉ giới thiệu "hàm thuộc là gì", mà còn đặt câu hỏi: vì sao toán

tử hợp trên tập mờ không thể chọn tùy ý, vì sao *T-norm* (chuẩn tam giác cho phép diễn giải phép "và" mờ) quan trọng, vì sao hệ Takagi–Sugeno phù hợp với học tham số, và vì sao khả năng diễn giải có thể suy giảm khi ta chèn thêm các khối học sâu vào sau lớp mờ.

Cuối cùng, Chương 11 sẽ trở lại với một file mã nguồn thật trong repo: `run_feature_group_anfis.` Mục tiêu của chương này là cho thấy một kỹ năng cực quan trọng của người học máy: đọc mô hình trong code, không chỉ trong bài báo. Mô hình trong code không phải lúc nào cũng giống mô hình trong mô tả. Đôi khi ý tưởng đúng nhưng triển khai sai. Đôi khi triển khai có vẻ chạy được nhưng giao thức đánh giá làm hỏng kết quả. Đôi khi "giải thích được" chỉ là một khẩu hiệu. Chương cuối sẽ chỉ rõ từng điểm như vậy.

Quy ước ký hiệu

Ta dùng chữ cái thường in đậm $\boldsymbol{x}$ cho vector đầu vào, y cho biến đích, θ cho tập tham số, và f_θ cho mô hình. Ký hiệu $\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$ chỉ tập dữ liệu huấn luyện hữu hạn. Ký hiệu

$$R(\theta) = \mathbb{E}_{(\boldsymbol{x},y)\sim P}\big[\ell(f_\theta(\boldsymbol{x}), y)\big]$$

chỉ rủi ro kỳ vọng, còn

$$\widehat{R}_n(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell(f_\theta(\boldsymbol{x}_i), y_i)$$

chỉ rủi ro thực nghiệm. Khi ta nói "học", ta thường đang tìm một nghiệm gần đúng của bài toán

$$\theta^\star \in \arg\min_\theta \widehat{R}_n(\theta) + \lambda\Omega(\theta),$$

trong đó Ω là số hạng điều chuẩn.

Với dữ liệu chuỗi thời gian, ta sẽ dùng chỉ số thời điểm t. Biến x_t là quan sát ở thời điểm t, còn dự báo một bước trước là $\hat{y}_{t+1|t}$, nghĩa là dự báo cho thời điểm $t+1$ dựa trên thông tin có sẵn đến hết thời điểm t.

Trong logic mờ, tập mờ A trên miền X được mô tả bởi hàm thuộc

$$\mu_A : X \to [0, 1].$$

Giá trị $\mu_A(x)$ không phải xác suất "x thuộc A" mà là mức độ mà x thỏa khái niệm A. Đây là phân biệt rất quan trọng và thường bị bỏ qua.

Triết lý sử dụng toán học

Tài liệu này không có chủ đích giảm nhẹ toán học chỉ để "dễ đọc". Các chứng minh và lập luận sẽ được viết chặt chẽ trong phạm vi cần thiết. Tuy nhiên, chặt chẽ không có nghĩa là lan man kỹ thuật. Chứng minh được đưa vào khi nó giúp ta hiểu lý do mô hình hoạt động, nhận ra điều kiện để kết luận đúng, hoặc thấy điểm mỏng để mô hình thất bại. Chứng minh không được đưa vào chỉ để trang trí.

Ví dụ, ta sẽ không chỉ viết công thức *normal equation* (phương trình chuẩn) cho hồi quy tuyến tính, mà sẽ chỉ ra nó đến từ điều kiện gradient bằng không, và vì sao tính xác định dương của ma trận $X^\top X$ lại quan trọng. Ta sẽ không chỉ viết công thức cập nhật LSTM, mà sẽ phân tích vai trò của cổng nhớ và tương tác của nó với hiện tượng triệt tiêu gradient [4]. Ta sẽ không chỉ viết ANFIS có năm lớp, mà sẽ chỉ rõ từng lớp đang tính toán đại lượng nào, nơi nào có thể học bằng bình phương tối thiểu, nơi nào phải dùng hạ dốc theo gradient [6].

Điểm kết nối cuối cùng

Nếu bạn học đến hết tài liệu này, điều quan trọng nhất không phải là thuộc lòng thêm nhiều công thức, mà là hình thành một thói quen trí tuệ: mỗi khi gặp một mô hình mới, hãy hỏi liên tiếp năm câu hỏi.

Tham số của nó biểu diễn điều gì?

Hàm mất mát có phù hợp với bài toán không?

Dữ liệu đã được tách *train/validation/test* (huấn luyện/thẩm định/kiểm tra) đúng cách chưa?

Chỉ số đánh giá có thực sự đo đúng thứ cần đo không?

Và cuối cùng, điều gì trong mô hình là giải thích được, điều gì chỉ là ảo giác diễn giải?

Nếu năm câu hỏi này trở thành phản xạ, bạn đã đi được rất xa trên con đường học máy.

Contents

Lời nói đầu **ii**

Bảng chú giải thuật ngữ cốt lõi iii

Cách đọc tài liệu iv

Quy ước ký hiệu v

Triết lý sử dụng toán học vi

Điểm kết nối cuối cùng vi

1 Tổng quan thống nhất về học máy, học sâu và logic mờ **1**

1.1 Học máy là bài toán xấp xỉ hàm dưới ràng buộc dữ liệu 1

1.2 Ba vấn đề lớn: biểu diễn, tổng quát hóa và diễn giải 2

1.2.1 Biểu diễn 2

1.2.2 Tổng quát hóa 2

1.2.3 Diễn giải 2

1.3 Thống kê học máy và kỹ thuật biểu diễn 3

1.4 Khung nhìn Bayes và khung nhìn tối ưu 3

1.5 Từ mô hình cổ điển đến học sâu 4

1.6 Từ logic kinh điển đến logic mờ 4

1.7 ANFIS như một cây cầu nối 5

1.8 Chuỗi thời gian, dự báo và những cái bẫy đánh giá 5

1.9 Một số nguyên lý để giữ trên bàn làm việc 6

1.10 Kết 6

2 Nền tảng toán học: đại số tuyến tính, xác suất và tối ưu **7**

2.1 Vì sao người học máy cần một nền toán vững 7

2.2 Vectơ, ma trận, chuẩn và hình học 7
2.3 Trị riêng, vectơ riêng và giảm chiều 8
2.4 Đạo hàm, gradient, Jacobian và Hessian 9
2.5 Quy tắc dây chuyền và lan truyền ngược được xây trên nó 11
2.6 Xác suất như ngôn ngữ mô tả bất định 11
2.7 Xác suất có điều kiện, Bayes và cực đại hóa hàm khả năng 12
2.8 Kỳ vọng có điều kiện và dự báo tối ưu theo MSE 13
2.9 Entropy, cross-entropy và độ lệch KL 14
2.10 Tối ưu lồi, Lagrange và điều kiện tối ưu 15
2.11 Hạ dốc theo gradient trên hàm bậc hai 15
2.12 Đạo hàm ma trận mà ta sẽ cần nhiều lần 16
2.13 Nói về ổn định số học 16
2.14 Mẫu số chuẩn cho các chương sau 16

3 Học máy thống kê: từ rủi ro thực nghiệm đến điều chuẩn 18
3.1 Từ dữ liệu hữu hạn đến bài toán học 18
3.2 Hồi quy tuyến tính và phương trình chuẩn 19
3.2.1 Góc nhìn hình học 19
3.3 Đánh đổi độ chệch – phương sai 20
3.4 Hồi quy ridge: điều chuẩn như một tiên nghiệm Gaussian 20
3.5 Lasso và tính thưa 21
3.6 Hồi quy logistic và cực đại hóa hàm khả năng phân loại 21
3.7 Mô hình hóa xác suất và hàm mất mát 22
3.8 Mô hình, tập thẩm định, tập kiểm tra và giao thức chọn siêu tham số 22
3.9 Kiểm định chéo và giới hạn của nó trong chuỗi thời gian 23
3.10 Chỉ số nào cho bài toán nào 23
3.11 Nguồn gốc của rò rỉ thông tin 24
3.12 Những gì cần giữ lại trước khi sang các mô hình phi tuyến 24

4 Các mô hình cổ điển phi tuyến: cây, hạt nhân, tổ hợp và phân cụm 25
4.1 Vì sao vẫn phải học các mô hình “cổ điển” 25
4.2 k-NN: phi tham số theo nghĩa địa phương 25

4.3 Cây quyết định: phân hoạch không gian bằng luật nếu–thì 26
4.3.1 Tiêu chuẩn tách 26
4.4 *Bagging* và rừng ngẫu nhiên 26
4.5 *Boosting*: học lặp để sửa sai 27
4.6 Máy vectơ hỗ trợ và biên 27
4.7 Mẹo hạt nhân: phi tuyến nhưng vẫn lồi 28
4.8 Không gian Hilbert tái sinh và định lý biểu diễn 28
4.9 Phân cụm và K-Means 28
4.10 Hỗn hợp Gaussian và EM 29
4.11 PCA và giảm chiều trong quy trình học máy 29
4.12 Mô hình cổ điển dạy ta điều gì về diễn giải 30
4.13 Nội dung của chương này sẽ trở lại ở đâu 30

5 Mạng nơ-ron: từ perceptron đến lan truyền ngược 31
5.1 Perceptron và giới hạn của tuyến tính 31
5.2 Một lớp ẩn đã làm thay đổi tất cả 31
5.2.1 Định lý xấp xỉ phổ quát 32
5.3 Hàm kích hoạt và hình học của gradient 32
5.4 Suy dẫn lan truyền ngược từ quy tắc dây chuyền 32
5.5 Ma trận hóa trên *mini-batch* 33
5.6 Từ MLP sang học biểu diễn 33
5.7 Điều chuẩn trong mạng nơ-ron 33
5.8 Triệt tiêu và bùng nổ gradient 34
5.9 *Softmax* và entropy chéo cho phân loại nhiều lớp 34
5.10 Tối ưu trong mạng nơ-ron 35
5.11 Từ mạng nơ-ron đến tư duy hồi quy theo thời gian 35
5.12 Mạng nơ-ron và vấn đề diễn giải 35
5.13 Kết 35

6 Học sâu trong thực hành: tối ưu, điều chuẩn và động học học 37
6.1 Từ công thức mạng nơ-ron đến một hệ thống học hoàn chỉnh 37
6.2 Hình học của khởi tạo: tại sao bước đầu tiên lại có sức chi phối lớn 38

6.2.1 Khởi tạo Xavier và He 38
6.2.2 Khởi tạo và các hàm kích hoạt bão hòa 38
6.3 Mini-batch: tại sao nhiễu lại giúp học 39
6.3.1 Nhiễu giúp thoát vùng bằng phẳng và hố nông 39
6.3.2 Kích thước lô là một siêu tham số thật sự 39
6.4 Động lượng, RMSProp và Adam dưới góc nhìn hình học 40
6.4.1 Động lượng như trung bình động của hướng giảm 40
6.4.2 RMSProp và ý tưởng co giãn theo tọa độ 40
6.4.3 Adam như sự kết hợp giữa quán tính và co giãn 40
6.4.4 Một ngộ nhận phổ biến về Adam 41
6.5 Lịch trình tốc độ học: nghệ thuật điều khiển quy mô bước đi 41
6.5.1 Vì sao cần giảm tốc độ học theo thời gian 41
6.5.2 Warmup: tại sao có lúc phải tăng tốc độ học từ nhỏ lên lớn 41
6.5.3 Hai triết lý điều chỉnh tốc độ học 41
6.6 Chuẩn hóa trong mạng: không chỉ để làm đẹp dữ liệu 42
6.6.1 Batch normalization 42
6.6.2 Tại sao chuẩn hóa giúp học 42
6.6.3 Layer normalization và các biến thể 43
6.7 Điều chuẩn tường minh và điều chuẩn ngầm 43
6.7.1 Điều chuẩn tường minh 43
6.7.2 Điều chuẩn ngầm 43
6.8 Dropout được hiểu đúng là gì 44
6.9 Dừng sớm: một điều chuẩn mạnh nhưng dễ bị lạm dụng 44
6.10 Tối ưu thành công không đồng nghĩa với mô hình đúng 44
6.10.1 Giảm loss chỉ nói về bài toán tối ưu đang được giải 44
6.10.2 Hiệu năng cao có thể đến từ cấu trúc thống kê dễ dàng 45
6.11 Độ sâu, chiều rộng và thiên kiến quy nạp 45
6.12 Khi nào một mô hình học sâu thất bại dù công thức đều đúng 45
6.13 Mất mát, mục tiêu và bài toán ra quyết định 46
6.13.1 Hiệu chỉnh theo miền ứng dụng 46
6.14 Diễn giải trong học sâu: điều gì làm được, điều gì không 46

6.15 Quy trình huấn luyện nghiêm túc cho người mới 47
6.16 Một nhận xét quan trọng cho người học ANFIS và học sâu cùng lúc 47
6.17 Kết luận chương 47

7 Chuỗi thời gian, dự báo và RNN/LSTM/BiLSTM 48
7.1 Tư duy đúng về chuỗi thời gian: dữ liệu và thông tin không phải một 48
7.2 Tại sao dữ liệu chuỗi thời gian không thể bị đối xử như dữ liệu i.i.d. 48
7.3 Tính dừng, xu thế và lý do người ta thích dùng return 49
7.3.1 Một hiểu lầm thường gặp về return 50
7.4 Biến mục tiêu trong dự báo: phải nói chính xác mốc so sánh 50
7.5 Cửa sổ trượt: biến chuỗi thành tensor đầu vào 50
7.5.1 Điểm dễ sai nhất: căn chỉnh chỉ số 51
7.6 Dự báo một bước, nhiều bước, trực tiếp và đệ quy 51
7.6.1 Dự báo trực tiếp 51
7.6.2 Dự báo đệ quy 51
7.6.3 Dự báo hỗn hợp 51
7.7 Đánh giá chuỗi thời gian: backtesting quan trọng ngang mô hình 52
7.7.1 Ba giao thức đánh giá cơ bản 52
7.7.2 Tập kiểm tra không phải nơi chọn mô hình 52
7.8 RNN cơ bản và lý do nó gặp khó khăn 52
7.8.1 Một phân tích rất ngắn về triệt tiêu gradient 53
7.9 LSTM: cổng nhớ như một cấu trúc tối ưu hóa, không chỉ là một công thức . 53
7.9.1 Ý nghĩa của cổng quên 53
7.9.2 LSTM không phải ký ức vô hạn 54
7.10 BiLSTM trong dự báo: khi nào hợp lý, khi nào dễ bị hiểu nhầm 54
7.11 Dự báo tài chính với OHLC: bài toán có cấu trúc hình học nội tại 54
7.11.1 Một ví dụ về dự báo vô nghĩa nhưng RMSE vẫn đẹp 55
7.12 Chỉ số đánh giá cho chuỗi thời gian: cần đọc đúng ngữ nghĩa 55
7.12.1 Độ chính xác hướng 55
7.13 Dự báo xác suất và bất định trong chuỗi thời gian 55
7.14 Các nguồn rò rỉ thông tin đặc thù của chuỗi thời gian 56

7.15 Từ lý thuyết đến phần phân tích script 56
7.16 Kết luận chương 56

8 Cơ sở của logic mờ và tập mờ 58

8.1 Vì sao logic hai giá trị chưa đủ cho nhiều bài toán thật 58
8.2 Tập mờ, hàm thuộc, miền đỡ và lõi 59
8.3 α-cut và định lý biểu diễn 60
8.4 Các phép toán trên tập mờ và vai trò của chuẩn tam giác 61
8.5 Biến ngôn ngữ, phân hoạch mờ và quá trình mờ hóa 63
8.6 Quan hệ mờ, phép hợp thành và suy luận nếu–thì 64
8.7 Nguyên lý mở rộng và cách lan truyền độ mờ qua phép biến đổi 65
8.8 Số mờ, tính lồi và số học mờ 65
8.9 Logic mờ và xác suất: hai loại bất định khác nhau 66
8.10 Giải mờ và quyết định cuối cùng 66
8.11 Tính diễn giải không tự sinh ra từ việc gắn nhãn LOW, HIGH 67
8.12 Bài học chuyển tiếp sang ANFIS 68

9 Hệ suy diễn mờ: từ Mamdani đến Takagi–Sugeno 69

9.1 Luật nếu–thì là cách biến trực giác thành cấu trúc toán học 69
9.2 Làm mờ hóa như một phép trích xuất đặc trưng 70
9.3 Độ kích hoạt của luật và ý nghĩa của phép “và” 70
9.3.1 Hai lựa chọn kinh điển 70
9.3.2 Một ví dụ so sánh 71
9.4 Phép kéo theo mờ và vai trò của nó trong suy diễn 71
9.5 Hệ Mamdani: khi vế hệ quả vẫn là ngôn ngữ 71
9.5.1 Ba bước của Mamdani 71
9.5.2 Vì sao Mamdani hấp dẫn 72
9.5.3 Giới hạn của Mamdani 72
9.6 Hệ Takagi–Sugeno–Kang: khi hệ quả là mô hình cục bộ sắc nét 72
9.6.1 Đầu ra như tổ hợp của các mô hình cục bộ 72
9.6.2 Hệ Sugeno như một phân hoạch đơn vị 73
9.7 TSK bậc 0 và bậc 1: khác nhau không chỉ ở số tham số 73

9.7.1 TSK bậc 0 73
9.7.2 TSK bậc 1 74
9.8 Cơ sở luật được sinh ra như thế nào 74
9.8.1 Phân hoạch lưới toàn phần 74
9.8.2 Sinh luật dựa trên phân cụm 74
9.8.3 Luật do chuyên gia cung cấp 74
9.9 Sự đánh đổi giữa độ phủ và tính phân biệt 75
9.10 Hệ mờ như một mô hình chuyên gia cục bộ 75
9.11 Tính nhận dạng và sự mong manh của nhãn ngôn ngữ 75
9.12 Diễn giải và cái giá của diễn giải 75
9.13 Lỗi mà các chỉ số đánh giá không nhìn thấy 76
9.14 Từ hệ suy diễn mờ đến ANFIS 76

10 ANFIS: kiến trúc, học tham số và những giới hạn 77
10.1 ANFIS là gì nếu nói thật chặt 77
10.2 Mẫu hai đầu vào, hai luật: nơi mọi công thức của ANFIS lộ ra rõ nhất . . . 78
10.3 Năm lớp của ANFIS và ý nghĩa thật của từng lớp 78
10.3.1 Lớp 1: làm mờ hóa 78
10.3.2 Lớp 2: kích hoạt luật 79
10.3.3 Lớp 3: chuẩn hóa 79
10.3.4 Lớp 4: hệ quả 79
10.3.5 Lớp 5: tổng hợp 79
10.4 Điều gì làm ANFIS đẹp về mặt toán học 79
10.4.1 Lý do thứ nhất: nó là mô hình cục bộ hóa có cấu trúc 79
10.4.2 Lý do thứ hai: một nửa bài toán có dạng tuyến tính 80
10.4.3 Lý do thứ ba: nó giữ được cầu nối diễn giải 80
10.5 Học lai của ANFIS: suy ra từ dạng tuyến tính của hệ quả 80
10.5.1 Ý nghĩa của điều này 80
10.6 Ma trận hóa của phần hệ quả và mối liên hệ với hồi quy tuyến tính 81
10.7 Đạo hàm của phần tiền đề và động học học 81
10.8 Khởi tạo các hàm thuộc: vì sao phân cụm thường được dùng 82

10.9 Bùng nổ số luật: bài toán trung tâm của ANFIS 82
10.9.1 Các chiến lược giảm 82
10.10 ANFIS nhiều đầu ra: tiện nhưng làm diễn giải khó hơn 83
10.11 ANFIS và tính diễn giải: cần những điều kiện nào 83
10.12 Tính nhận dạng, hoán vị luật và sự mong manh của giải thích 83
10.13 ANFIS trong framework học sâu hiện đại: lợi và hại 84
10.13.1 Rủi ro thứ nhất: mất học lai 84
10.13.2 Rủi ro thứ hai: giữ bề ngoài mà mất bản chất 84
10.13.3 Rủi ro thứ ba: nhầm diễn giải cục bộ với diễn giải toàn cục 84
10.14 Khi nào không nên dùng ANFIS 84
10.15 Những lỗi lập trình ANFIS hay gặp và vì sao chúng không nhỏ 85
10.16 ANFIS trong bài báo và ANFIS trong ứng dụng thực tế 85
10.17 Kết luận chương 85

11 Phân tích lý thuyết và kiểm toán kỹ thuật file `run_feature_group_anfis.py` 87

11.1 Phải đọc chương này như thế nào để không rơi vào suy diễn cảm tính . . . 87
11.2 Bài toán toán học mà script thực sự đang giải là gì 88
11.3 Kiến trúc thực sự của mô hình và những ý tưởng đáng ghi nhận 89
11.4 Ba sai phạm phương pháp luận làm hỏng giá trị của chỉ số đánh giá 91
11.5 Tại sao tuyên bố “giải thích được” của mô hình hiện tại bị thổi phồng . . . 92
11.6 Những bất nhất mô hình hóa khác: không phải lỗi nào cũng cùng mức độ, nhưng đều đáng nói 94
11.7 Điều gì nên được nói công bằng về BiLSTM, K-Means và bài báo tham chiếu 95
11.8 Nếu muốn biến mô hình này thành một nghiên cứu đáng tin hơn, phải sửa theo thứ tự nào 96
11.9 Kết luận của chương và bài học phương pháp lớn hơn chính script này . . . 97

12 Tổng kết, định hướng tự học và bài tập 98

12.1 Sau khi đi hết giáo trình này, điều gì thật sự cần ở lại trong đầu người học . 98
12.2 Tự học tiếp như thế nào để không sa vào việc chỉ sưu tập tên mô hình . . . 99
12.3 Nếu viết lại mô hình trong repo như một nghiên cứu nghiêm túc, ta phải bắt đầu từ đâu 100
12.4 Bài tập tự luyện: từ toán nền đến đọc code thật 101

12.5 Kết lời 103

13 Phụ lục: ví dụ tính tay, quy trình chuẩn và khung sửa mô hình 104

13.1 Mục đích của phụ lục 104

13.2 Ví dụ tính tay 1: hồi quy tuyến tính, đạo hàm và nghiệm đóng 104

13.3 Ví dụ tính tay 2: suy diễn mờ Sugeno với hai luật 105

13.4 Ví dụ tính tay 3: ANFIS hai đầu vào, bốn luật 106

13.5 Quy trình chuẩn cho dữ liệu chuỗi thời gian có ANFIS và học sâu 107

13.5.1 Bước 1: xác định mục tiêu thật rõ 107

13.5.2 Bước 2: tách dữ liệu theo thời gian trước khi học bất kỳ biến đổi nào 107

13.5.3 Bước 3: khởi tạo khối mờ trên tập huấn luyện 108

13.5.4 Bước 4: huấn luyện với tập thẩm định đúng nghĩa 108

13.5.5 Bước 5: khóa mô hình rồi mới động đến tập kiểm tra 108

13.6 Mã giả cho quy trình sửa của script 108

13.7 Khung sửa mô hình để giữ ràng buộc OHLC 108

13.7.1 Phương án 1: dự báo $(O, C, \Delta_H, \Delta_L)$ 109

13.7.2 Phương án 2: thêm số hạng phạt vào hàm mất mát 109

13.8 Khung sửa mô hình để giữ nghĩa của nhãn LOW/HIGH 109

13.8.1 Cách 1: tham số hóa có thứ tự 109

13.8.2 Cách 2: sắp xếp lại sau mỗi vòng cập nhật 109

13.9 Khung sửa phần trích xuất luật 110

13.10 Một ví dụ về báo cáo lỗi tốt hơn cho mô hình 110

13.10.1 Nhóm 1: chỉ số dự báo 110

13.10.2 Nhóm 2: chỉ số hợp lệ cấu trúc 110

13.10.3 Nhóm 3: chỉ số diễn giải 110

13.10.4 Nhóm 4: chỉ số phương pháp luận 111

13.11 Một lời nhắc cuối về tinh thần học 111

14 Những ngộ nhận thường gặp khi học ML, DL, logic mờ và ANFIS 112

14.1 Ngộ nhận không chỉ là câu sai, mà là một khung nhìn sai 112

14.2 Cụm ngộ nhận thứ nhất: nhầm độ phức tạp kiến trúc và bảng chỉ số với sức mạnh khoa học 112

14.3 Cụm ngộ nhận thứ hai: nhầm logic mờ với xác suất, và nhầm sự hiện diện của luật với khả năng diễn giải 115
14.4 Cụm ngộ nhận thứ ba: nhầm dáng vẻ học thuật, trích dẫn và khả năng chạy một lần với chiều sâu thật sự 116
14.5 Một quy trình tự kiểm để chống ngộ nhận khi đọc mô hình 118
14.6 Kết luận chương 118

Chuong 1

Tổng quan thống nhất về học máy, học sâu và logic mờ

1.1 Học máy là bài toán xấp xỉ hàm dưới ràng buộc dữ liệu

Một cách nhìn đơn giản nhưng cực mạnh về học máy là xem nó như bài toán xấp xỉ hàm. Ta có một không gian đầu vào $\mathcal{X}$, một không gian đầu ra $\mathcal{Y}$, và một quan hệ tiềm ẩn giữa chúng mà ta không biết trực tiếp. Nếu dữ liệu được sinh bởi một phân phối $P(\boldsymbol{x}, y)$, thì về bản chất ta muốn tìm một hàm f sao cho $f(\boldsymbol{x})$ "gần" y theo một khái niệm gần nào đó, được mô tả bởi *hàm mất mát* (*loss function*) ℓ.

Về mặt hình thức, nếu được phép tìm trên tất cả các hàm đo được, bài toán lý tưởng là

$$f^\star \in \arg\min_f \mathbb{E}[\ell(f(\boldsymbol{x}), y)].$$

Nhưng bài toán này không trực tiếp giải được vì ít nhất ba lý do. Thứ nhất, ta không biết phân phối sinh dữ liệu. Thứ hai, không gian tất cả các hàm là quá lớn. Thứ ba, ngay cả khi biết phân phối, bài toán tối ưu vẫn có thể vô cùng khó. Bởi vậy, học máy đặt ra một chiến lược gồm ba bước:

1. chọn một họ tham số hóa $\{f_\theta : \theta \in \Theta\}$;
2. dùng dữ liệu hữu hạn để xây dựng rủi ro thực nghiệm $\widehat{R}_n(\theta)$;
3. giải một bài toán tối ưu hữu hạn trên Θ.

Ba bước này xuất hiện ở hầu hết mọi mô hình, từ hồi quy tuyến tính đến mạng nơ-ron sâu. Hồi quy tuyến tính chọn họ hàm affine. *Kernel method* (phương pháp hạt nhân) chọn một không gian Hilbert tái sinh phong phú hơn. Cây quyết định chọn lớp hàm dạng phân hoạch từng phần. Mạng nơ-ron chọn lớp hàm là hợp thành của các biến đổi affine và phi tuyến. ANFIS chọn lớp hàm gồm các luật mờ có trọng số và *consequent* (vế hệ quả) Sugeno.

Điều quan trọng là: mô hình không chỉ khác nhau ở "sức mạnh biểu diễn", mà còn khác nhau ở hình học của bài toán học. Hồi quy *ridge* là bài toán lồi, có nghiệm đóng. Hồi quy logistic là lồi nhưng không có nghiệm đóng đơn giản. Mạng sâu tổng quát là không lồi. ANFIS là mô hình lai: nếu cố định *premise parameters* (tham số tiền đề), phần *consequent* có thể trở thành bài toán bình phương tối thiểu; nếu phần tiền đề cũng học, bài toán trở thành phi lồi [6]. Vì vậy, khi nói một mô hình "tốt", ta phải học cách trả lời: tốt theo nghĩa nào? Tốt về khả năng xấp xỉ? Tốt về khả năng học? Tốt về tổng quát hóa? Hay tốt về diễn giải?

1.2 Ba vấn đề lớn: biểu diễn, tổng quát hóa và diễn giải

1.2.1 Biểu diễn

Sức mạnh biểu diễn của mô hình là khả năng của lớp hàm được chọn trong việc mô tả những quan hệ phức tạp. Hồi quy tuyến tính có sức mạnh hạn chế nếu biến đầu vào đi vào một cách trực tiếp, nhưng có thể mạnh hơn rất nhiều nếu ta mở rộng đặc trưng. *Kernel trick* (mẹo hạt nhân) cho phép thực hiện điều đó một cách gián tiếp. Học sâu tự học biểu diễn thông qua các lớp trung gian. Hệ mờ đưa sức mạnh biểu diễn vào sự kết hợp các luật ngôn ngữ và các hàm thuộc. Mỗi cách biểu diễn sẽ áp đặt một độ trơn nhất định lên hàm cần học.

1.2.2 Tổng quát hóa

Tổng quát hóa là khả năng hoạt động tốt trên dữ liệu chưa thấy. Đây không phải là chi tiết phụ. Một mô hình khớp cực tốt trên tập huấn luyện nhưng thua trên tập kiểm tra về cơ bản là không giải được bài toán học máy. Vấn đề tổng quát hóa đưa đến các khái niệm như *bias–variance trade-off* (đánh đổi giữa độ chệch và phương sai), *overfitting* (quá khớp), điều chuẩn, *cross-validation* (kiểm định chéo), và *sample complexity* (độ phức tạp theo số mẫu) [1, 3]. Trong chuỗi thời gian, tổng quát hóa còn nhạy cảm hơn vì thứ tự thời gian áp đặt cấu trúc thông tin: tương lai không được rò rỉ vào quá khứ.

1.2.3 Diễn giải

Con người hiếm khi hài lòng với một con số dự báo nếu không hiểu được nó được tạo ra thế nào. Tuy nhiên, diễn giải không phải một khái niệm đơn. Một mô hình có thể diễn giải ở cấp cục bộ nhưng không ở cấp toàn cục. Một hệ mờ có thể đọc được luật, nhưng nếu sau lớp mờ ta lại nối vào một khối học sâu không đọc được, thì khả năng diễn giải sẽ chỉ còn một phần. Đây là lý do ta sẽ không vô điều kiện đồng nhất "có fuzzy" với "có thể giải thích được".

1.3 Thống kê học máy và kỹ thuật biểu diễn

Trong nhiều tài liệu, *Machine Learning* (học máy) được chia thành *supervised learning* (học có giám sát), *unsupervised learning* (học không giám sát), *self-supervised learning* (tự giám sát), và *reinforcement learning* (học tăng cường). Cách chia này hữu dụng, nhưng nếu dùng nó quá sớm, người mới dễ bỏ lỡ tính thống nhất bên dưới.

Trong học có giám sát, ta có nhãn mục tiêu và muốn học ánh xạ từ $\boldsymbol{x}$ sang y. Hồi quy và phân loại nằm ở đây. Trong học không giám sát, ta muốn tìm cấu trúc nội tại của dữ liệu mà không có nhãn rõ ràng: phân cụm, giảm chiều, mô hình mật độ, biến ẩn. Trong tự giám sát, ta tạo ra nhãn giả từ chính dữ liệu. Trong học tăng cường, dữ liệu đến từ tương tác có phản hồi trì hoãn.

Được nhìn từ xa hơn, tất cả các kiểu học này đều trả lời câu hỏi: làm sao tạo ra một biểu diễn hữu ích từ dữ liệu, và làm sao sử dụng biểu diễn đó để ra quyết định? Học sâu nổi bật ở chỗ nó khiến quá trình tạo biểu diễn trở thành phần bên trong của bài toán tối ưu. Trong những mô hình cổ điển, ta thường phải thiết kế đặc trưng. Trong học sâu, đặc trưng được học. Trong logic mờ, đặc trưng vẫn tồn tại, nhưng ta còn gán cho chúng ý nghĩa ngôn ngữ. ANFIS đứng giữa hai thế giới: hàm thuộc là một dạng trích xuất đặc trưng phi tuyến, trong khi vế hệ quả Sugeno giống một mô hình cục bộ có cấu trúc tuyến tính.

1.4 Khung nhìn Bayes và khung nhìn tối ưu

Có hai cách nhìn lớn về học máy. Cách nhìn thứ nhất là tối ưu thuần túy: chọn hàm mất mát, rồi tối thiểu hóa nó. Cách nhìn thứ hai là xác suất/Bayes: chọn mô hình xác suất $p_\theta(y|\boldsymbol{x})$, đặt *prior* (phân phối tiên nghiệm) lên tham số nếu cần, rồi sử dụng suy diễn để tìm *posterior* (phân phối hậu nghiệm) hay nghiệm *MAP* (cực đại hậu nghiệm).

Hai cách nhìn này thường hội tụ vào nhau. Ví dụ, nếu giả sử

$$y = \boldsymbol{w}^\top \boldsymbol{x} + b + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma^2),$$

thì tối đa hóa *likelihood* (hàm khả năng) tương đương với tối thiểu hóa tổng bình phương sai số. Nếu ta đặt *prior* Gaussian lên $\boldsymbol{w}$, bài toán MAP tương đương với hồi quy ridge. Sự song song giữa thống kê và tối ưu là một chủ đề lớn của tài liệu này: rất nhiều "thuật toán" hóa ra là hệ quả của một giả thiết xác suất.

Với logic mờ, khung Bayes không xuất hiện theo cách trực tiếp như vậy, vì *membership degree* (độ thuộc) không phải xác suất. Tuy nhiên, một tinh thần chung vẫn còn: ta đang mô hình hóa bất định và mơ hồ theo một ngôn ngữ hình thức nào đó. Xác suất mô tả bất định do thiếu thông tin ngẫu nhiên; logic mờ mô tả độ mờ của khái niệm. Hai thứ này liên quan nhưng không đồng nhất.

1.5 Từ mô hình cổ điển đến học sâu

Sẽ là sai nếu ta xem học sâu là một cuộc cắt đứt khỏi học máy cổ điển. Mạng nơ-ron sâu là một phần của lịch sử dài hơn về xấp xỉ hàm và tối ưu. *Perceptron* nối tiếp bộ phân loại tuyến tính. *Multi-layer perceptron* (mạng nhiều lớp) tổng quát hóa bằng các lớp phi tuyến. *Backpropagation* đơn giản là sự áp dụng quy tắc dây chuyền có hệ thống. Các kỹ thuật điều chuẩn, *mini-batch*, *momentum*, và sau này là *Adam* chỉ là những biện pháp để giải bài toán tối ưu tham số lớn và không lồi [2].

Tuy nhiên, học sâu bổ sung hai thứ quan trọng. Thứ nhất, độ sâu của biểu diễn làm cho cùng một số lượng tham số có thể biểu diễn những hàm khó hơn so với mạng nông. Thứ hai, nó cho phép học biểu diễn đặc trưng trực tiếp từ dữ liệu thô, nhất là khi dữ liệu có cấu trúc phức tạp như ảnh, âm thanh, văn bản, hay chuỗi thời gian đã biến đổi.

Trong dự báo tài chính, học sâu thường được sử dụng để học phụ thuộc thời gian, thay vì chỉ nhìn ánh xạ tĩnh giữa vectơ đặc trưng và đầu ra. *LSTM* được thiết kế để giải quyết vấn đề *vanishing gradient* (triệt tiêu gradient) trong *RNN* chuẩn [4]. *BiLSTM* thêm một hướng đọc ngược chuỗi [10]. Tuy nhiên, sức mạnh này không đồng nghĩa là nó luôn phù hợp; ta phải hỏi chuỗi dữ liệu có đủ dài, có ổn định, và có tri thức tiên nghiệm nào giúp điều chuẩn hay không.

1.6 Từ logic kinh điển đến logic mờ

Logic kinh điển làm việc với mệnh đề đúng hoặc sai. Trong rất nhiều bài toán kỹ thuật, ta gặp những khái niệm không có biên giới sắc nét. Nhiệt độ 29°C có "nóng" không? Biến động giá 1.8% là "tăng mạnh" hay "tăng vừa"? Nếu ép mọi khái niệm về 0 hoặc 1, ta mất đi tính liên tục mà con người thực sự sử dụng khi suy luận.

Fuzzy set theory (lý thuyết tập mờ) của Zadeh [11] đề xuất dùng mức độ thuộc trong đoạn $[0, 1]$ để mô tả những khái niệm này. Từ đó, ta có thể xây dựng quy tắc như

Nếu biến động giá là CAO và độ rộng dao động là THẤP thì chế độ thị trường là ỔN ĐỊNH.

Khác với hệ thống xác suất, mức "0.7" ở đây không có nghĩa là biến cố có xác suất 70% xảy ra; nó chỉ ra mức độ mà một quan sát phù hợp với khái niệm ngôn ngữ "CAO".

Logic mờ trở nên đặc biệt hấp dẫn khi con người cần giao tiếp với mô hình. Trong bối cảnh tài chính, ta có thể muốn nhìn được quy tắc "nếu khoảng nhảy giữa giá mở và giá đóng của ngày trước lớn và biến động trong ngày hôm nay thấp thì kỳ vọng giá đóng cửa của ngày mai có xu hướng tăng nhẹ". Đúng hay sai của quy tắc là vấn đề dữ liệu, nhưng ngôn ngữ của nó là ngôn ngữ mà nhà phân tích có thể thảo luận.

1.7 ANFIS như một cây cầu nối

Hệ *ANFIS* (*Adaptive Network-based Fuzzy Inference System*, hệ suy diễn mờ dựa trên mạng thích nghi) của Jang [6] là một cột mốc quan trọng. Ý tưởng là biểu diễn hệ suy diễn mờ Sugeno dưới dạng một mạng thích nghi có thể học bằng dữ liệu. Mỗi lớp của mạng ứng với một bước trong suy diễn mờ:

1. tính giá trị hàm thuộc cho từng đặc trưng;
2. kết hợp chúng thành *firing strengths* (độ kích hoạt luật) của từng luật;
3. chuẩn hóa độ kích hoạt;
4. tính phần hệ quả của từng luật;
5. lấy tổng có trọng số để ra đầu ra.

Điểm đẹp của ANFIS là cấu trúc này vừa giống mô hình có thể đọc được, vừa phù hợp với tính toán gradient. Phần hệ quả Sugeno bậc nhất còn cho phép sử dụng *least squares* (bình phương tối thiểu) khi phần tiền đề được cố định. Vì vậy, ANFIS không chỉ là "fuzzy logic + neural network" theo kiểu ghép khẩu hiệu, mà là một mô hình có gốc toán học cụ thể.

Tuy nhiên, ANFIS có một vấn đề lớn: số lượng luật tăng theo cấp số nhân. Nếu mỗi đặc trưng có m hàm thuộc và ta có d đặc trưng, số luật có thể là m^d. Đây là *rule explosion* (bùng nổ số luật). Trong thực tế, rất nhiều biến thể ANFIS được đề xuất để giảm vấn đề này: phân cụm, chia nhóm đặc trưng, hệ mờ phân cấp, hay cơ sở luật thưa. File `run_feature_group_anfis.py` mà ta phân tích ở cuối tài liệu cũng đi theo tinh thần đó: tách đặc trưng thành nhóm *returns* (tỷ suất thay đổi) và *indicators* (chỉ báo) để tránh nổ luật.

1.8 Chuỗi thời gian, dự báo và những cái bẫy đánh giá

Forecasting (dự báo) chuỗi thời gian đặt ra những ràng buộc đặc biệt. Với dữ liệu i.i.d., ta thường xáo trộn và chia tập huấn luyện/kiểm tra. Với dữ liệu theo thời gian, cách làm đó dễ gây *leakage*. Nếu thông tin từ tương lai rò vào quá khứ dù chỉ qua một bộ *scaler* (bộ chuẩn hóa) được fit trên toàn bộ dữ liệu, kết quả sẽ lạc quan giả tạo. Hyndman and Athanasopoulos [5] nhấn mạnh rằng đánh giá dự báo phải tôn trọng thứ tự thời gian, và tập thẩm định/tập kiểm tra phải được tách theo hướng tiến tới.

Trong tài chính, một cái bẫy khác là *metric* (chỉ số đánh giá). *MAPE* có thể không ổn định khi mẫu số gần 0. R^2 cao trên mức giá có thể đến chủ yếu từ xu hướng giá dài hạn. *Directional accuracy* (độ chính xác hướng) nghe có vẻ hợp lý nhưng phụ thuộc mạnh vào

định nghĩa "hướng". Huấn luyện trên tỷ suất sinh lợi nhưng đánh giá trên giá tuyệt đối có thể hợp lý nếu biến đổi nhất quán; nếu không, toàn bộ câu chuyện đánh giá có thể bị méo.

Chính vì vậy, phần cuối tài liệu sẽ không chỉ nói mô hình trong code "làm gì", mà còn hỏi nó "đánh giá cái gì, bằng cách nào, và có đang vô tình tự đánh lừa mình hay không".

1.9 Một số nguyên lý để giữ trên bàn làm việc

Nguyên lý thứ nhất: không có mô hình thần kỳ. Mỗi mô hình mạnh ở đâu đó và yếu ở đâu đó. Hồi quy tuyến tính đẹp vì nó rõ ràng. Cây quyết định đẹp vì nó phân mảnh dữ liệu theo logic dễ đọc. Mạng nơ-ron đẹp vì nó học biểu diễn phức tạp. Hệ mờ đẹp vì nó giữ ý nghĩa ngôn ngữ. ANFIS đẹp vì cố gắng kết hợp hai thế giới. Nhưng đẹp ở mặt này thường phải trả giá ở mặt khác.

Nguyên lý thứ hai: dữ liệu và giao thức đánh giá quan trọng hơn kiểu mô hình. Một mô hình bình thường nhưng giao thức đúng có giá trị hơn một mô hình phô trương nhưng rò rỉ thông tin nặng. Nếu không tin điều này, hãy đợi đến Chương 11: ta sẽ thấy một vài dòng code nhỏ có thể làm thay đổi hoàn toàn ý nghĩa của các *metric* báo cáo.

Nguyên lý thứ ba: khả năng diễn giải phải được phân tích theo chuỗi xử lý, không theo nhãn dán. Nếu một nhánh mờ sinh ra các "điểm số luật" rồi sau đó bị trộn với *BiLSTM* và *Dense layer* (lớp kết nối đầy đủ), thì phần mờ chỉ giải thích được một lát cắt của hệ thống. Phần còn lại vẫn là một ánh xạ học máy không minh bạch. Gọi cả mô hình là "explainable" (giải thích được) khi không nói rõ phạm vi giải thích là một cách nói quá tay.

Nguyên lý cuối cùng: học máy là môn của kỷ luật trí tuệ. Kỷ luật ở định nghĩa, kỷ luật ở tách dữ liệu, kỷ luật ở lập luận, và kỷ luật ở cách nghi ngờ kết quả quá đẹp. Người mới rất cần được nghe điều này sớm, vì quá nhiều hướng dẫn ngắn trên mạng vô tình dạy người ta cách chạy theo chỉ số đẹp trước khi học cách tin được nó.

1.10 Kết

Sau chương này, ta có thể tóm lại bố cục của cuốn bài giảng bằng một câu. Học máy cung cấp khung thống kê và tối ưu để học từ dữ liệu; học sâu đẩy khung đó đến mức biểu diễn sâu; logic mờ giữ lại khả năng lý giải bằng ngôn ngữ khái niệm; ANFIS tìm cách nối hai phía; dự báo chuỗi thời gian buộc tất cả phải đối mặt với kỷ luật đánh giá. Chính trên nền đó, các chương sau sẽ đi từ toán học căn bản đến phân tích chi tiết mô hình cụ thể.

Chuong 2

Nền tảng toán học: đại số tuyến tính, xác suất và tối ưu

2.1 Vì sao người học máy cần một nền toán vững

Rất nhiều khó khăn khi học học máy và học sâu không phát sinh từ sự "khó" của mô hình, mà từ việc bỏ qua nền toán học. Khi đọc một công thức, nếu ta không rõ một đại lượng là số vô hướng hay vectơ, một hàm đang ánh xạ từ đâu đến đâu, hay một kỳ vọng đang lấy theo phân phối nào, thì việc hiểu sẽ nhanh chóng chuyển thành học thuộc. Mục tiêu của chương này không phải dạy hết toán, mà là xây dựng một bộ công cụ đủ để tất cả các chương sau không bị đứt mạch.

2.2 Vectơ, ma trận, chuẩn và hình học

Ta bắt đầu với không gian Euclid $\mathbb{R}^d$. Một vectơ

$$\boldsymbol{x} = (x_1, \ldots, x_d)^\top \in \mathbb{R}^d$$

có thể được xem vừa là một đối tượng đại số, vừa là một điểm hình học, vừa là một bản ghi dữ liệu. Tích vô hướng

$$\langle \boldsymbol{x}, \boldsymbol{y} \rangle = \boldsymbol{x}^\top \boldsymbol{y} = \sum_{j=1}^{d} x_j y_j$$

đo mức độ thẳng hàng và cho phép ta định nghĩa độ dài

$$\|\boldsymbol{x}\|_2 = \sqrt{\langle \boldsymbol{x}, \boldsymbol{x} \rangle}.$$

Trong học máy, chuẩn không chỉ đo "kích thước". Nó còn là ngôn ngữ để điều chuẩn.

Chuẩn L^2 trừng phạt các vectơ có năng lượng lớn; chuẩn L^1 khuyến khích nhiều hệ số bằng 0; chuẩn vô cùng đo thành phần lớn nhất. Khác biệt này sẽ trở lại trong *ridge*, *lasso*, và trong cách ta đánh giá độ nhạy của hàm dự báo.

Định nghĩa 2.1. Cho $p \geq 1$, chuẩn L^p của vectơ $\boldsymbol{x} \in \mathbb{R}^d$ là

$$\|\boldsymbol{x}\|_p = \left(\sum_{j=1}^{d} |x_j|^p \right)^{1/p}.$$

Chuẩn L^∞ được định nghĩa bởi

$$\|\boldsymbol{x}\|_\infty = \max_j |x_j|.$$

Mệnh đề 2.2. *Với mọi $p \geq 1$, $\|\cdot\|_p$ thỏa ba tính chất của một chuẩn: tính dương xác định, đồng nhất thức, và bất đẳng thức tam giác.*

Proof. Hai tính chất đầu tiên theo trực tiếp từ định nghĩa. Tính chất thứ ba dựa trên bất đẳng thức Minkowski:

$$\left(\sum_j |x_j + y_j|^p \right)^{1/p} \leq \left(\sum_j |x_j|^p \right)^{1/p} + \left(\sum_j |y_j|^p \right)^{1/p}.$$

Bất đẳng thức này là hệ quả của bất đẳng thức Hölder. Trong phạm vi tài liệu này, điều quan trọng không phải kỹ thuật chi tiết, mà là ý nghĩa: phép cộng vectơ không làm độ dài tăng hơn tổng độ dài từng vectơ. □

Một ma trận $X \in \mathbb{R}^{n \times d}$ có thể được xem là tập hợp n điểm dữ liệu trong d chiều, mỗi hàng là một mẫu. Trong hồi quy tuyến tính, ta sẽ gặp biểu thức $X\boldsymbol{w}$ liên tục. Về hình học, vectơ $X\boldsymbol{w}$ là tổ hợp tuyến tính của các cột của X, nên thuộc không gian cột của X. Điểm nhìn hình học này sẽ giải thích vì sao nghiệm bình phương tối thiểu là hình chiếu trực giao của $\boldsymbol{y}$ lên không gian cột của X.

2.3 Trị riêng, vectơ riêng và giảm chiều

Giả sử A là ma trận vuông. Nếu tồn tại vectơ khác 0 sao cho

$$A\boldsymbol{v} = \lambda \boldsymbol{v},$$

thì $\boldsymbol{v}$ là vectơ riêng và λ là trị riêng. Khái niệm này cực kỳ quan trọng ở hai nơi. Thứ nhất, nó diễn tả các hướng mà biến đổi tuyến tính chỉ co giãn hoặc co lại, không xoay sang hướng

khác. Thứ hai, nó nằm ở trung tâm của *PCA* (phân tích thành phần chính) và nhiều phương pháp phân tích phổ.

Nếu A đối xứng thực, ta có một hệ vectơ riêng trực giao, và A có thể được chéo hóa trực giao. Điều này cho phép ta giảm nhiều bài toán ma trận về bài toán trên các trục độc lập. Trong PCA, ma trận hiệp phương sai mẫu

$$\widehat{\Sigma} = \frac{1}{n}\sum_{i=1}^{n}(\boldsymbol{x}_i - \bar{\boldsymbol{x}})(\boldsymbol{x}_i - \bar{\boldsymbol{x}})^\top$$

là đối xứng dương bán xác định, nên có cơ sở vectơ riêng trực giao. Hướng có trị riêng lớn nhất là hướng phương sai lớn nhất.

Định lý 2.3. *Thành phần chính thứ nhất của PCA là nghiệm của bài toán*

$$\max_{\|\boldsymbol{w}\|_2=1} \boldsymbol{w}^\top \widehat{\Sigma}\boldsymbol{w}.$$

Proof. Ta cần tối đa hóa phương sai của dữ liệu khi chiếu lên hướng đơn vị $\boldsymbol{w}$. Đại lượng đó là

$$\frac{1}{n}\sum_{i=1}^{n}\left(\boldsymbol{w}^\top(\boldsymbol{x}_i - \bar{\boldsymbol{x}})\right)^2 = \boldsymbol{w}^\top\widehat{\Sigma}\boldsymbol{w}.$$

Đặt hàm Lagrange

$$\mathcal{L}(\boldsymbol{w}, \lambda) = \boldsymbol{w}^\top\widehat{\Sigma}\boldsymbol{w} - \lambda(\boldsymbol{w}^\top\boldsymbol{w} - 1).$$

Lấy gradient theo $\boldsymbol{w}$ và cho bằng 0, ta được

$$2\widehat{\Sigma}\boldsymbol{w} - 2\lambda\boldsymbol{w} = 0 \quad \Longrightarrow \quad \widehat{\Sigma}\boldsymbol{w} = \lambda\boldsymbol{w}.$$

Nghiệm tối ưu phải là vectơ riêng, và giá trị mục tiêu bằng trị riêng tương ứng. Để tối đa hóa, ta chọn trị riêng lớn nhất. □

Chứng minh này là mẫu hình đẹp cho một nguyên lý quan trọng: rất nhiều bài toán học máy được giải bằng cách đổi ngôn ngữ "tối đa hóa đại lượng có ý nghĩa thống kê" sang ngôn ngữ "giải bài toán trị riêng/gradient/đối ngẫu".

2.4 Đạo hàm, gradient, Jacobian và Hessian

Nếu $f : \mathbb{R}^d \to \mathbb{R}$ là hàm vô hướng, gradient của nó là

$$\nabla f(\boldsymbol{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_d} \end{bmatrix}.$$

Gradient chỉ hướng tăng nhanh nhất cục bộ của hàm. Do đó, phương pháp *gradient descent* (hạ dốc theo gradient) cập nhật theo hướng âm của gradient:

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k - \eta_k \nabla f(\boldsymbol{x}_k).$$

Nếu $F : \mathbb{R}^d \to \mathbb{R}^m$, *Jacobian* là ma trận $m \times d$ gồm các đạo hàm riêng cấp một. Nếu $f : \mathbb{R}^d \to \mathbb{R}$, *Hessian* là ma trận đạo hàm cấp hai

$$H_f(\boldsymbol{x}) = \left[\frac{\partial^2 f}{\partial x_i \partial x_j}\right]_{i,j=1}^{d}.$$

Hessian mã hóa độ cong cục bộ của hàm. Nếu Hessian dương xác định trên mọi điểm, hàm là lồi địa phương; nếu trên mọi điểm của một miền lồi, hàm là lồi trên miền đó.

Định nghĩa 2.4. Một hàm f trên tập lồi C được gọi là lồi nếu với mọi $\boldsymbol{x}, \boldsymbol{y} \in C$ và mọi $\lambda \in [0, 1]$,

$$f(\lambda \boldsymbol{x} + (1-\lambda)\boldsymbol{y}) \le \lambda f(\boldsymbol{x}) + (1-\lambda) f(\boldsymbol{y}).$$

Tính lồi quan trọng vì nó làm đơn giản hóa bài toán tối ưu. Với hàm lồi, mọi cực tiểu địa phương đều là cực tiểu toàn cục. Trong học máy cổ điển, điều này dẫn đến các thuật toán có bảo đảm hội tụ rõ ràng. Trong học sâu, ta hiếm khi có xa xỉ như thế, nên kỹ năng thực hành và hiểu động học tối ưu trở nên quan trọng hơn.

Mệnh đề 2.5. *Nếu $f : \mathbb{R}^d \to \mathbb{R}$ khả vi hai lần và Hessian của nó dương bán xác định ở mọi điểm trong một tập lồi C, thì f là hàm lồi trên C.*

Proof. Lấy bất kỳ $\boldsymbol{x}, \boldsymbol{y} \in C$, đặt $\boldsymbol{d} = \boldsymbol{y} - \boldsymbol{x}$ và xét

$$g(t) = f(\boldsymbol{x} + t\boldsymbol{d}), \qquad t \in [0, 1].$$

Ta có

$$g''(t) = \boldsymbol{d}^\top H_f(\boldsymbol{x} + t\boldsymbol{d})\boldsymbol{d} \ge 0.$$

Do đó g là hàm lồi trên đoạn $[0, 1]$. Tính lồi một chiều của g cho

$$g(\lambda) \le \lambda g(1) + (1-\lambda) g(0),$$

tương đương với tính lồi của f trên đoạn nối $\boldsymbol{x}$ và $\boldsymbol{y}$. □

2.5 Quy tắc dây chuyền và lan truyền ngược được xây trên nó

Nếu $h(\boldsymbol{x}) = f(g(\boldsymbol{x}))$ với $g : \mathbb{R}^d \to \mathbb{R}^m$ và $f : \mathbb{R}^m \to \mathbb{R}$, thì

$$\nabla h(\boldsymbol{x}) = J_g(\boldsymbol{x})^\top \nabla f(g(\boldsymbol{x})).$$

Công thức đơn giản này là cột sống của *backpropagation* (lan truyền ngược). Trong một mạng nhiều lớp, hàm đầu ra là hợp thành của nhiều ánh xạ. Thay vì tính đạo hàm từ đầu mỗi lần, lan truyền ngược truyền ngược một "tín hiệu lỗi" qua từng lớp, tại mỗi lớp nhân thêm Jacobian cục bộ. Bản chất của nó không phải phép màu thuật, mà là quy tắc dây chuyền được tổ chức lại một cách tiết kiệm.

Cho một lớp affine

$$\boldsymbol{z} = W\boldsymbol{x} + \boldsymbol{b},$$

và một phi tuyến từng phần tử $\boldsymbol{a} = \phi(\boldsymbol{z})$. Nếu *loss* là L, ta có

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial \boldsymbol{z}} \boldsymbol{x}^\top, \qquad \frac{\partial L}{\partial \boldsymbol{b}} = \frac{\partial L}{\partial \boldsymbol{z}}, \qquad \frac{\partial L}{\partial \boldsymbol{x}} = W^\top \frac{\partial L}{\partial \boldsymbol{z}}.$$

Những công thức này sẽ được suy ra lại một cách có hệ thống ở chương về mạng nơ-ron.

2.6 Xác suất như ngôn ngữ mô tả bất định

Học máy hiện đại gần như không thể tách khỏi xác suất. Xác suất cung cấp ngôn ngữ mô tả sự bất định và tạo cầu nối giữa mô hình và dữ liệu. Một biến ngẫu nhiên X là một hàm đo được trên không gian mẫu. Trong thực hành, ta thường làm việc với phân phối của X qua các mật độ, xác suất có điều kiện, và kỳ vọng.

Kỳ vọng của một hàm $g(X)$ là

$$\mathbb{E}[g(X)] = \int g(x)\, p(x)\, dx$$

nếu X liên tục, hoặc là tổng nếu X rời rạc. Phương sai đo độ phân tán:

$$\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2].$$

Hiệp phương sai giữa hai biến X và Y là

$$\mathrm{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])].$$

Những đại lượng này không chỉ là ký hiệu. Chúng giúp ta phân tích tổng quát hóa. Ví dụ, *MSE* (sai số bình phương trung bình) có thể tách thành thành phần độ chệch và phương sai. Nhiều thuật toán học tham số bằng trung bình mẫu, và tính tốt của ước lượng phụ thuộc vào kỳ vọng và phương sai của nó.

2.7 Xác suất có điều kiện, Bayes và cực đại hóa hàm khả năng

Xác suất có điều kiện của Y khi biết $X = x$ là

$$p(y|x) = \frac{p(x, y)}{p(x)},$$

miễn là $p(x) > 0$. Công thức Bayes nói rằng

$$p(\theta|\mathcal{D}) = \frac{p(\mathcal{D}|\theta)p(\theta)}{p(\mathcal{D})}.$$

Nó là cầu nối giữa dữ liệu và tri thức tiên nghiệm.

Maximum likelihood (cực đại hóa hàm khả năng) chọn tham số làm cho dữ liệu quan sát "có khả năng xuất hiện cao nhất":

$$\hat{\theta}_{\text{MLE}} \in \arg\max_{\theta} p(\mathcal{D}|\theta).$$

Vì log là hàm tăng, ta thường tối đa hóa *log-likelihood*

$$\ell(\theta) = \sum_{i=1}^{n} \log p(y_i|\boldsymbol{x}_i, \theta).$$

Trong rất nhiều mô hình, tối thiểu hóa một *loss* phổ biến chính là tối đa hóa *likelihood* dưới một giả thiết nhiễu cụ thể.

Ví dụ 2.6. Nếu

$$y_i = \boldsymbol{w}^\top \boldsymbol{x}_i + b + \varepsilon_i, \qquad \varepsilon_i \overset{i.i.d.}{\sim} \mathcal{N}(0, \sigma^2),$$

thì

$$p(y_i|\boldsymbol{x}_i, \boldsymbol{w}, b) \propto \exp\left(-\frac{(y_i - \boldsymbol{w}^\top \boldsymbol{x}_i - b)^2}{2\sigma^2}\right).$$

Tối đa hóa *likelihood* tương đương với tối thiểu hóa tổng bình phương sai số. Vì vậy *least squares* (bình phương tối thiểu) có thể được xem là nghiệm cực đại khả năng trong mô hình nhiễu Gaussian.

2.8 Kỳ vọng có điều kiện và dự báo tối ưu theo MSE

Một kết quả cực kỳ quan trọng, nhưng thường bị đọc lướt, là: nếu *loss* là bình phương sai số, hàm dự báo tối ưu là kỳ vọng có điều kiện.

Định lý 2.7. *Xét bài toán tìm hàm f tối thiểu hóa*

$$\mathbb{E}\big[(Y - f(X))^2\big].$$

Khi đó mọi nghiệm tối ưu đều thỏa

$$f^\star(x) = \mathbb{E}[Y|X = x]$$

gần như khắp nơi.

Proof. Xét một hàm bất kỳ f. Ta viết

$$Y - f(X) = \big(Y - \mathbb{E}[Y|X]\big) + \big(\mathbb{E}[Y|X] - f(X)\big).$$

Bình phương và lấy kỳ vọng:

$$\begin{aligned}\mathbb{E}[(Y - f(X))^2] = \mathbb{E}[(Y - \mathbb{E}[Y|X])^2] \\ &+ 2\mathbb{E}\Big[(Y - \mathbb{E}[Y|X])(\mathbb{E}[Y|X] - f(X))\Big] \\ &+ \mathbb{E}[(\mathbb{E}[Y|X] - f(X))^2].\end{aligned}$$

Số hạng giữa bằng 0 vì

$$\mathbb{E}[Y - \mathbb{E}[Y|X] \mid X] = 0$$

và $\mathbb{E}[Y|X] - f(X)$ là hàm của X. Do đó

$$\mathbb{E}[(Y - f(X))^2] = \mathbb{E}[(Y - \mathbb{E}[Y|X])^2] + \mathbb{E}[(\mathbb{E}[Y|X] - f(X))^2].$$

Số hạng đầu không phụ thuộc vào f, số hạng sau luôn không âm và bằng 0 khi và chỉ khi $f(X) = \mathbb{E}[Y|X]$ gần như chắc chắn. □

Kết quả này rất sâu sắc. Nó nói rằng dự báo theo MSE không phải "đoán giá trị chân thực duy nhất", mà là ước lượng trung bình có điều kiện. Nếu phân phối có điều kiện lệch, đa đỉnh, hay có đuôi dày, giá trị tối ưu theo MSE vẫn chỉ là trung bình. Đây là lý do một *loss* phù hợp có ý nghĩa rất lớn.

2.9 Entropy, cross-entropy và độ lệch KL

Trong phân loại và mô hình xác suất, *entropy* (nhiễu loạn thông tin) đo mức độ bất định của một phân phối:

$$H(P) = -\sum_x P(x) \log P(x).$$

Cross-entropy (entropy chéo) giữa P và Q là

$$H(P, Q) = -\sum_x P(x) \log Q(x).$$

KL divergence (độ lệch Kullback–Leibler) là

$$\mathrm{KL}(P\|Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}.$$

Ta có hệ thức

$$H(P, Q) = H(P) + \mathrm{KL}(P\|Q).$$

Do $H(P)$ không phụ thuộc vào Q, tối thiểu hóa *cross-entropy* tương đương với tối thiểu hóa độ lệch KL. Trong học sâu cho phân loại, điều này giải thích vì sao entropy chéo là lựa chọn tự nhiên khi đầu ra của mô hình là một phân phối xấp xỉ.

Mệnh đề 2.8.

$$\mathrm{KL}(P\|Q) \geq 0$$

với dấu bằng khi và chỉ khi $P = Q$ gần như khắp nơi.

Proof. Đây là bất đẳng thức Gibbs. Từ tính lồi của hàm $-\log$, ta có

$$-\log t \geq 1 - t, \qquad t > 0.$$

Thay $t = Q(x)/P(x)$ và nhân với $P(x)$, tổng lại ta được

$$\sum_x P(x) \log \frac{P(x)}{Q(x)} \geq 0.$$

Dấu bằng xảy ra khi $Q(x)/P(x) = 1$ trên các điểm có $P(x) > 0$. □

2.10 Tối ưu lồi, Lagrange và điều kiện tối ưu

Nhiều bài toán học máy là bài toán tối ưu có ràng buộc. Phương pháp *Lagrange* đưa ràng buộc vào hàm mục tiêu bằng các bội số:

$$\mathcal{L}(\boldsymbol{x}, \lambda) = f(\boldsymbol{x}) + \lambda g(\boldsymbol{x})$$

nếu ràng buộc là $g(\boldsymbol{x}) = 0$. Điều kiện cần để tối ưu nói rằng gradient theo các biến bằng 0. Trong bài toán lồi, điều kiện *KKT* tổng quát hóa kỹ thuật này cho ràng buộc bất đẳng thức.

Tại sao người học máy cần điều này? Bởi vì rất nhiều bài toán cổ điển như *SVM* (máy vectơ hỗ trợ) được hiểu rõ nhất qua Lagrangian và đối ngẫu. Thậm chí, nhiều thuật toán hiện đại cũng ẩn chứa ý tưởng này: điều chuẩn là một cách thay thế bài toán có ràng buộc bằng bài toán không ràng buộc.

2.11 Hạ dốc theo gradient trên hàm bậc hai

Một bài toán mẫu để hiểu *gradient descent* là tối ưu hàm bậc hai

$$f(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^\top A\boldsymbol{x} - \boldsymbol{b}^\top \boldsymbol{x},$$

với A đối xứng dương xác định. Gradient là

$$\nabla f(\boldsymbol{x}) = A\boldsymbol{x} - \boldsymbol{b}.$$

Nghiệm duy nhất thỏa $A\boldsymbol{x}^\star = \boldsymbol{b}$. Nếu cập nhật

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k - \eta(A\boldsymbol{x}_k - \boldsymbol{b}),$$

và đặt sai số $\boldsymbol{e}_k = \boldsymbol{x}_k - \boldsymbol{x}^\star$, ta có

$$\boldsymbol{e}_{k+1} = (I - \eta A)\boldsymbol{e}_k.$$

Do đó hội tụ phụ thuộc vào phổ trị riêng của A. Nếu $0 < \eta < 2/\lambda_{\max}(A)$, phương pháp hội tụ. Đây là bài học quan trọng: tốc độ học không thể tách khỏi hình học của bài toán.

2.12 Đạo hàm ma trận mà ta sẽ cần nhiều lần

Trong thực hành, một số công thức đạo hàm ma trận xuất hiện liên tục:

$$\nabla_{\boldsymbol{w}}(\boldsymbol{a}^\top \boldsymbol{w}) = \boldsymbol{a}, \qquad \nabla_{\boldsymbol{w}}(\boldsymbol{w}^\top A \boldsymbol{w}) = (A + A^\top)\boldsymbol{w},$$

và nếu A đối xứng thì bằng $2A\boldsymbol{w}$. Nếu

$$f(\boldsymbol{w}) = \|X\boldsymbol{w} - \boldsymbol{y}\|_2^2,$$

thì

$$\nabla f(\boldsymbol{w}) = 2X^\top(X\boldsymbol{w} - \boldsymbol{y}).$$

Công thức cuối này là nền của hồi quy tuyến tính, *ridge*, và nhiều bài toán bình phương tối thiểu trong phần hệ quả mờ.

2.13 Nói về ổn định số học

Toán đúng chưa đủ; tính toán trên máy tính cần ổn định số học. Khi nhân nhiều xác suất nhỏ, dữ liệu có thể *underflow* (tụt dưới ngưỡng biểu diễn). Khi tính exp của số lớn, ta có thể *overflow* (tràn số). Khi ma trận gần kỳ dị, nghiệm bình phương tối thiểu sẽ không ổn định. Học sâu và ANFIS đều gặp những vấn đề này.

Ví dụ, trong logic mờ, độ kích hoạt của một luật thường là tích của nhiều độ thuộc. Nếu số đặc trưng lớn, tích này có thể rất nhỏ. Trong mã nguồn, người ta thường thêm ε để tránh chia cho 0 khi chuẩn hóa. Trong RNN, nhân lặp ma trận qua nhiều bước thời gian dễ dẫn đến triệt tiêu/nổ gradient. Trong bình phương tối thiểu, điều chuẩn kiểu ridge giúp giải quyết ma trận Gram kém điều kiện.

Người mới thường xem những chi tiết như "ép width dương", "thêm epsilon 1e-8", hay "gradient clipping" (cắt gradient) là tiểu tiết lập trình. Không phải. Chúng là dấu vết của một vấn đề toán học sâu xa: một biểu thức đúng về đại số chưa chắc đã ổn định trong tính toán số.

2.14 Mẫu số chuẩn cho các chương sau

Ta kết thúc chương này bằng một "mẫu số" tư duy sẽ xuất hiện nhiều lần.

1. Xác định đại lượng cần tối ưu và lý do tại sao nó có ý nghĩa.
2. Viết nó dưới dạng kỳ vọng lý tưởng và dạng xấp xỉ mẫu.

3. Chọn họ hàm hoặc tham số.
4. Tính gradient, nghiệm đóng, hay điều kiện tối ưu.
5. Kiểm tra tính ổn định số học và ý nghĩa thống kê.

Nếu một mô hình không thể được đọc bằng năm bước này, khả năng cao là ta chưa hiểu nó đủ sâu. Chương tiếp theo sẽ áp dụng mẫu số đó cho học máy thống kê.

Chuong 3

Học máy thống kê: từ rủi ro thực nghiệm đến điều chuẩn

3.1 Từ dữ liệu hữu hạn đến bài toán học

Ta đã biết bài toán lý tưởng là tối thiểu hóa rủi ro kỳ vọng

$$R(\theta) = \mathbb{E}[\ell(f_\theta(\boldsymbol{x}), y)].$$

Nhưng trong thực tế, phân phối sinh dữ liệu là ẩn. Do đó, ta thay nó bằng rủi ro thực nghiệm

$$\widehat{R}_n(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell(f_\theta(\boldsymbol{x}_i), y_i).$$

Nguyên lý *empirical risk minimization* (tối thiểu hóa rủi ro thực nghiệm) nói rằng ta chọn

$$\hat{\theta} \in \arg\min_{\theta\in\Theta} \widehat{R}_n(\theta).$$

Ban đầu, nguyên lý này có vẻ quá đơn giản. Nhưng hầu hết học máy hiện đại vẫn sống trong khung này, chỉ khác ở cách chọn không gian giả thuyết Θ, cách chọn hàm mất mát, và cách điều chuẩn.

Vấn đề cần cảnh báo ngay lập tức là: tối ưu hóa tập huấn luyện không đồng nghĩa với tổng quát hóa tốt. Nếu Θ quá rộng, ta có thể học thuộc dữ liệu. Nếu Θ quá hẹp, ta *underfit* (khớp thiếu). Toàn bộ vấn đề của học máy thống kê có thể được đọc như nghệ thuật điều hòa giữa hai cực này [1, 3].

3.2 Hồi quy tuyến tính và phương trình chuẩn

Xét mô hình

$$\hat{y}_i = \boldsymbol{w}^\top \boldsymbol{x}_i + b.$$

Gộp hệ số chặn vào vectơ đặc trưng bằng cách thêm cột 1, ta viết gọn

$$\hat{\boldsymbol{y}} = X\boldsymbol{w},$$

trong đó $X \in \mathbb{R}^{n\times d}$ là ma trận thiết kế. Hàm mất mát bình phương cho ta bài toán

$$\min_{\boldsymbol{w}} \|X\boldsymbol{w} - \boldsymbol{y}\|_2^2.$$

Gradient theo $\boldsymbol{w}$ là

$$\nabla_{\boldsymbol{w}} \|X\boldsymbol{w} - \boldsymbol{y}\|_2^2 = 2X^\top(X\boldsymbol{w} - \boldsymbol{y}).$$

Cho bằng 0, ta nhận được *normal equation* (phương trình chuẩn)

$$X^\top X\boldsymbol{w} = X^\top \boldsymbol{y}.$$

Nếu $X^\top X$ khả nghịch, nghiệm duy nhất là

$$\hat{\boldsymbol{w}} = (X^\top X)^{-1} X^\top \boldsymbol{y}.$$

Nhận xét 3.1. Công thức trên đẹp nhưng nguy hiểm nếu dùng máy móc. Nếu các cột của X gần phụ thuộc tuyến tính, $X^\top X$ sẽ kém điều kiện. Trong thực hành, ta ưu tiên giải bình phương tối thiểu bằng QR, SVD, hoặc thêm điều chuẩn.

3.2.1 Góc nhìn hình học

Nghiệm bình phương tối thiểu là hình chiếu trực giao của $\boldsymbol{y}$ lên không gian cột của X. Sai số dư

$$\boldsymbol{r} = \boldsymbol{y} - X\hat{\boldsymbol{w}}$$

trực giao với mọi cột của X, vì

$$X^\top \boldsymbol{r} = 0.$$

Do đó, bình phương tối thiểu không chỉ là kết quả của một phép lấy đạo hàm. Nó là kết quả của một phép chiếu hình học.

3.3 Đánh đổi độ chệch – phương sai

Giả sử dữ liệu được sinh bởi

$$y = f^\star(\boldsymbol{x}) + \varepsilon, \qquad \mathbb{E}[\varepsilon|\boldsymbol{x}] = 0, \quad \mathrm{Var}(\varepsilon|\boldsymbol{x}) = \sigma^2.$$

Xét một ước lượng $\hat{f}$ được học từ dữ liệu mẫu. Lỗi dự báo tại điểm $\boldsymbol{x}$ có thể tách thành

$$\mathbb{E}\big[(\hat{f}(\boldsymbol{x}) - f^\star(\boldsymbol{x}))^2\big] = \big(\mathbb{E}[\hat{f}(\boldsymbol{x})] - f^\star(\boldsymbol{x})\big)^2 + \mathrm{Var}(\hat{f}(\boldsymbol{x})).$$

Nếu tính cả nhiễu không thể loại bỏ được, ta cộng thêm σ^2. Độ chệch cao nghĩa là mô hình quá cứng, không đủ sức biểu diễn. Phương sai cao nghĩa là mô hình quá nhạy với mẫu huấn luyện. Điều chuẩn và chọn độ phức tạp mô hình thực chất là điều khiển sự cân bằng này.

3.4 Hồi quy ridge: điều chuẩn như một tiên nghiệm Gaussian

Hồi quy *ridge* giải bài toán

$$\min_{\boldsymbol{w}} \|X\boldsymbol{w} - \boldsymbol{y}\|_2^2 + \lambda\|\boldsymbol{w}\|_2^2.$$

Lấy gradient và cho bằng 0:

$$2X^\top(X\boldsymbol{w} - \boldsymbol{y}) + 2\lambda\boldsymbol{w} = 0,$$

nên

$$(X^\top X + \lambda I)\boldsymbol{w} = X^\top\boldsymbol{y}.$$

Vì $X^\top X + \lambda I$ dương xác định khi $\lambda > 0$, *ridge* có nghiệm duy nhất

$$\hat{\boldsymbol{w}}_{\text{ridge}} = (X^\top X + \lambda I)^{-1}X^\top\boldsymbol{y}.$$

Mệnh đề 3.2. *Hồi quy* ridge *tương đương với ước lượng cực đại hậu nghiệm trong mô hình nhiễu Gaussian và tiên nghiệm Gaussian*

$$\boldsymbol{w} \sim \mathcal{N}(\boldsymbol{0}, \tau^2 I).$$

Proof. Log hậu nghiệm, bỏ qua hằng số, bằng

$$-\frac{1}{2\sigma^2}\|X\boldsymbol{w} - \boldsymbol{y}\|_2^2 - \frac{1}{2\tau^2}\|\boldsymbol{w}\|_2^2.$$

Tối đa hóa hậu nghiệm tương đương với tối thiểu hóa

$$\|X\boldsymbol{w} - \boldsymbol{y}\|_2^2 + \frac{\sigma^2}{\tau^2}\|\boldsymbol{w}\|_2^2.$$

Đặt $\lambda = \sigma^2/\tau^2$, ta thu được *ridge*. □

Ý nghĩa này rất đẹp: điều chuẩn không chỉ là "mẹo tránh quá khớp", mà là cách mã hóa tri thức tiên nghiệm rằng hệ số lớn là ít đáng tin.

3.5 Lasso và tính thưa

Lasso giải bài toán

$$\min_{\boldsymbol{w}} \frac{1}{2}\|X\boldsymbol{w} - \boldsymbol{y}\|_2^2 + \lambda\|\boldsymbol{w}\|_1.$$

Khác với *ridge*, chuẩn L^1 không khả vi tại 0. Chính sự không trơn này làm xuất hiện tính *sparsity* (thưa): nhiều hệ số có thể bằng 0 đúng nghĩa. Về hình học, các đường đồng mức của $\|\boldsymbol{w}\|_1$ có góc nhọn, nên điểm tiếp xúc với đường đồng mức của hàm mất mát dễ rơi vào trục tọa độ.

Lasso quan trọng không chỉ vì giảm số chiều, mà còn vì dạy một bài học tổng quát: hình học của bộ điều chuẩn ảnh hưởng trực tiếp đến cấu trúc nghiệm. Trong hệ mờ, nếu ta điều chuẩn số lượng luật hay mức chồng lấp của hàm thuộc, ta cũng đang can thiệp vào hình học của bài toán học một cách tương tự.

3.6 Hồi quy logistic và cực đại hóa hàm khả năng phân loại

Cho bài toán phân loại nhị phân $y_i \in \{0, 1\}$, hồi quy logistic đặt

$$\mathbb{P}(y = 1|\boldsymbol{x}) = \sigma(\boldsymbol{w}^\top \boldsymbol{x} + b), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Hàm khả năng là

$$\prod_{i=1}^{n} \sigma(z_i)^{y_i}(1 - \sigma(z_i))^{1-y_i}.$$

Lấy log, ta được

$$\ell(\boldsymbol{w}, b) = \sum_{i=1}^{n} \left(y_i z_i - \log(1 + e^{z_i})\right), \qquad z_i = \boldsymbol{w}^\top \boldsymbol{x}_i + b.$$

Hàm mất mát là âm log-likelihood, chính là *binary cross-entropy* (entropy chéo nhị phân). Gradient theo $\boldsymbol{w}$:

$$\nabla_{\boldsymbol{w}}(-\ell) = \sum_{i=1}^{n}(\sigma(z_i) - y_i)\boldsymbol{x}_i = X^\top(\hat{\boldsymbol{p}} - \boldsymbol{y}).$$

Mệnh đề 3.3. *Âm log-likelihood của hồi quy logistic là một hàm lồi theo* $(\boldsymbol{w}, b)$.

Proof. Hàm $\phi(z) = \log(1 + e^z)$ có đạo hàm cấp hai

$$\phi''(z) = \sigma(z)(1 - \sigma(z)) \geq 0,$$

nên ϕ lồi. Vì

$$-(yz - \log(1 + e^z)) = \phi(z) - yz$$

là tổng của một hàm lồi và một hàm affine, nó lồi theo z. Do z là hàm affine của $(\boldsymbol{w}, b)$, âm log-likelihood là lồi theo $(\boldsymbol{w}, b)$. □

Kết quả này cho ta một mô hình phân loại có bài toán học dễ chịu. Khác với *perceptron* có thể có vô số nghiệm hoặc không có nếu dữ liệu không tách được, hồi quy logistic vẫn hoạt động như một mô hình xác suất mềm.

3.7 Mô hình hóa xác suất và hàm mất mát

Có một bài học lớn ở đây: hàm mất mát không nên chọn theo thói quen, mà theo giả thiết về dữ liệu.

Nếu ta cho rằng nhiễu có điều kiện là Gaussian, MSE là hợp lý. Nếu dữ liệu là biến đếm hiếm, *Poisson loss* (mất mát Poisson) có thể hợp lý hơn. Nếu bài toán là phân loại nhiều lớp, entropy chéo là tự nhiên khi đầu ra là phân phối *categorical* (phân phối hạng mục). Trong dự báo tài chính, dự báo trung bình có điều kiện không phải lúc nào cũng đủ; ta có thể quan tâm đến phân vị, tổn thất kỳ vọng ở đuôi, hay toàn bộ phân phối.

3.8 Mô hình, tập thẩm định, tập kiểm tra và giao thức chọn siêu tham số

Một hệ thống học máy nghiêm túc cần phân biệt rõ ba vai trò:

1. **Train** (tập huấn luyện): học tham số của mô hình.
2. **Validation** (tập thẩm định): chọn siêu tham số, *early stopping* (dừng sớm), và *model selection* (chọn mô hình).

3. **Test** (tập kiểm tra): đánh giá cuối cùng, chỉ dùng một cách tiết kiệm.

Nếu ta dùng tập kiểm tra để chọn số *epoch*, chọn *seed*, chọn kiểu mô hình, hay chọn ngưỡng, ta đã dùng *test* như *validation*. Khi đó, *test* không còn là *test* nữa. Lỗi này rất phổ biến vì code vẫn "chạy" và kết quả vẫn đẹp. Nhưng về mặt thống kê, nó làm sai lệch ước lượng hiệu năng tổng quát hóa.

Trong dữ liệu chuỗi thời gian, tác hại còn nặng hơn. Tập thẩm định phải nằm sau tập huấn luyện theo thời gian, và tập kiểm tra phải nằm sau tập thẩm định. Bất kỳ biến đổi nào được fit trên toàn bộ dữ liệu trước khi tách một cách hợp lệ đều có thể gây *leakage*. Điều này bao gồm chuẩn hóa, PCA, chọn đặc trưng, và thậm chí cả phân cụm để khởi tạo hàm thuộc.

3.9 Kiểm định chéo và giới hạn của nó trong chuỗi thời gian

Với dữ liệu i.i.d., *k-fold cross-validation* (kiểm định chéo k phần) là công cụ mạnh. Dữ liệu được chia thành k phần, lần lượt giữ một phần để kiểm tra và huấn luyện trên phần còn lại. Nhưng trong chuỗi thời gian, xáo trộn phá vỡ thứ tự thông tin. Vì vậy ta cần các biến thể như *rolling-origin evaluation* (đánh giá theo gốc cuộn) hay *expanding window* (cửa sổ mở rộng) [5].

Nguyên lý cơ bản là: mọi dự báo tại thời điểm t chỉ được sử dụng thông tin có sẵn trước hoặc tại t. Nếu phương pháp thẩm định không tôn trọng nguyên lý này, kết quả sẽ lạc quan.

3.10 Chỉ số nào cho bài toán nào

Không có một *metric* duy nhất tối ưu cho mọi tình huống. *RMSE* phạt nặng lỗi lớn. *MAE* ổn định hơn trước ngoại lệ. *MAPE* dễ diễn giải theo tỷ lệ nhưng vô nghĩa khi mẫu số nhỏ. R^2 đo mức cải thiện so với dự báo bằng trung bình, nhưng trong chuỗi có xu hướng nó có thể "đẹp" một cách dễ dàng. *Directional accuracy* nghe đúng tinh chất giao dịch, nhưng nó phụ thuộc mạnh vào cách định nghĩa hướng và có thể bỏ qua quy mô lỗi.

Mục tiêu của ta là phải nói rõ *metric* đang đo điều gì. Trong dự báo giá, nếu quyết định giao dịch phụ thuộc vào hướng tăng/giảm và rủi ro cực trị, chỉ số được chọn cần phản ánh được điều đó. Nếu ta dự báo 4 biến OHLC cùng lúc, chỉ số cho từng biến riêng lẻ có thể bỏ qua cấu trúc hợp lý giữa chúng.

3.11 Nguồn gốc của rò rỉ thông tin

Rò rỉ thông tin không chỉ là việc vô tình chèn cột "future return" (tỷ suất tương lai) vào đặc trưng. Nhiều dạng rò rỉ tinh vi hơn:

1. fit bộ chuẩn hóa trên cả tập huấn luyện lẫn tập kiểm tra;
2. fit PCA trên cả bộ dữ liệu;
3. chọn *seed* tốt nhất dựa trên tập kiểm tra;
4. dùng tập kiểm tra cho *early stopping*;
5. tính đặc trưng ở thời điểm t bằng cách dùng thông tin xuất hiện sau t;
6. làm sạch dữ liệu dựa trên thống kê của cả tập dữ liệu.

Ta cần coi *leakage* như một lỗi logic, không chỉ là lỗi kỹ thuật. Nó làm thói quen lập luận về tổng quát hóa bị hư hỏng. Trong Chương 11, chúng ta sẽ gặp đồng thời nhiều dạng rò rỉ trong cùng một script, và thấy rõ vì sao con số đẹp trở thành không đáng tin.

3.12 Những gì cần giữ lại trước khi sang các mô hình phi tuyến

Đến đây, có ba thông điệp cần nắm chắc.

Thứ nhất, học máy thống kê là nghệ thuật xây dựng và kiểm soát bài toán tối ưu từ dữ liệu hữu hạn.

Thứ hai, điều chuẩn là trung tâm của tổng quát hóa, và có thể được hiểu vừa về mặt tối ưu vừa về mặt Bayes.

Thứ ba, giao thức đánh giá là một phần của mô hình. Nếu giao thức sai, kết quả đẹp cũng vô nghĩa.

Với ba thông điệp đó, ta đã sẵn sàng nhìn các mô hình phi tuyến cổ điển nhưng vẫn giữ được kỷ luật thống kê.

Chuong 4

Các mô hình cổ điển phi tuyến: cây, hạt nhân, tổ hợp và phân cụm

4.1 Vì sao vẫn phải học các mô hình "cổ điển"

Người mới thường nóng lòng đi ngay vào học sâu và xem những mô hình cổ điển là câu chuyện lịch sử. Đây là một sai lầm lớn. Trước khi học một mô hình rất sâu, ta cần biết các cách tạo phi tuyến khác đã tồn tại như thế nào, mạnh ở đâu, và dạy ta điều gì về bài toán. Các mô hình cổ điển thường dễ chẩn đoán dữ liệu hơn, dễ *debug* hơn, và trong nhiều bài toán có kích thước vừa phải, hiệu năng của chúng rất cạnh tranh.

Chủ đề của chương này là: phi tuyến không đồng nghĩa với sâu. Ta có nhiều cách tạo phi tuyến mà không cần mạng nơ-ron sâu. Những cách này còn giúp ta hiểu tốt hơn lý do học sâu thành công.

4.2 *k*-NN: phi tham số theo nghĩa địa phương

Với *k-nearest neighbors* (*k*-láng giềng gần nhất), dự báo tại điểm mới $\boldsymbol{x}$ được xây dựng từ k điểm huấn luyện gần nhất theo một *metric* (thước đo khoảng cách), thường là khoảng cách Euclid. Trong hồi quy, ta lấy trung bình nhãn của các láng giềng. Trong phân loại, ta lấy đa số phiếu.

$$\hat{f}(\boldsymbol{x}) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\boldsymbol{x})} y_i.$$

k-NN đẹp vì nó cực kỳ đơn giản và gần như không "học" tham số toàn cục. Nhưng chính vì thế, nó gặp lời nguyền số chiều: khi số chiều tăng, khái niệm "gần" trở nên kém ý nghĩa. Bài học ở đây là: có được một *metric* tốt là có được nửa mô hình.

4.3 Cây quyết định: phân hoạch không gian bằng luật nếu–thì

Cây quyết định phân hoạch không gian đầu vào bằng các ngưỡng trên từng đặc trưng. Mỗi nút nội bộ chọn một đặc trưng j và một ngưỡng s, rồi tách dữ liệu thành hai tập:

$$\{x_j \le s\}, \qquad \{x_j > s\}.$$

Trong hồi quy, mỗi lá thường chứa trung bình nhãn của các mẫu rơi vào lá đó. Trong phân loại, mỗi lá chứa một phân phối lớp kinh nghiệm.

Ưu điểm của cây là dễ đọc và có thể biểu diễn tương tác phi tuyến mà không cần biến đổi đặc trưng thủ công. Nhược điểm là dễ quá khớp và không ổn định: thay đổi nhỏ trong dữ liệu có thể dẫn đến cây khác. Ta đang mua tính dễ đọc và tính phi tuyến bằng cái giá của sự không ổn định.

4.3.1 Tiêu chuẩn tách

Trong phân loại, người ta dùng *entropy* hoặc *Gini impurity* (độ hỗn tạp Gini). Ví dụ, nếu nút có tỉ lệ lớp $p_1, \dots, p_K$, entropy là

$$H = -\sum_{k=1}^{K} p_k \log p_k.$$

Tách tốt là tách làm giảm độ hỗn tạp nhiều nhất. Trong hồi quy, ta dùng tổng bình phương sai số trong hai nút con. Cách chọn này cho thấy cây vẫn nằm trong logic tối thiểu hóa rủi ro thực nghiệm, chỉ là cách tham số hóa khác.

4.4 *Bagging* và rừng ngẫu nhiên

Random forest (rừng ngẫu nhiên) là một bài học đẹp về phương sai. Một cây đơn lẻ có độ chệch thấp nhưng phương sai cao. Nếu ta huấn luyện nhiều cây trên các *bootstrap samples* (mẫu rút lại có hoàn lại) và trung bình kết quả, phương sai sẽ giảm. Rừng ngẫu nhiên đi xa hơn *bagging* bằng cách chỉ cho phép mỗi nút xem một tập con ngẫu nhiên của đặc trưng, qua đó giảm tương quan giữa các cây.

Nếu $T_1, \dots, T_M$ là các ước lượng có cùng phương sai σ^2 và tương quan trung bình ρ, thì

phương sai của trung bình xấp xỉ

$$\mathrm{Var}\left(\frac{1}{M}\sum_{m=1}^{M} T_m\right) \approx \rho\sigma^2 + \frac{1-\rho}{M}\sigma^2.$$

Công thức này nhắc ta rằng không chỉ cần nhiều mô hình, mà cần nhiều mô hình đủ khác nhau.

4.5 *Boosting*: học lặp để sửa sai

Khác với *bagging*, *boosting* không độc lập giữa các *base learner* (mô hình cơ sở). Nó thêm mô hình mới để sửa lỗi của mô hình cũ. Trong *gradient boosting* cho hồi quy, ta xây dựng mô hình cộng

$$F_M(\boldsymbol{x}) = \sum_{m=1}^{M} \nu h_m(\boldsymbol{x}),$$

trong đó h_m xấp xỉ gradient âm của hàm mất mát tại bước $m-1$. Ý tưởng này cực kỳ tổng quát: *boosting* có thể được xem như hạ dốc theo gradient trong không gian hàm.

Điều đẹp ở đây là ta thấy một lần nữa tính thống nhất của học máy. "Thêm một cây nữa" hóa ra là một bước theo gradient, nhưng gradient này sống trong không gian hàm thay vì không gian Euclid thông thường.

4.6 Máy vectơ hỗ trợ và biên

Cho bài toán phân loại nhị phân tách tuyến tính, *SVM* (máy vectơ hỗ trợ) tìm siêu phẳng

$$\boldsymbol{w}^\top \boldsymbol{x} + b = 0$$

có *margin* (biên) lớn nhất. Bài toán *hard-margin* là

$$\min_{\boldsymbol{w},b} \frac{1}{2}\|\boldsymbol{w}\|_2^2 \quad \text{s.t.} \quad y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) \geq 1,\ \forall i.$$

Biên hình học bằng $2/\|\boldsymbol{w}\|_2$. Do đó tối thiểu hóa chuẩn của $\boldsymbol{w}$ là tối đa hóa biên.

Khi dữ liệu không tách được, ta thêm *slack variables* (biến trễ) và *hinge loss*:

$$\min_{\boldsymbol{w},b} \frac{1}{2}\|\boldsymbol{w}\|_2^2 + C\sum_{i=1}^{n} \max(0, 1 - y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b)).$$

SVM dạy một bài học lớn: điều chuẩn và hàm mất mát không tách rời, mà được thiết kế cùng

nhau để phản ánh hình học của bài toán.

4.7 Mẹo hạt nhân: phi tuyến nhưng vẫn lồi

Nếu dữ liệu không tách được trong không gian gốc, ta có thể ánh xạ nó vào một không gian đặc trưng cao chiều $\phi(\boldsymbol{x})$ rồi xây bộ phân loại tuyến tính ở đó. *Kernel trick* (mẹo hạt nhân) cho phép tính tích vô hướng

$$K(\boldsymbol{x}, \boldsymbol{x}') = \langle \phi(\boldsymbol{x}), \phi(\boldsymbol{x}') \rangle$$

mà không cần biết rõ ϕ. Những hàm hạt nhân phổ biến gồm đa thức, Gaussian RBF, và sigmoid.

Về mặt khái niệm, phương pháp hạt nhân là một cách "ẩn" việc mở rộng đặc trưng trong hình học của tích vô hướng. Học sâu, trái lại, học ϕ từ dữ liệu. Sự khác biệt này sẽ giúp ta hiểu vì sao học sâu được xem là học biểu diễn, còn phương pháp hạt nhân là biểu diễn được chỉ định gián tiếp qua hạt nhân.

4.8 Không gian Hilbert tái sinh và định lý biểu diễn

Một trong những kết quả đẹp nhất của phương pháp hạt nhân là *representer theorem* (định lý biểu diễn): nghiệm tối ưu của một bài toán rủi ro thực nghiệm có điều chuẩn trong *RKHS* (không gian Hilbert tái sinh) nằm trong bao tuyến tính của các hàm hạt nhân đặt tại điểm dữ liệu:

$$f^\star(\cdot) = \sum_{i=1}^{n} \alpha_i K(\boldsymbol{x}_i, \cdot).$$

Kết quả này biến bài toán tối ưu vô hạn chiều thành bài toán hữu hạn chiều theo số mẫu. Bài học sâu xa là: cấu trúc toán học đúng có thể biến bài toán tưởng như không thể giải thành bài toán khả thi.

4.9 Phân cụm và K-Means

Clustering (phân cụm) là phần lớn của học không giám sát. Trong các phương pháp phân cụm, *K-Means* đơn giản nhưng có ảnh hưởng cực lớn. Bài toán là chia dữ liệu thành K cụm và tìm tâm cụm $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K$ để tối thiểu hóa tổng bình phương khoảng cách trong cụm:

$$\min_{\{\boldsymbol{\mu}_k\}, \{c_i\}} \sum_{i=1}^{n} \|\boldsymbol{x}_i - \boldsymbol{\mu}_{c_i}\|_2^2.$$

Thuật toán Lloyd lặp giữa hai bước:

1. gán mỗi điểm vào tâm gần nhất;
2. cập nhật mỗi tâm bằng trung bình các điểm trong cụm.

Đây là thuật toán giảm xen kẽ, bảo đảm làm giảm mục tiêu qua mỗi bước, nhưng không bảo đảm tối ưu toàn cục. Khởi tạo quan trọng. Trong script mà ta phân tích sau này, *K-Means* được dùng để khởi tạo tâm của các hàm thuộc cho ANFIS. Điều đó có lý, nhưng cũng đặt câu hỏi: phân cụm đang được fit trên phần dữ liệu nào? Và ý nghĩa "LOW/HIGH" có còn đúng sau khi huấn luyện hay không?

4.10 Hỗn hợp Gaussian và EM

K-Means có thể xem là một giới hạn của *Gaussian mixture model* (mô hình hỗn hợp Gaussian) khi hiệp phương sai là dạng cầu, bằng nhau, và phân công cụm trở nên cứng. GMM mô tả dữ liệu bằng tổng có trọng số của các Gaussian:

$$p(\boldsymbol{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \Sigma_k).$$

Thuật toán *EM* (kỳ vọng – cực đại) lặp giữa:

1. **E-step**: tính *responsibilities* (trách nhiệm mềm) r_{ik};
2. **M-step**: tối đa hóa kỳ vọng log-likelihood đầy đủ.

EM cho ta một nguyên lý bao quát: khi có biến ẩn, ta có thể lặp giữa suy diễn trên biến ẩn và cập nhật tham số. Tư duy này có điểm đồng với học lai của ANFIS, nơi một phần tham số có thể học điều kiện theo phần khác.

4.11 PCA và giảm chiều trong quy trình học máy

PCA không chỉ là một bài tập đại số. Trong quy trình học máy, PCA giúp giảm nhiễu, nén thông tin, và trích xuất hướng biến động chính. Nhưng PCA là một biến đổi học từ dữ liệu. Nếu fit PCA trên toàn bộ dữ liệu rồi mới tách huấn luyện/kiểm tra, ta đã gây rò rỉ thông tin. Bài học này giống hệt với chuẩn hóa và phân cụm.

Dưới góc nhìn ANFIS, hàm thuộc cũng là một dạng biểu diễn. Khởi tạo và học hàm thuộc có thể xem như một dạng giảm chiều phi tuyến cục bộ ở từng đặc trưng. Nếu không tôn trọng kỷ luật huấn luyện/thẩm định/kiểm tra cho các bước này, ta không còn đánh giá mô hình một cách sạch sẽ.

4.12 Mô hình cổ điển dạy ta điều gì về diễn giải

Cây quyết định dễ đọc nhất ở cấp quy tắc, nhưng không ổn định. Rừng ngẫu nhiên ổn định hơn nhưng khó đọc hơn vì là tập hợp nhiều cây. SVM hạt nhân có thể đạt biên quyết định rất đẹp nhưng khó giải thích với người dùng cuối. *K-Means* cho tâm cụm để nhìn, nhưng nhãn cụm có thể thay đổi theo khởi tạo. Bài học ở đây là: diễn giải luôn có giá của nó.

ANFIS được ưa chuộng một phần vì hứa hẹn đạt được một điểm cân bằng đẹp: luật đọc được, tham số học được. Nhưng ta cần giữ tinh thần cảnh giác từ các mô hình cổ điển: chỉ cần thêm một lớp kết hợp phức tạp ở cuối, tính diễn giải có thể mất rất nhanh.

4.13 Nội dung của chương này sẽ trở lại ở đâu

Khi sang học sâu, ta sẽ gặp lại hạ dốc theo gradient, điều chuẩn, và xấp xỉ hàm, nhưng trên lớp hàm phong phú hơn. Khi sang logic mờ, ta sẽ gặp lại luật nếu–thì, nhưng được làm mềm bằng hàm thuộc. Khi sang ANFIS, ta sẽ gặp lại phân cụm, bình phương tối thiểu, và gradient trong cùng một cấu trúc. Các mô hình “cổ điển” vì vậy không hề bị bỏ lại; chúng sẽ trở thành các mảnh ghép trong một bức tranh lớn hơn.

Chuong 5

Mạng nơ-ron: từ perceptron đến lan truyền ngược

5.1 Perceptron và giới hạn của tuyến tính

Perceptron có thể được xem là mô hình phân loại tuyến tính có kích hoạt ngưỡng:

$$\hat{y} = \text{sign}(\boldsymbol{w}^\top \boldsymbol{x} + b).$$

Nếu dữ liệu tách tuyến tính, thuật toán perceptron có thể tìm một siêu phẳng tách. Nhưng khi quan hệ giữa đầu vào và đầu ra không tuyến tính, perceptron đơn lẻ thất bại. Bài toán XOR là ví dụ kinh điển.

Điều này dẫn đến câu hỏi tự nhiên: nếu một lớp tuyến tính không đủ, ta xếp nhiều lớp lại có đủ không? Câu trả lời là có, miễn là giữa các lớp ta đặt một ánh xạ phi tuyến. Chính từ đây, *multi-layer perceptron* (mạng nhiều lớp) ra đời.

5.2 Một lớp ẩn đã làm thay đổi tất cả

Xét một mạng một lớp ẩn:

$$\boldsymbol{h} = \phi(W_1 \boldsymbol{x} + \boldsymbol{b}_1), \qquad \hat{\boldsymbol{y}} = W_2 \boldsymbol{h} + \boldsymbol{b}_2.$$

Nếu ϕ là affine, toàn bộ mạng vẫn chỉ là affine. Vì vậy, phi tuyến là bắt buộc. Các hàm kích hoạt phổ biến gồm *sigmoid*, tanh, và *ReLU* (hàm tuyến tính chỉnh lưu). Mỗi hàm có ưu và nhược điểm. *Sigmoid* và tanh trơn và dễ liên hệ tới xác suất/chuẩn hóa, nhưng dễ bão hòa. *ReLU* đơn giản, gradient ổn ở miền dương, nhưng có thể "chết" ở miền âm.

5.2.1 Định lý xấp xỉ phổ quát

Một kết quả có sức cổ vũ lớn cho mạng nơ-ron là *universal approximation theorem* (định lý xấp xỉ phổ quát): với số nơ-ron đủ lớn và phi tuyến phù hợp, mạng một lớp ẩn có thể xấp xỉ bất kỳ hàm liên tục nào trên tập compact đến độ chính xác tùy ý. Điều này không nói rằng học là dễ, không nói rằng cần ít nơ-ron, và không nói rằng tổng quát hóa sẽ tốt. Nó chỉ nói lớp hàm của MLP là rất phong phú.

Vì vậy, sức mạnh của mạng nơ-ron không đến từ một công thức thần bí, mà đến từ việc ta được quyền xây dựng các biểu diễn phi tuyến rất giàu.

5.3 Hàm kích hoạt và hình học của gradient

Hãy nhìn vào đạo hàm:

$$\text{sigmoid}'(z) = \text{sigmoid}(z)(1-\text{sigmoid}(z)), \qquad \tanh'(z) = 1-\tanh^2(z), \qquad \text{ReLU}'(z) = \mathbf{1}_{z>0}.$$

Với *sigmoid* và tanh, khi $|z|$ lớn thì đạo hàm gần 0. Trong mạng nhiều lớp, nhân lặp nhiều đạo hàm nhỏ dễ dẫn đến *vanishing gradient* (triệt tiêu gradient). *ReLU* giảm bớt vấn đề này trên miền dương, nhưng đổi lại không khả vi tại 0 và có miền âm vô hoạt.

Lựa chọn hàm kích hoạt vì vậy không phải vấn đề thẩm mỹ. Nó thay đổi động học học. Nhiều quy tắc thực hành trong học sâu thực ra là cách khống chế động học gradient.

5.4 Suy dẫn lan truyền ngược từ quy tắc dây chuyền

Ta xét mạng một lớp ẩn cho bài toán hồi quy, với hàm mất mát trên một mẫu:

$$L = \frac{1}{2}\|\hat{\boldsymbol{y}} - \boldsymbol{y}\|_2^2, \qquad \hat{\boldsymbol{y}} = W_2\boldsymbol{h} + \boldsymbol{b}_2, \qquad \boldsymbol{h} = \phi(\boldsymbol{z}_1), \qquad \boldsymbol{z}_1 = W_1\boldsymbol{x} + \boldsymbol{b}_1.$$

Đặt

$$\boldsymbol{\delta}_2 = \frac{\partial L}{\partial \boldsymbol{z}_2} = \hat{\boldsymbol{y}} - \boldsymbol{y},$$

vì $\boldsymbol{z}_2 = \hat{\boldsymbol{y}}$ trong lớp cuối affine. Khi đó

$$\frac{\partial L}{\partial W_2} = \boldsymbol{\delta}_2\boldsymbol{h}^\top, \qquad \frac{\partial L}{\partial \boldsymbol{b}_2} = \boldsymbol{\delta}_2.$$

Để lan truyền ngược về lớp ẩn, ta dùng quy tắc dây chuyền:

$$\boldsymbol{\delta}_1 = \frac{\partial L}{\partial \boldsymbol{z}_1} = \left(W_2^\top \boldsymbol{\delta}_2\right) \odot \phi'(\boldsymbol{z}_1).$$

Do đó

$$\frac{\partial L}{\partial W_1} = \boldsymbol{\delta}_1 \boldsymbol{x}^\top, \qquad \frac{\partial L}{\partial \boldsymbol{b}_1} = \boldsymbol{\delta}_1.$$

Nhận xét 5.1. Lan truyền ngược không phải một thuật toán "khác" hạ dốc theo gradient. Nó là kỹ thuật tính gradient hiệu quả để sau đó dùng trong hạ dốc theo gradient hay bất kỳ bộ tối ưu nào khác.

5.5 Ma trận hóa trên *mini-batch*

Trong thực hành, ta huấn luyện trên *mini-batch* (lô nhỏ) kích thước B. Nếu đặt $X \in \mathbb{R}^{d \times B}$ là ma trận gồm các vectơ đầu vào theo cột, thì

$$Z_1 = W_1 X + \boldsymbol{b}_1 \mathbf{1}^\top, \qquad H = \phi(Z_1), \qquad \hat{Y} = W_2 H + \boldsymbol{b}_2 \mathbf{1}^\top.$$

Đạo hàm trên cả lô là tổng hoặc trung bình đạo hàm từng mẫu. Thao tác ma trận hóa này không chỉ nhanh hơn về tính toán, mà còn bộc lộ rõ hơn cấu trúc tuyến tính của phép lan truyền tiến và lan truyền ngược.

5.6 Từ MLP sang học biểu diễn

Mỗi lớp ẩn có thể được xem là một phép biến đổi biểu diễn. Đầu vào ban đầu có thể là các đặc trưng thô, nhưng sau nhiều lớp, mạng tự tạo ra các đặc trưng trung gian hữu ích cho hàm mất mát cuối. Chính vì vậy, học sâu được mô tả là *representation learning* (học biểu diễn).

Hãy hình dung bài toán phân loại ảnh chữ số viết tay. Một mô hình cổ điển cần ta thiết kế đặc trưng: nét ngang, nét dọc, điểm giao, v.v. MLP sâu trên dữ liệu ảnh có thể học những biến đổi trung gian đó tự động. Trong dự báo chuỗi thời gian, *LSTM* sẽ học biểu diễn trạng thái tạm thời của quá trình sinh dữ liệu.

5.7 Điều chuẩn trong mạng nơ-ron

Vì mạng nơ-ron có sức biểu diễn lớn, quá khớp là nguy cơ thường trực. Các cơ chế điều chuẩn phổ biến gồm:

1. phạt chuẩn của hệ số;
2. *early stopping* (dừng sớm);
3. *dropout*;
4. *data augmentation* (tăng cường dữ liệu);

5. chia sẻ tham số và ràng buộc kiến trúc.

Dropout có thể được xem như một hình thức trung bình mô hình xấp xỉ: trong mỗi bước huấn luyện, một tập con nơ-ron bị tắt ngẫu nhiên. Điều này ép mạng không phụ thuộc quá mạnh vào bất kỳ đường dẫn nào.

Early stopping là điều chuẩn đặc biệt quan trọng. Nhưng nó chỉ đúng nghĩa khi được ra quyết định trên tập thẩm định, không phải tập kiểm tra. Đây là một điểm nhỏ trong code nhưng lớn trong logic thống kê.

5.8 Triệt tiêu và bùng nổ gradient

Trong mạng sâu, gradient tại lớp đầu có thể chứa tích của nhiều Jacobian:

$$\frac{\partial L}{\partial \boldsymbol{h}^{(1)}} = \left(\prod_{\ell=2}^{L} J_\ell^\top \right) \frac{\partial L}{\partial \boldsymbol{h}^{(L)}}.$$

Nếu các Jacobian này có chuẩn phổ biến nhỏ hơn 1, gradient giảm dần về 0. Nếu lớn hơn 1, nó có thể nổ. Vấn đề này biểu lộ rõ trong RNN, nơi cùng một ma trận được nhân lặp theo thời gian.

Nhiều kỹ thuật sau này thực ra là phản hồi với vấn đề này: khởi tạo tốt, *ReLU*, chuẩn hóa theo lô, kết nối phần dư, *LSTM gates* (các cổng của LSTM), và *gradient clipping* (cắt gradient).

5.9 *Softmax* và entropy chéo cho phân loại nhiều lớp

Nếu đầu ra có K lớp, ta đặt *logit* $\boldsymbol{z} \in \mathbb{R}^K$ và

$$\hat{p}_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}.$$

Với nhãn *one-hot*, *cross-entropy* là

$$L = -\sum_{k=1}^{K} y_k \log \hat{p}_k.$$

Một công thức quan trọng là

$$\frac{\partial L}{\partial z_k} = \hat{p}_k - y_k.$$

Công thức này làm cho lan truyền ngược của lớp cuối trở nên gọn gàng và là một lý do thực dụng khiến cặp *softmax* + *cross-entropy* rất phổ biến.

5.10 Tối ưu trong mạng nơ-ron

Sau khi có gradient, ta cần một bộ tối ưu. Hạ dốc theo gradient toàn bộ dữ liệu quá tốn kém; do đó ta dùng *stochastic* hay *mini-batch gradient descent*. *Momentum* thêm trí nhớ quán tính. *Adam* chuẩn hóa gradient theo ước lượng moment bậc nhất và bậc hai. Các bộ tối ưu hiện đại không thay thế nhu cầu hiểu hàm mất mát và dữ liệu; chúng chỉ là công cụ lái bài toán tối ưu.

Một điểm cần ghi nhớ: bộ tối ưu không thể cứu một giao thức đánh giá sai. Nếu tập huấn luyện/tập thẩm định/tập kiểm tra bị dùng lẫn, dù dùng Adam hay SGD cũng không làm kết quả đáng tin hơn.

5.11 Từ mạng nơ-ron đến tư duy hồi quy theo thời gian

MLP xử lý đầu vào có kích thước cố định và không thể hiện rõ cấu trúc thứ tự. Chuỗi thời gian, văn bản, âm thanh lại có bản chất tuần tự. Nếu ta cắt chuỗi thành vectơ và ném vào MLP, ta có thể mất thông tin thứ tự. Điều này dẫn đến *recurrent neural network* (mạng nơ-ron hồi tiếp), nơi tham số được chia sẻ qua các bước thời gian:

$$\boldsymbol{h}_t = \phi(W_x \boldsymbol{x}_t + W_h \boldsymbol{h}_{t-1} + \boldsymbol{b}).$$

Nhưng ở đây, vấn đề triệt tiêu/bùng nổ gradient trở nên nghiêm trọng hơn vì nhân lặp cùng một ma trận trong nhiều bước. LSTM được sinh ra để giải quyết điều này. Chương về chuỗi thời gian sẽ quay lại kỹ hơn.

5.12 Mạng nơ-ron và vấn đề diễn giải

MLP mạnh về biểu diễn nhưng yếu về minh bạch. Ta có thể xem gradient, activation, *feature importance* (mức quan trọng của đặc trưng) cục bộ, nhưng không dễ đọc một mạng như đọc một cây quyết định hay một bộ luật mờ. Vì vậy, các nỗ lực lai ghép fuzzy và mạng nơ-ron có sức hút tự nhiên. Tuy nhiên, bài học từ mạng nơ-ron là: chỉ cần một lớp kết hợp cuối cùng đủ phức tạp, khả năng diễn giải toàn cục đã giảm mạnh. Chúng ta sẽ phải nhớ điều này khi nhìn mô hình lai *fuzzy + BiLSTM*.

5.13 Kết

Chương này cần được nhớ bằng một ý tưởng cốt lõi: mạng nơ-ron không phải hộp đen vì nó không có toán học, mà vì toán học của nó được tổ chức trong một không gian tham số

lớn, phi tuyến, và khó đọc. Lan truyền ngược dựa trên quy tắc dây chuyền. Sức mạnh của nó đến từ học biểu diễn. Vấn đề của nó đến từ tối ưu, tổng quát hóa, và diễn giải. Toàn bộ ba vấn đề ấy sẽ trở lại nhiều lần trong phần học sâu và ANFIS.

Chuong 6

Học sâu trong thực hành: tối ưu, điều chuẩn và động học học

6.1 Từ công thức mạng nơ-ron đến một hệ thống học hoàn chỉnh

Khi mới học về mạng nơ-ron, người ta thường có cảm giác rằng cốt lõi của học sâu đã nằm trọn trong hai thứ: công thức lan truyền tiến và công thức lan truyền ngược. Cảm giác ấy đúng một phần, nhưng nếu dừng lại ở đó thì ta chưa hiểu vì sao học sâu lại thành công trong thực hành. Một mạng nơ-ron chỉ thực sự trở thành một hệ thống học khi ít nhất sáu lớp quyết định cùng vận hành:

1. cách tham số được khởi tạo;
2. cách gradient được ước lượng;
3. cách tốc độ học được điều khiển;
4. cách kiến trúc tạo ra thiên kiến quy nạp;
5. cách điều chuẩn được áp đặt;
6. cách giao thức đánh giá được thiết kế.

Nếu một trong sáu lớp quyết định này bị làm cẩu thả, mô hình có thể vẫn "chạy", thậm chí còn cho kết quả đẹp trên một tập dữ liệu cụ thể, nhưng ta sẽ không còn hiểu rõ mình đang thấy điều gì. Đây là lý do học sâu, hơn nhiều lĩnh vực khác, đòi hỏi một tinh thần kỹ nghệ gắn liền với tinh thần toán học. Không đủ chỉ biết công thức; ta còn phải biết công thức ấy sống như thế nào trong một quy trình.

6.2 Hình học của khởi tạo: tại sao bước đầu tiên lại có sức chi phối lớn

Hãy xét một mạng nhiều lớp tuyến tính hóa cục bộ quanh điểm khởi tạo. Nếu trọng số được chọn quá nhỏ, tín hiệu đi qua mỗi lớp sẽ co lại; nếu trọng số quá lớn, tín hiệu sẽ bùng nổ. Điều này không chỉ xảy ra với giá trị kích hoạt, mà còn xảy ra với gradient khi lan truyền ngược. Nói cách khác, khởi tạo là cách ta chọn hệ tọa độ ban đầu cho bài toán tối ưu.

Giả sử đầu vào đã được chuẩn hóa xấp xỉ có trung bình 0 và phương sai 1. Với một nơ-ron tuyến tính

$$z = \sum_{j=1}^{n_{\text{in}}} w_j x_j,$$

nếu các x_j độc lập gần đúng và w_j độc lập có trung bình 0, thì

$$\text{Var}(z) \approx n_{\text{in}} \text{Var}(w_j) \text{Var}(x_j).$$

Muốn $\text{Var}(z)$ không tăng cũng không giảm quá mạnh qua lớp, ta phải chọn $\text{Var}(w_j)$ tỉ lệ nghịch với n_{in}. Từ trực giác này, các khởi tạo kiểu Xavier/Glorot và He ra đời.

6.2.1 Khởi tạo Xavier và He

Với các hàm kích hoạt gần đối xứng quanh 0 như tanh, *Xavier initialization* chọn phương sai của trọng số xấp xỉ

$$\text{Var}(w_{ij}) \approx \frac{2}{n_{\text{in}} + n_{\text{out}}}.$$

Với ReLU, do một nửa tín hiệu thường bị chặn ở 0, *He initialization* tăng phương sai lên:

$$\text{Var}(w_{ij}) \approx \frac{2}{n_{\text{in}}}.$$

Những công thức này không phải chân lý tuyệt đối; chúng là kết quả của một phân tích gần đúng về lan truyền phương sai. Nhưng chính sự gần đúng ấy đã thay đổi thực hành học sâu. Nó chỉ ra rằng nhiều vấn đề tưởng như "mô hình khó học" thực ra chỉ là "khởi tạo đặt hệ vào một chế độ tín hiệu không ổn".

6.2.2 Khởi tạo và các hàm kích hoạt bão hòa

Với *sigmoid* hay tanh, nếu z ban đầu đã rơi quá sâu vào miền bão hòa, đạo hàm gần 0 và gradient sớm suy yếu. Vì vậy, khởi tạo nhỏ quá mức chưa chắc tốt; khởi tạo phải đủ để tín hiệu vào vùng hoạt động, nhưng không quá lớn để mọi nơ-ron bão hòa ngay. Đây là một ví

dụ điển hình cho thấy "nhỏ" và "an toàn" không phải lúc nào cũng đồng nghĩa.

6.3 Mini-batch: tại sao nhiễu lại giúp học

Nếu ta có toàn bộ dữ liệu $\mathcal{D}$, gradient thật của hàm mất mát là

$$\nabla L(\theta) = \frac{1}{n} \sum_{i=1}^{n} \nabla \ell_i(\theta).$$

Trong thực hành, ta dùng một lô nhỏ $B \subset \{1, \ldots, n\}$ và thay bằng

$$\nabla \widehat{L}_B(\theta) = \frac{1}{|B|} \sum_{i \in B} \nabla \ell_i(\theta).$$

Ước lượng này không trùng gradient thật, nhưng thường là ước lượng không chệch. Điều đáng chú ý là nhiễu do việc lấy mẫu lô nhỏ không chỉ là một cái giá phải trả; nó còn là một nguồn lợi.

6.3.1 Nhiễu giúp thoát vùng bằng phẳng và hố nông

Trong một bề mặt mất mát phi lồi, rất nhiều điểm dừng không phải là cực tiểu sâu. Một gradient toàn bộ dữ liệu có thể đẩy quỹ đạo học vào những vùng phẳng rồi tiến rất chậm. Gradient nhiễu từ mini-batch, ngược lại, đôi khi khiến quỹ đạo học không bị kẹt quá lâu ở những hố nông hoặc những điểm yên ngựa. Cách hiểu này không nên bị tuyệt đối hóa, nhưng nó giúp ta nhận ra vì sao "ước lượng gradient sai" lại có thể dẫn đến lời giải tốt.

6.3.2 Kích thước lô là một siêu tham số thật sự

Lô quá nhỏ làm gradient quá nhiễu, thống kê không ổn, và hiệu quả phần cứng kém. Lô quá lớn làm động học tiến gần hạ dốc toàn phần, đôi khi giảm khả năng tổng quát hóa và làm yêu cầu về bộ nhớ tăng mạnh. Bởi vậy, kích thước lô không phải chỉ là tham số hệ thống; nó là một phần của động học học.

6.4 Động lượng, RMSProp và Adam dưới góc nhìn hình học

Nếu chỉ dùng hạ dốc theo gradient cơ bản

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t),$$

ta đang di chuyển theo một hệ tọa độ Euclid cố định với cùng độ lớn bước trên mọi hướng. Nhưng bề mặt mất mát trong học sâu rất *dị hướng*: có hướng cong mạnh, có hướng phẳng, có hướng gradient đảo dấu liên tục. Đây là lý do các bộ tối ưu hiện đại tìm cách tái tham số hóa bài toán hoặc thêm quán tính.

6.4.1 Động lượng như trung bình động của hướng giảm

Momentum dùng

$$\boldsymbol{v}_{t+1} = \beta \boldsymbol{v}_t + (1-\beta)\nabla L(\theta_t), \qquad \theta_{t+1} = \theta_t - \eta \boldsymbol{v}_{t+1}.$$

Nếu gradient thay đổi cùng chiều trong nhiều bước, động lượng tích lũy và tăng tốc. Nếu gradient dao động trái dấu mạnh theo một hướng, trung bình động sẽ làm suy giảm dao động đó. Hình ảnh vật lý gần đúng là một hạt chuyển động trên mặt năng lượng với quán tính.

6.4.2 RMSProp và ý tưởng co giãn theo tọa độ

Trong nhiều bài toán, một số tham số luôn có gradient lớn, một số khác luôn có gradient nhỏ. Dùng cùng một tốc độ học cho mọi chiều là kém hiệu quả. *RMSProp* khắc phục bằng cách chia gradient cho căn bậc hai của trung bình động bình phương gradient. Trực giác là: hướng nào vốn đã có gradient lớn thì giảm bước lại; hướng nào gradient nhỏ thì cho đi xa hơn.

6.4.3 Adam như sự kết hợp giữa quán tính và co giãn

Adam kết hợp cả trung bình động bậc một và bậc hai:

$$\boldsymbol{m}_{t+1} = \beta_1 \boldsymbol{m}_t + (1-\beta_1)\boldsymbol{g}_t, \qquad \boldsymbol{v}_{t+1} = \beta_2 \boldsymbol{v}_t + (1-\beta_2)\boldsymbol{g}_t^2,$$

và cập nhật

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{\boldsymbol{m}}_{t+1}}{\sqrt{\hat{\boldsymbol{v}}_{t+1}} + \varepsilon}.$$

Adam đặc biệt hữu ích khi gradient thưa, tỷ lệ gradient giữa các chiều rất chênh lệch, hoặc khi ta cần một bộ tối ưu ổn định để khởi động nhanh.

6.4.4 Một ngộ nhận phổ biến về Adam

Ngộ nhận hay gặp là: "Adam luôn tốt hơn SGD." Điều này sai. Adam thường giúp tối ưu dễ hơn, nhưng lời giải mà nó tìm được không phải lúc nào cũng tổng quát hóa tốt hơn. Có những bài toán mà SGD với động lượng, dù chậm hơn, lại cho nghiệm ổn định hơn trên dữ liệu ngoài mẫu. Vì vậy, tối ưu nhanh và tổng quát hóa tốt là hai tiêu chuẩn có liên quan nhưng không đồng nhất.

6.5 Lịch trình tốc độ học: nghệ thuật điều khiển quy mô bước đi

Tốc độ học η thường là siêu tham số có ảnh hưởng lớn nhất. Điều này không khó hiểu: trong tối ưu phi lồi, ta không chỉ cần biết đi theo hướng nào, mà còn phải biết đi xa bao nhiêu. Chỉ thay đổi η đôi khi làm mô hình đi từ "không học được gì" sang "học rất tốt".

6.5.1 Vì sao cần giảm tốc độ học theo thời gian

Trong giai đoạn đầu, ta muốn đi nhanh để khám phá địa hình của hàm mất mát. Về sau, khi đã vào vùng nghiệm tốt, ta muốn đi chậm hơn để ổn định quanh nghiệm. Cách nghĩ này dẫn đến các lịch trình giảm η theo thời gian: giảm bậc thang, giảm theo hàm mũ, giảm theo cosine, *warm restarts*, *one-cycle*.

6.5.2 Warmup: tại sao có lúc phải tăng tốc độ học từ nhỏ lên lớn

Ở các mô hình rất sâu hoặc có chuẩn hóa phức tạp, dùng ngay một tốc độ học lớn có thể làm huấn luyện bất ổn ở vài epoch đầu. Khi đó người ta dùng *warmup*: bắt đầu với tốc độ nhỏ rồi tăng dần đến giá trị đích. Điều này đặc biệt phổ biến ở các mô hình dựa trên *transformer*, nhưng ý tưởng thì tổng quát hơn.

6.5.3 Hai triết lý điều chỉnh tốc độ học

Có hai triết lý hay gặp:

1. **điều chỉnh theo lịch có sẵn**: định nghĩa trước tốc độ học theo epoch;

2. **điều chỉnh theo tín hiệu thẩm định**: ví dụ `ReduceLROnPlateau`.

Mỗi cách có ưu điểm riêng. Cách thứ nhất rõ ràng, dễ lặp lại, ít phụ thuộc nhiễu của tập thẩm định. Cách thứ hai phản ứng với thực tế huấn luyện. Nhưng nếu tập thẩm định không được tách đúng nghĩa, cách thứ hai sẽ kéo theo toàn bộ sai lệch thống kê của quy trình.

6.6 Chuẩn hóa trong mạng: không chỉ để làm đẹp dữ liệu

Nhiều người mới hiểu chuẩn hóa như một bước xử lý dữ liệu đầu vào: trừ trung bình, chia độ lệch chuẩn. Trong học sâu, chuẩn hóa còn diễn ra bên trong mạng. Điều này làm thay đổi hoàn toàn cách ta nhìn các lớp trung gian.

6.6.1 Batch normalization

Batch normalization chuẩn hóa activation theo từng lô:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \qquad y = \gamma \hat{x} + \beta.$$

Hai tham số γ, β cho phép lớp học lại tỷ lệ và độ dời mong muốn. Bởi vậy, *batch normalization* không làm mất sức biểu diễn của mạng; nó chỉ tái tham số hóa mạng về một hệ tọa độ thuận lợi hơn.

6.6.2 Tại sao chuẩn hóa giúp học

Chuẩn hóa giúp theo nhiều cơ chế cùng lúc:

1. kiểm soát quy mô của activation;
2. làm bề mặt mất mát ít méo hơn;
3. cho phép dùng tốc độ học lớn hơn;
4. đóng vai trò điều chuẩn nhẹ do phụ thuộc vào thống kê của lô.

Điều quan trọng là không nên biến những giải thích này thành thần thoại duy nhất. Không có một cơ chế đơn lẻ được mọi người đồng ý là “toàn bộ lý do” vì sao *batch normalization* thành công.

6.6.3 Layer normalization và các biến thể

Trong mô hình hồi tiếp hoặc mô hình mà kích thước lô nhỏ, *layer normalization* thường phù hợp hơn. Thay vì chuẩn hóa theo lô, ta chuẩn hóa theo chiều đặc trưng trong từng mẫu. Ý tưởng chung vẫn vậy: tái điều chỉnh hình học của tín hiệu bên trong mạng.

6.7 Điều chuẩn tường minh và điều chuẩn ngầm

Một tài liệu về học sâu sẽ thiếu sót nếu chỉ nói về điều chuẩn bằng vài từ khóa như *dropout* hay *weight decay*. Điều chuẩn trong học sâu có ít nhất hai lớp.

6.7.1 Điều chuẩn tường minh

Đây là những cơ chế được viết trực tiếp vào hàm mục tiêu hoặc quy trình học:

1. phạt chuẩn L^2 lên trọng số;
2. *dropout*;
3. *label smoothing*;
4. tăng cường dữ liệu;
5. ràng buộc cấu trúc kiến trúc.

6.7.2 Điều chuẩn ngầm

Ngoài những cơ chế viết tường minh, rất nhiều yếu tố khác cũng điều chuẩn ngầm:

1. cách khởi tạo;
2. kích thước lô;
3. bộ tối ưu;
4. dừng sớm;
5. bản thân kiến trúc.

Ví dụ, một mạng tích chập tự nhiên có thiên kiến quy nạp về tính cục bộ và tính tịnh tiến. Một mạng hồi tiếp có thiên kiến quy nạp về phụ thuộc tuần tự. Một ANFIS có thiên kiến quy nạp rằng quan hệ đầu vào–đầu ra có thể được phân mảnh thành những mô hình cục bộ gắn với các khái niệm mờ.

6.8 Dropout được hiểu đúng là gì

Dropout thường được giới thiệu như "ngẫu nhiên tắt một số nơ-ron để tránh quá khớp". Mô tả đó không sai, nhưng quá sơ sài. Điều đáng quan tâm là: tại sao việc phá cấu trúc tạm thời này lại giúp tổng quát hóa?

Một cách hiểu hữu ích là xem *dropout* như việc huấn luyện một họ rất lớn các mạng con chia sẻ tham số. Ở mỗi bước, ta lấy một mạng con ngẫu nhiên. Khi suy diễn, ta xấp xỉ trung bình của họ mạng con ấy bằng một mạng đầy đủ đã được co giãn thích hợp.

Nhưng ta cũng phải thấy mặt trái. Trong các chuỗi thời gian ngắn hoặc dữ liệu có cấu trúc mong manh, *dropout* mạnh có thể làm vỡ tín hiệu tuần tự. Vì vậy, không nên áp dụng *dropout* như một nghi thức vô điều kiện.

6.9 Dừng sớm: một điều chuẩn mạnh nhưng dễ bị lạm dụng

Hãy tưởng tượng đồ thị *loss* trên tập huấn luyện luôn giảm, còn *loss* trên tập thẩm định giảm rồi tăng. Điểm cực tiểu của đường thẩm định đánh dấu thời điểm mà mô hình vừa đủ học được quy luật hữu ích nhưng chưa học quá nhiều nhiễu của tập huấn luyện. Dừng tại đây là hợp lý.

Điều cốt lõi là: tập dùng để quyết định dừng phải là tập thẩm định thật sự. Nếu dùng tập kiểm tra, ta đã biến một công cụ điều chuẩn thành một nguồn rò rỉ thông tin. Đây không phải ngụy biện học thuật; nó trực tiếp làm méo ước lượng hiệu năng ngoài mẫu.

6.10 Tối ưu thành công không đồng nghĩa với mô hình đúng

Một sai lầm nhận thức phổ biến là: nếu hàm mất mát giảm tốt và các chỉ số trên tập kiểm tra cao, mô hình hẳn đã "hiểu đúng" bài toán. Câu suy luận này sai ở nhiều tầng.

6.10.1 Giảm loss chỉ nói về bài toán tối ưu đang được giải

Nếu biến mục tiêu được định nghĩa lệch, hoặc giao thức đánh giá sai, tối ưu vẫn có thể thành công trên một bài toán sai. Mô hình không phân biệt được nó đang bị giao một nhiệm vụ hợp lý hay không; nó chỉ biết tối ưu cái ta giao.

6.10.2 Hiệu năng cao có thể đến từ cấu trúc thống kê dễ dàng

Trong chuỗi giá, chỉ cần nắm được xu hướng mức giá chung cũng đã có thể cho R^2 khá đẹp trên giá tuyệt đối. Điều đó không nhất thiết nghĩa là mô hình hiểu cấu trúc vi mô của chuyển động giá. Đây là một ví dụ cho thấy chỉ số đẹp có thể che giấu bản chất dự báo nghèo.

6.11 Độ sâu, chiều rộng và thiên kiến quy nạp

Vì sao một số quan hệ phù hợp với mạng sâu hơn mạng nông? Một phần câu trả lời nằm ở khả năng biểu diễn hiệu quả: có những hàm mà mạng nông vẫn biểu diễn được, nhưng cần số nơ-ron rất lớn; trong khi mạng sâu biểu diễn với tham số gọn hơn nhờ tái sử dụng các biểu diễn trung gian.

Nhưng phần quan trọng hơn cho người thực hành là thiên kiến quy nạp. Độ sâu cho phép phân tầng khái niệm. Mạng tích chập phù hợp dữ liệu có tính cục bộ và tái lặp theo không gian. Mạng hồi tiếp phù hợp dữ liệu có ký ức theo thời gian. Hệ mờ phù hợp dữ liệu mà quan hệ đầu vào–đầu ra có thể được diễn giải bằng các chế độ cục bộ. Nếu chọn sai thiên kiến quy nạp, tăng thêm tham số thường chỉ làm mô hình dễ quá khớp hơn.

6.12 Khi nào một mô hình học sâu thất bại dù công thức đều đúng

Ta có thể liệt kê một số chế độ thất bại rất hay gặp:

1. đầu vào không được chuẩn hóa hợp lý;
2. tốc độ học quá lớn hoặc quá nhỏ;
3. mục tiêu học không khớp với mục tiêu đánh giá;
4. tập thẩm định và tập kiểm tra bị dùng lẫn;
5. mô hình quá mạnh so với quy mô dữ liệu;
6. chỉ số đánh giá không phản ánh cấu trúc thật của bài toán;
7. pipeline có rò rỉ thông tin;
8. đầu ra không bị ép tuân theo các ràng buộc logic nội tại của miền ứng dụng.

Điều đáng chú ý là phần lớn lỗi trên không nằm ở định lý backpropagation hay công thức của LSTM. Chúng nằm ở giao diện giữa toán học của mô hình và logic của thí nghiệm.

6.13 Mất mát, mục tiêu và bài toán ra quyết định

Không phải lúc nào tối thiểu hóa MSE cũng là điều ta thật sự cần. Nếu mục tiêu cuối là xếp hạng, phân loại xu hướng, hay điều khiển rủi ro, hàm mất mát nên phản ánh đúng cấu trúc quyết định.

Ví dụ, trong dự báo tài chính, sai số 1 đơn vị ở một ngày bình thường và 1 đơn vị ở một ngày khủng hoảng có thể mang ý nghĩa quyết định rất khác nhau. Tối ưu hóa MSE thuần túy không biết điều đó. Nếu ta quan tâm tới đuôi phân phối, ta có thể phải dùng *quantile loss*, *expectile loss*, hoặc mô hình hóa toàn bộ phân phối có điều kiện.

6.13.1 Hiệu chỉnh theo miền ứng dụng

Nếu đầu ra là bộ OHLC, bản thân miền ứng dụng yêu cầu các quan hệ hình học như

$$H \geq \max(O, C), \qquad L \leq \min(O, C).$$

Một hàm mất mát không ý thức được những ràng buộc này là một hàm mất mát chưa hoàn chỉnh cho bài toán. Nó có thể vẫn cho mô hình học được cái gì đó, nhưng không đảm bảo học đúng đối tượng cần dùng.

6.14 Diễn giải trong học sâu: điều gì làm được, điều gì không

Các công cụ giải thích như *saliency map*, *integrated gradients*, *SHAP*, *attention visualization* rất hữu ích, nhưng chúng không phải chiếc đũa thần biến một mạng sâu thành một hệ thống suy luận minh bạch như sách giáo khoa logic.

Một cách phân biệt hữu ích là:

1. **giải thích cơ chế**: mô hình hoạt động bằng cấu trúc nào;
2. **giải thích cục bộ**: vì sao mẫu này ra kết quả này;
3. **giải thích thống kê**: đặc trưng nào thường có ảnh hưởng lớn;
4. **giải thích khái niệm**: mô hình có gắn được các đại lượng nội bộ với khái niệm con người hiểu hay không.

Hệ mờ và ANFIS mạnh nhất ở loại thứ tư, nhưng lại yếu dần khi bị bọc trong một khối mạng sâu phức tạp. Đây là một điểm sẽ trở nên rất quan trọng ở chương phân tích mô hình cụ thể.

6.15 Quy trình huấn luyện nghiêm túc cho người mới

Để tránh biến học sâu thành "ma thuật phụ thuộc may mắn", ta có thể giữ một checklist rất chặt:

1. xác định chính xác biến mục tiêu và chỉ số đánh giá;
2. mô tả bằng lời cách mỗi đặc trưng được xây từ dữ liệu thô;
3. tách train/validation/test trước mọi biến đổi học từ dữ liệu;
4. fit bộ chuẩn hóa trên train;
5. huấn luyện với logging đầy đủ của train và validation;
6. chỉ dùng validation cho mọi quyết định chọn mô hình;
7. giữ test cho báo cáo cuối;
8. kiểm tra cả lỗi số học, lỗi logic, và lỗi diễn giải.

Checklist này nghe có vẻ giản dị, nhưng những mô hình yếu nhất và mạnh nhất thường khác nhau chính ở chỗ ai giữ được kỷ luật này đến cùng.

6.16 Một nhận xét quan trọng cho người học ANFIS và học sâu cùng lúc

Khi người học chuyển từ hệ mờ sang học sâu, họ thường có hai phản ứng cực đoan. Một là xem học sâu như một hộp đen mạnh hơn nên không cần diễn giải. Hai là xem cứ gắn một lớp mờ vào đâu đó thì mô hình lập tức giải thích được. Cả hai đều sai.

Điều đúng hơn là: học sâu cung cấp năng lực biểu diễn rất mạnh; hệ mờ cung cấp cấu trúc khái niệm; ANFIS cố gắng nối hai phía; nhưng khả năng diễn giải thật sự phụ thuộc vào việc luật mờ có còn chi phối đầu ra cuối hay không, và pipeline có đủ sạch để ta tin vào các kết luận hay không.

6.17 Kết luận chương

Học sâu trong thực hành là bài toán phối hợp nhiều lớp quyết định: khởi tạo, gradient nhiễu, bộ tối ưu, lịch trình tốc độ học, điều chuẩn, và giao thức đánh giá. Nếu chỉ học công thức lan truyền ngược mà bỏ qua các lớp quyết định này, ta mới học được phần cú pháp của mô hình, chưa học được ngữ nghĩa của quá trình huấn luyện. Đây là cây cầu cần thiết để bước sang chuỗi thời gian và ANFIS một cách trưởng thành hơn.

Chuong 7

Chuỗi thời gian, dự báo và RNN/LSTM/BiLSTM

7.1 Tư duy đúng về chuỗi thời gian: dữ liệu và thông tin không phải một

Khi nhìn một bảng dữ liệu chuỗi thời gian, ta dễ quên rằng mỗi hàng dữ liệu không chỉ là một điểm trong không gian đặc trưng, mà còn gắn với một trạng thái thông tin. Trong dự báo, điều quyết định không chỉ là "ta có dữ liệu gì", mà là "đến thời điểm này ta được phép biết những gì". Hai câu hỏi đó không hoàn toàn giống nhau.

Để diễn đạt chặt chẽ hơn, ta có thể nghĩ đến một họ tập thông tin

$$\mathcal{F}_t,$$

trong đó $\mathcal{F}_t$ chứa mọi thông tin hợp lệ có thể dùng đến thời điểm t. Một dự báo một bước hợp lệ phải có dạng

$$\hat{y}_{t+1|t} = g(\mathcal{F}_t).$$

Nếu một đặc trưng nào đó sử dụng dữ liệu của $t+1$, dù chỉ gián tiếp qua một thống kê chuẩn hóa hay một phép chọn tham số, thì nó không còn là hàm của $\mathcal{F}_t$ nữa. Đây chính là lõi logic của rò rỉ thông tin trong chuỗi thời gian.

7.2 Tại sao dữ liệu chuỗi thời gian không thể bị đối xử như dữ liệu i.i.d.

Với dữ liệu i.i.d., ta có thể xáo trộn và chia ngẫu nhiên tập huấn luyện và tập kiểm tra. Với chuỗi thời gian, xáo trộn như vậy thường phá vỡ cấu trúc thông tin. Nếu mẫu ở năm 2025 bị

đưa vào tập huấn luyện còn mẫu ở năm 2020 lại bị đưa vào tập kiểm tra, mô hình đã được phép học từ tương lai của chính dữ liệu mà nó bị yêu cầu kiểm tra trên quá khứ.

Nguyên lý này không chỉ áp dụng cho nhãn mục tiêu. Nó áp dụng cho mọi bước học từ dữ liệu:

1. chuẩn hóa;
2. giảm chiều;
3. chọn đặc trưng;
4. phân cụm;
5. chỉnh siêu tham số;
6. dừng sớm;
7. chọn *seed* tốt nhất.

Do đó, chuỗi thời gian không phải là "dữ liệu thường nhưng có thêm chỉ số thời gian". Nó là một miền ứng dụng mà giao thức đánh giá là một phần của định nghĩa bài toán.

7.3 Tính dừng, xu thế và lý do người ta thích dùng return

Nhiều chuỗi tài chính ở mức giá tuyệt đối có xu thế, thay đổi thang đo, và thay đổi độ biến động theo thời gian. Điều này làm cho việc học trực tiếp trên mức giá tuyệt đối trở nên khó diễn giải và dễ tạo chỉ số ảo đẹp. Nếu giá tăng dần theo thời gian, một mô hình rất đơn giản vẫn có thể đạt R^2 cao bằng cách bám xu thế mức giá.

Vì vậy người ta hay chuyển sang các đại lượng ổn định hơn như *return*:

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}, \qquad \tilde{r}_t = \log P_t - \log P_{t-1}.$$

Log-return có tính cộng dồn theo thời gian:

$$\log \frac{P_t}{P_{t-k}} = \sum_{j=0}^{k-1} \log \frac{P_{t-j}}{P_{t-j-1}}.$$

Đây là ưu điểm lớn trong nhiều phân tích lý thuyết. Tuy nhiên, *simple return* lại dễ giải thích trực quan hơn cho người mới.

7.3.1 Một hiểu lầm thường gặp về return

Nói rằng return "dừng hơn" không có nghĩa là return luôn dừng. Nhiều chuỗi return tài chính vẫn có hiện tượng cụm biến động, đuôi dày, bất đối xứng, và thay đổi chế độ. Vì vậy, chuyển sang return giúp ích, nhưng không tự động biến bài toán thành đơn giản.

7.4 Biến mục tiêu trong dự báo: phải nói chính xác mốc so sánh

Giả sử ta muốn dự báo giá đóng ngày mai. Có ít nhất ba cách định nghĩa mục tiêu:

1. dự báo trực tiếp C_{t+1};
2. dự báo $\frac{C_{t+1}}{C_t} - 1$;
3. dự báo $\log C_{t+1} - \log C_t$.

Ba bài toán này liên quan, nhưng không giống nhau. Hàm mất mát, chỉ số đánh giá, và cách quy đổi kết quả đều phụ thuộc vào lựa chọn này. Nếu tài liệu không nói rõ mình đang dùng định nghĩa nào, rất nhiều tranh luận sau đó sẽ trở nên mơ hồ.

Đối với bộ OHLC, tình huống còn tinh tế hơn. Ta có thể định nghĩa mục tiêu của Open_{t+1} theo Open_t, theo Close_t, hoặc theo log-ratio tương ứng. Mỗi cách có một ý nghĩa kinh tế riêng. Điều quan trọng không phải là chỉ có một cách đúng, mà là phải nhất quán và thành thật trong diễn giải.

7.5 Cửa sổ trượt: biến chuỗi thành tensor đầu vào

Một cách chuẩn để đưa chuỗi vào mô hình học là dùng cửa sổ độ dài L:

$$X_t = \begin{bmatrix} \boldsymbol{x}_{t-L+1} \\ \boldsymbol{x}_{t-L+2} \\ \vdots \\ \boldsymbol{x}_t \end{bmatrix} \in \mathbb{R}^{L\times d}, \qquad y_t = \text{mục tiêu tại } t+1.$$

Mỗi mẫu huấn luyện khi đó không còn là một vectơ, mà là một khối gồm L bước liên tiếp. Điều này cho phép mô hình học không chỉ tương quan đồng thời giữa các đặc trưng, mà còn phụ thuộc theo độ trễ.

7.5.1 Điểm dễ sai nhất: căn chỉnh chỉ số

Giả sử $L = 3$. Nếu dùng $X_t = (x_{t-2}, x_{t-1}, x_t)$ để dự báo y_{t+1}, thì mẫu đầu tiên hợp lệ là khi $t = 3$ nếu chỉ số bắt đầu từ 1. Chỉ cần lệch một đơn vị, ta có thể:

1. vô tình dùng cả x_{t+1} trong cửa sổ;
2. hoặc bỏ phí một lượng dữ liệu hợp lệ;
3. hoặc căn chỉnh sai giữa giá hiện tại và giá tương lai.

Đây là lý do tại sao trong phần phân tích code, ta phải đọc rất kỹ từng phép `shift`, từng chỉ số vòng lặp, chứ không thể chỉ nghe mô tả bằng lời.

7.6 Dự báo một bước, nhiều bước, trực tiếp và đệ quy

Không phải mọi bài toán chuỗi thời gian đều là dự báo một bước. Nếu muốn dự báo h bước tới, ta có nhiều chiến lược.

7.6.1 Dự báo trực tiếp

Ta học riêng một mô hình cho mỗi chân trời:

$$\hat{y}_{t+h|t} = f_h(\mathcal{F}_t).$$

Ưu điểm là tránh tích lũy sai số đệ quy. Nhược điểm là phải học nhiều mô hình hoặc một mô hình đa đầu ra phức tạp hơn.

7.6.2 Dự báo đệ quy

Ta học một mô hình một bước

$$\hat{y}_{t+1|t} = f(\mathcal{F}_t),$$

rồi dùng chính dự báo đó làm đầu vào để đi tiếp đến $t + 2, t + 3, \ldots$. Cách này gọn, nhưng sai số có thể chồng chất.

7.6.3 Dự báo hỗn hợp

Có những cách lai giữa trực tiếp và đệ quy, ví dụ dùng mô hình trực tiếp cho vài bước đầu rồi đệ quy tiếp, hoặc học cả trạng thái ẩn lẫn dự báo. Điều cần giữ trong đầu là: chiến lược nhiều bước không phải chi tiết phụ; nó thay đổi hoàn toàn cấu trúc lỗi.

7.7 Đánh giá chuỗi thời gian: backtesting quan trọng ngang mô hình

Trong học máy tổng quát, người ta hay nhấn mạnh kiến trúc. Trong dự báo chuỗi thời gian, *backtesting* (kiểm thử lùi theo thời gian) có thể quan trọng ngang kiến trúc. Một mô hình bình thường nhưng *backtest* đúng đáng tin hơn nhiều so với một mô hình phức tạp nhưng dùng tập kiểm tra như tập thẩm định.

7.7.1 Ba giao thức đánh giá cơ bản

1. **Hold-out theo thời gian**: lấy đoạn đầu làm huấn luyện, đoạn giữa làm thẩm định, đoạn cuối làm kiểm tra.
2. **Rolling window**: cửa sổ huấn luyện trượt, kích thước cố định.
3. **Expanding window**: cửa sổ huấn luyện mở rộng dần theo thời gian.

Mỗi giao thức phản ánh một giả thiết vận hành khác nhau. Nếu hệ thống ngoài đời luôn được huấn luyện lại với dữ liệu mới, *rolling* hoặc *expanding* sẽ hợp lý hơn một phép chia giữ nguyên đơn giản.

7.7.2 Tập kiểm tra không phải nơi chọn mô hình

Trong chuỗi thời gian, quy tắc này càng nghiêm hơn vì số mẫu hiệu dụng thường ít hơn dữ liệu i.i.d. Nếu ta vừa dừng sớm theo tập kiểm tra, vừa chọn *seed* tốt nhất theo tập kiểm tra, rồi lại báo cáo tập kiểm tra, thì ta đang dùng cùng một chuỗi dữ liệu cho ba mục đích khác nhau. Khi đó, không còn lý do gì để tin rằng chỉ số báo cáo phản ánh đúng hiệu năng ngoài mẫu.

7.8 RNN cơ bản và lý do nó gặp khó khăn

RNN chuẩn đặt

$$\boldsymbol{h}_t = \phi(W_x \boldsymbol{x}_t + W_h \boldsymbol{h}_{t-1} + \boldsymbol{b}), \qquad \hat{\boldsymbol{y}}_t = W_y \boldsymbol{h}_t + \boldsymbol{c}.$$

Vì trạng thái $\boldsymbol{h}_t$ được tính từ trạng thái trước đó, mô hình có một cơ chế ghi nhớ tự nhiên. Nhưng chính vì dùng lặp cùng ma trận W_h, gradient theo các trạng thái cũ có dạng tích lũy của nhiều Jacobian. Đây là nguồn gốc của triệt tiêu và bùng nổ gradient trong RNN.

7.8.1 Một phân tích rất ngắn về triệt tiêu gradient

Nếu bỏ qua phi tuyến để nhìn gần đúng tuyến tính, ta có

$$\boldsymbol{h}_t \approx W_h \boldsymbol{h}_{t-1}.$$

Khi đó

$$\boldsymbol{h}_t \approx W_h^k \boldsymbol{h}_{t-k}.$$

Nếu bán kính phổ của W_h nhỏ hơn 1, W_h^k suy giảm rất nhanh theo k. Gradient lan truyền ngược cũng chịu quy luật tương tự. Đây là lý do RNN chuẩn khó học phụ thuộc dài hạn.

7.9 LSTM: cổng nhớ như một cấu trúc tối ưu hóa, không chỉ là một công thức

LSTM được đề xuất để tạo ra một đường truyền thông tin dài hạn ổn định hơn. Các công thức của nó là:

$$\begin{aligned}
\boldsymbol{i}_t &= \sigma(W_i \boldsymbol{x}_t + U_i \boldsymbol{h}_{t-1} + \boldsymbol{b}_i),\\
\boldsymbol{f}_t &= \sigma(W_f \boldsymbol{x}_t + U_f \boldsymbol{h}_{t-1} + \boldsymbol{b}_f),\\
\boldsymbol{o}_t &= \sigma(W_o \boldsymbol{x}_t + U_o \boldsymbol{h}_{t-1} + \boldsymbol{b}_o),\\
\tilde{\boldsymbol{c}}_t &= \tanh(W_c \boldsymbol{x}_t + U_c \boldsymbol{h}_{t-1} + \boldsymbol{b}_c),\\
\boldsymbol{c}_t &= \boldsymbol{f}_t \odot \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \odot \tilde{\boldsymbol{c}}_t,\\
\boldsymbol{h}_t &= \boldsymbol{o}_t \odot \tanh(\boldsymbol{c}_t).
\end{aligned}$$

Nếu đọc hời hợt, ta chỉ thấy đây là một tập công thức nhiều ký hiệu. Nếu đọc đúng, ta thấy LSTM thực chất chèn một "đường cao tốc" cho thông tin đi qua trạng thái nhớ $\boldsymbol{c}_t$. Cổng quên điều khiển phần nào của quá khứ được giữ lại; cổng vào điều khiển phần nào của thông tin mới được chèn thêm; cổng ra điều khiển phần nào của trạng thái nhớ được lộ ra cho đầu ra.

7.9.1 Ý nghĩa của cổng quên

Nếu $\boldsymbol{f}_t$ gần 1 trong một số chiều, thành phần tương ứng của $\boldsymbol{c}_{t-1}$ gần như đi xuyên qua sang $\boldsymbol{c}_t$. Nói cách khác, mô hình có thể học cách "đừng động vào thông tin này quá sớm". Đây là cơ chế mà RNN chuẩn không có.

7.9.2 LSTM không phải ký ức vô hạn

Tuy nhiên, ta cũng không nên thần thoại hóa. LSTM *có khả năng học* phụ thuộc dài hạn tốt hơn RNN chuẩn, nhưng không có nghĩa là cứ tăng độ dài chuỗi thì LSTM sẽ nắm được hết mọi cấu trúc. Khả năng đó còn phụ thuộc vào dữ liệu, hàm mất mát, kiến trúc, và tối ưu.

7.10 BiLSTM trong dự báo: khi nào hợp lý, khi nào dễ bị hiểu nhầm

BiLSTM kết hợp một hướng đọc xuôi và một hướng đọc ngược trên *chính cửa sổ đã quan sát*. Điều này rất hữu ích trong các bài toán như gán nhãn chuỗi, xử lý tiếng nói, hoặc phân tích toàn chuỗi, nơi tại vị trí t ta được phép nhìn cả bối cảnh trước và sau trong cùng một chuỗi đầu vào.

Trong dự báo, câu hỏi là: đọc ngược như vậy có phạm luật thông tin không? Câu trả lời là:

1. nếu cửa sổ đầu vào chỉ gồm các bước đến thời điểm hiện tại t, BiLSTM không nhìn vào tương lai thật sự $t+1, t+2, \ldots$, nên không tự động gây rò rỉ;

2. nhưng BiLSTM làm cho cách diễn giải động học khó hơn, vì đầu ra cuối được xây từ hai hướng quét khác nhau trên cùng cửa sổ quá khứ.

Đây là một điểm phải nói thật rõ. Không phải cứ thấy “bi-directional” là kết luận có rò rỉ. Nhưng cũng không được ngây thơ nghĩ rằng BiLSTM hoàn toàn vô hại về mặt diễn giải.

7.11 Dự báo tài chính với OHLC: bài toán có cấu trúc hình học nội tại

Bộ giá trị (O, H, L, C) không phải bốn con số độc lập. Chúng phải thỏa:

$$H \geq \max(O, C), \qquad L \leq \min(O, C).$$

Ngoài ra, khoảng $H - L$ phải không âm, và trong nhiều bài toán ta còn quan tâm đến vị trí tương đối của O và C trong khoảng đó. Vì vậy, nếu ta cho mô hình dự báo bốn đầu ra độc lập rồi đánh giá từng đầu ra riêng lẻ, ta đang bỏ qua cấu trúc hình học quan trọng nhất của dữ liệu.

7.11.1 Một ví dụ về dự báo vô nghĩa nhưng RMSE vẫn đẹp

Giả sử giá thật là

$$(O, H, L, C) = (100, 106, 98, 104),$$

còn dự báo là

$$(\hat{O}, \hat{H}, \hat{L}, \hat{C}) = (101, 103, 99, 105).$$

Sai số trên từng biến không quá lớn. Nhưng dự báo này vô lý vì $\hat{H} = 103 < \hat{C} = 105$. Nếu hệ thống phía sau cần mô phỏng chiến lược giao dịch, chỉ một vi phạm như vậy đã đủ làm mất nghĩa của mẫu dự báo, cho dù RMSE có thể vẫn chấp nhận được.

7.12 Chỉ số đánh giá cho chuỗi thời gian: cần đọc đúng ngữ nghĩa

RMSE nhạy với sai số lớn. *MAE* dễ hiểu hơn về thang đo. *MAPE* dễ đọc theo phần trăm nhưng kém ổn khi mẫu số nhỏ. *SMAPE* sửa một phần vấn đề ấy. *MASE* có lợi thế chuẩn hóa theo dự báo ngây thơ. *Pinball loss* phù hợp với dự báo phân vị.

7.12.1 Độ chính xác hướng

Trong tài chính, người ta rất thích dùng *directional accuracy*:

$$\mathrm{DA} = \frac{1}{n} \sum_{t=1}^{n} \mathbf{1}\{\mathrm{sign}(y_t - y_{t-1}) = \mathrm{sign}(\hat{y}_t - y_{t-1})\}.$$

Nhưng chỉ số này chỉ có ý nghĩa khi $y_t - y_{t-1}$ phản ánh đúng khái niệm "hướng" mà mô hình thật sự đang học. Nếu mô hình huấn luyện trên return so với một mốc nền khác, còn DA lại đo so với giá trước đó của chính biến, thì ta đang đặt hai khái niệm khác nhau vào cùng một bảng báo cáo.

7.13 Dự báo xác suất và bất định trong chuỗi thời gian

Một ngộ nhận rất phổ biến là: dự báo chỉ cần một con số. Trong nhiều ứng dụng, điều ta cần không chỉ là kỳ vọng có điều kiện, mà là toàn bộ phân phối có điều kiện. Ví dụ, trong quản trị rủi ro, đuôi trái của phân phối thường quan trọng hơn trung bình.

Điều này dẫn đến các mô hình dự báo phân vị, mô hình *distributional forecasting*, và các chỉ số như *CRPS*. Ngay cả nếu tài liệu này tập trung nhiều vào dự báo điểm, người học vẫn nên ghi nhớ rằng dự báo điểm chỉ là một lát cắt của bài toán bất định.

7.14 Các nguồn rò rỉ thông tin đặc thù của chuỗi thời gian

Ngoài những lỗi tổng quát của học máy, chuỗi thời gian có các nguồn rò rỉ đặc thù:

1. dùng bộ chuẩn hóa fit trên toàn bộ chuỗi;
2. tính đặc trưng tại t bằng cách vô tình dùng $t + 1$ qua các phép `shift` và `rolling`;
3. dùng tập kiểm tra cho dừng sớm;
4. chọn lần chạy tốt nhất trên tập kiểm tra;
5. tinh chỉnh chân trời dự báo sau khi đã nhìn kết quả kiểm tra.

Những lỗi này rất nguy hiểm vì mô hình vẫn cho ra bảng số rất thuyết phục. Không có lỗi cú pháp nào nhắc ta rằng mình vừa phá hỏng tính hợp lệ của thí nghiệm.

7.15 Từ lý thuyết đến phần phân tích script

Khi đọc một script dự báo chuỗi thời gian, ta nên tự hỏi ít nhất tám câu:

1. đầu vào nào là hợp lệ ở thời điểm dự báo;
2. biến mục tiêu được định nghĩa theo mốc nào;
3. cửa sổ được căn chỉnh ra sao;
4. chuẩn hóa được fit trên phần nào;
5. tập thẩm định có tách thật không;
6. tập kiểm tra có bị dùng trước báo cáo cuối không;
7. đầu ra có tuân ràng buộc logic của miền không;
8. chỉ số đánh giá có đồng bộ với mục tiêu học không.

Nếu một script không trả lời được trọn bộ tám câu này một cách minh bạch, thì chưa nên tin vào các chỉ số của nó, bất kể kiến trúc trông hiện đại đến đâu.

7.16 Kết luận chương

Chuỗi thời gian không chỉ là dữ liệu có thêm cột thời gian; nó là dữ liệu chịu ràng buộc thông tin. Dự báo đúng vì vậy đòi hỏi đồng thời ba tầng kỷ luật: kỷ luật trong định nghĩa biến mục tiêu, kỷ luật trong thiết kế giao thức thẩm định, và kỷ luật trong việc tôn trọng cấu trúc logic

của đầu ra. Những tầng kỷ luật này sẽ là nền để ta phán xét chính xác hơn mô hình ANFIS + BiLSTM ở phần sau.

Chuong 8

Cơ sở của logic mờ và tập mờ

8.1 Vì sao logic hai giá trị chưa đủ cho nhiều bài toán thật

Logic hai giá trị, hay logic kinh điển, là một trong những thành tựu đẹp nhất của toán học. Nó làm việc rất tốt khi biên giới của khái niệm là sắc nét: một số nguyên hoặc chẵn hoặc lẻ; một khóa điện hoặc bật hoặc tắt; một file hoặc tồn tại hoặc không tồn tại. Trong các tình huống ấy, việc gán cho mỗi mệnh đề một giá trị đúng hoặc sai là hoàn toàn tự nhiên. Ta có thể mô hình hóa một tập kinh điển $A \subseteq X$ bằng hàm đặc trưng

$$\chi_A(x) = \begin{cases} 1, & x \in A, \\ 0, & x \notin A. \end{cases}$$

Nếu đối tượng của ta thật sự có biên giới rạch ròi, cách mô tả này vừa gọn vừa mạnh.

Nhưng phần lớn khái niệm mà con người dùng trong điều khiển, tài chính, kinh tế học, và học máy lại không có biên giới như vậy. Ta nói "giá cao", "biến động mạnh", "mức rủi ro trung bình", "đầu ra tương đối ổn định", "tăng nhẹ", "giảm sâu". Những cụm này không hoàn toàn tùy tiện, nhưng cũng không có ranh giới sắc như một định lý số học. Nếu ép chúng về 0 hoặc 1 bằng một ngưỡng cứng, ta đã thay một khái niệm mềm bằng một quyết định cứng. Điều đó đôi khi cần thiết cho hệ thống ra quyết định cuối cùng, nhưng lại quá thô ở tầng mô hình hóa.

Chính chỗ này làm nảy sinh nhu cầu về logic mờ. Ý tưởng cốt lõi không phải là bác bỏ logic kinh điển, mà là mở rộng nó để mô tả những khái niệm có biên giới mềm. Một giá trị x có thể "thuộc" khái niệm A ở mức 0.2, 0.6, hay 0.95, tùy mức độ phù hợp của nó với ngữ nghĩa mà ta đang gán cho A. Đây là một ngôn ngữ toán học cho *mơ hồ của khái niệm*, chứ không phải cho *ngẫu nhiên của biến cố*.

Phân biệt này rất quan trọng. Nếu ngày mai trời mưa hay không là một biến cố ngẫu nhiên, xác suất là công cụ tự nhiên. Nhưng nếu ta muốn nói nhiệt độ 31 độ C "khá nóng" đến mức nào, thì thứ ta cần không phải xác suất 31 độ C sẽ nóng, mà là một mức độ phù hợp

của 31 với khái niệm "nóng". Logic mờ xuất hiện chính xác ở điểm này [11].

Trong bối cảnh học máy, logic mờ hấp dẫn vì hai lý do. Thứ nhất, nó cho phép ta đưa các nhãn ngôn ngữ như *thấp*, *vừa*, *cao* vào mô hình mà không phải giả vờ rằng ranh giới giữa chúng là tuyệt đối. Thứ hai, nó tạo ra một cây cầu để chuyển từ cách con người phát biểu quy tắc sang một cấu trúc toán học có thể tính toán được. Câu "nếu mức biến động cao và khoảng dao động lớn thì rủi ro tăng" không còn chỉ là câu chữ; nó có thể trở thành một luật mờ có tham số, có mức kích hoạt, và có thể ghép vào hệ suy diễn.

8.2 Tập mờ, hàm thuộc, miền đỡ và lõi

Định nghĩa 8.1. Cho một miền nền X. Một *tập mờ* A trên X là một ánh xạ

$$\mu_A : X \to [0, 1],$$

trong đó $\mu_A(x)$ được gọi là *hàm thuộc* của A. Giá trị $\mu_A(x)$ biểu diễn mức độ mà phần tử x phù hợp với khái niệm A.

Nếu $\mu_A(x) \in \{0, 1\}$ với mọi $x \in X$, ta thu lại đúng tập kinh điển. Vì vậy, tập mờ không phá hủy lý thuyết tập cổ điển; nó bao hàm lý thuyết đó như một trường hợp biên. Đó là một điểm sư phạm quan trọng: khi học logic mờ, ta không học một vũ trụ mới hoàn toàn tách biệt, mà học một sự mở rộng liên tục của điều mình đã biết.

Để nói về cấu trúc của một tập mờ, người ta thường dùng thêm vài khái niệm:

$$\text{supp}(A) = \{x \in X : \mu_A(x) > 0\}, \qquad \text{core}(A) = \{x \in X : \mu_A(x) = 1\}.$$

$\text{supp}(A)$ là *miền đỡ* của tập mờ, tức miền nơi khái niệm còn có chút liên quan. $\text{core}(A)$ là *lõi*, tức phần hoàn toàn phù hợp với khái niệm.

Người ta cũng định nghĩa *chiều cao* của tập mờ:

$$h(A) = \sup_{x \in X} \mu_A(x).$$

Nếu $h(A) = 1$, tập mờ được gọi là *chuẩn hóa* hay *normal*. Trực giác ở đây là: khái niệm ấy có ít nhất một vùng mà nó đạt mức phù hợp tối đa. Nhiều hệ suy diễn mờ giả định ngầm hoặc khuyến khích các tập mờ chuẩn hóa, vì nếu chiều cao thấp hơn 1 thì chính nhãn ngôn ngữ đã bị "yếu đi" ngay từ đầu.

Trong thực hành, hàm thuộc thường được chọn trong một họ tham số hóa đơn giản. Ba

dạng hay gặp nhất là tam giác, hình thang và Gaussian:

$$\mu_{\text{tri}}(x; a, b, c) = \begin{cases} 0, & x \le a, \\ \dfrac{x-a}{b-a}, & a < x \le b, \\ \dfrac{c-x}{c-b}, & b < x < c, \\ 0, & x \ge c, \end{cases}$$

$$\mu_{\text{trap}}(x; a, b, c, d) = \begin{cases} 0, & x \le a, \\ \dfrac{x-a}{b-a}, & a < x \le b, \\ 1, & b < x \le c, \\ \dfrac{d-x}{d-c}, & c < x < d, \\ 0, & x \ge d, \end{cases}$$

$$\mu_{\text{Gauss}}(x; c, \sigma) = \exp\left(-\frac{(x-c)^2}{2\sigma^2}\right).$$

Hàm tam giác và hình thang dễ trực quan hóa; Gaussian trơn, khả vi và đặc biệt thuận lợi cho tối ưu bằng gradient trong các mô hình lai như ANFIS. Không có một dạng nào luôn tốt nhất. Dạng hàm thuộc phải được chọn theo mục tiêu: ưu tiên khả năng diễn giải, ưu tiên tính trơn, hay ưu tiên ổn định trong tối ưu.

Nhận xét 8.2. Điều người mới hay ngộ nhận là xem hàm thuộc như một "xác suất" hoặc một "điểm tin cậy". Cách nói ấy rất dễ làm hỏng trực giác. Hàm thuộc không nói về tần suất xảy ra, mà nói về mức độ phù hợp với một khái niệm. Đây là hai lớp ý nghĩa khác nhau.

8.3 α-cut và định lý biểu diễn

Định nghĩa 8.3. Cho tập mờ A trên X và $\alpha \in (0, 1]$. α-cut của A là tập kinh điển

$$A_\alpha = \{x \in X : \mu_A(x) \ge \alpha\}.$$

Người ta cũng dùng *strong α-cut*:

$$A^\alpha = \{x \in X : \mu_A(x) > \alpha\}.$$

α-cut là một dụng cụ cực kỳ quan trọng vì nó biến một đối tượng mờ thành một họ tập kinh điển lồng nhau. Nói cách khác, thay vì nghĩ tập mờ như một bức tranh màu xám liên tục, ta có thể nghĩ nó như một chồng các "mặt cắt sắc nét" ứng với những mức khắt khe khác

nhau.

Mệnh đề 8.4. *Nếu* $0 < \alpha_1 \leq \alpha_2 \leq 1$ *thì*

$$A_{\alpha_2} \subseteq A_{\alpha_1}.$$

Proof. Lấy $x \in A_{\alpha_2}$. Theo định nghĩa, $\mu_A(x) \geq \alpha_2$. Vì $\alpha_2 \geq \alpha_1$, suy ra $\mu_A(x) \geq \alpha_1$, nên $x \in A_{\alpha_1}$. □

Tuy mệnh đề này đơn giản, nó có ý nghĩa lớn. Khi ngưỡng α tăng lên, ta đang trở nên "khắt khe" hơn với câu hỏi "phần tử này có thật sự thuộc khái niệm không". Vì vậy, tập phần tử được chấp nhận phải nhỏ dần. Đây là lý do người ta thường xem một tập mờ như một họ tập kinh điển lồng nhau theo mức độ nghiêm ngặt.

Mệnh đề 8.5. *Với mọi* $x \in X$,

$$\mu_A(x) = \sup\{\alpha \in (0, 1] : x \in A_\alpha\}.$$

Proof. Đặt

$$s(x) = \sup\{\alpha \in (0, 1] : x \in A_\alpha\}.$$

Theo định nghĩa của A_α, điều kiện $x \in A_\alpha$ tương đương $\mu_A(x) \geq \alpha$. Do đó tập lấy supremum chính là khoảng $(0, \mu_A(x)]$ nếu $\mu_A(x) > 0$, hoặc rỗng nếu $\mu_A(x) = 0$. Trong cả hai trường hợp, supremum đều bằng $\mu_A(x)$. Vậy $s(x) = \mu_A(x)$. □

Kết quả trên nói rằng toàn bộ hàm thuộc có thể được phục hồi từ họ α-cut. Đây chính là cơ sở của nhiều kỹ thuật tính toán trên số mờ: thay vì thao tác trực tiếp với hàm thuộc, ta thao tác trên các khoảng A_α, rồi ghép chúng lại.

8.4 Các phép toán trên tập mờ và vai trò của chuẩn tam giác

Trong tập kinh điển, hợp, giao và phủ định được mô tả bằng ba quy tắc rất quen:

$$\chi_{A\cap B}(x) = \min\{\chi_A(x), \chi_B(x)\}, \qquad \chi_{A\cup B}(x) = \max\{\chi_A(x), \chi_B(x)\}, \qquad \chi_{\bar{A}}(x) = 1-\chi_A(x).$$

Khi đi sang tập mờ, ta có thể mở rộng trực tiếp ba công thức này thành:

$$\mu_{A\cap B}(x) = \min\{\mu_A(x), \mu_B(x)\},$$

$$\mu_{A\cup B}(x) = \max\{\mu_A(x), \mu_B(x)\},$$

$$\mu_{\bar{A}}(x) = 1 - \mu_A(x).$$

Đây là bộ toán tử mờ cổ điển của Zadeh. Nó đơn giản, dễ hiểu và có liên hệ rất chặt với logic kinh điển.

Tuy nhiên, logic mờ không dừng ở một bộ toán tử duy nhất. Thay vì mặc định phép giao phải là min, ta tổng quát hóa bằng khái niệm *T-norm* (triangular norm, chuẩn tam giác).

Định nghĩa 8.6. Một T-norm là một ánh xạ $T : [0,1]^2 \to [0,1]$ thỏa bốn tính chất:

1. giao hoán: $T(a,b) = T(b,a)$;
2. kết hợp: $T(a, T(b,c)) = T(T(a,b), c)$;
3. đơn điệu: nếu $a_1 \le a_2$ và $b_1 \le b_2$ thì $T(a_1, b_1) \le T(a_2, b_2)$;
4. phần tử đơn vị: $T(a, 1) = a$.

Các ví dụ quan trọng là

$$T_{\min}(a,b) = \min(a,b), \qquad T_{\Pi}(a,b) = ab, \qquad T_{\text{Luk}}(a,b) = \max(a+b-1, 0).$$

Mỗi T-norm tạo ra một cách hiểu khác nhau của từ "và". Với $T_{\min}$, mệnh đề kết hợp đúng ở mức của mắt xích yếu nhất. Với T_{Π}, mức đúng bị giảm theo kiểu nhân, nên việc cùng lúc có nhiều điều kiện chỉ hơi yếu cũng sẽ làm sức kích hoạt giảm đáng kể. Với chuẩn Lukasiewicz, ta cho phép một kiểu bù trừ hạn chế giữa các mức đúng.

Tương tự, *S-norm* (còn gọi là T-conorm) là phiên bản tổng quát cho phép "hoặc". Các ví dụ quan trọng là

$$S_{\max}(a,b) = \max(a,b), \qquad S_{\text{prob}}(a,b) = a + b - ab, \qquad S_{\text{Luk}}(a,b) = \min(a+b, 1).$$

Nhận xét 8.7. Trong ANFIS và nhiều mạng mờ khả vi, người ta thường chọn $T_{\Pi}(a,b) = ab$, tức chuẩn tam giác kiểu tích. Lý do không chỉ vì nó quen thuộc, mà còn vì nó trơn trên miền trong và tương thích tốt với lan truyền ngược.

Với phủ định, lựa chọn phổ biến nhất là *phủ định chuẩn*

$$N(a) = 1 - a.$$

Khi đi cùng $T_{\min}$ và $S_{\max}$, nó thỏa các luật De Morgan theo đúng hình thức cổ điển.

Mệnh đề 8.8. *Với $N(a) = 1 - a$, $T_{\min}(a,b) = \min(a,b)$, và $S_{\max}(a,b) = \max(a,b)$, ta có*

$$N(T_{\min}(a,b)) = S_{\max}(N(a), N(b)),$$

$$N(S_{\max}(a,b)) = T_{\min}(N(a), N(b)).$$

Proof. Ta có

$$N(T_{\min}(a,b)) = 1 - \min(a,b) = \max(1-a, 1-b) = S_{\max}(N(a), N(b)).$$

Đẳng thức thứ hai chứng minh tương tự:

$$N(S_{\max}(a,b)) = 1 - \max(a,b) = \min(1-a, 1-b) = T_{\min}(N(a), N(b)).$$

□

Điều quan trọng ở đây không phải chỉ là vài công thức đại số. Khi chọn T-norm và S-norm, ta đang chọn *ngữ nghĩa tính toán* cho cách các mức độ đúng tương tác với nhau. Vì vậy, nói "dùng min hay tích cũng được" là một phát biểu quá hời hợt. Hai lựa chọn đó có thể dẫn tới hành vi suy diễn khác nhau rõ rệt.

8.5 Biến ngôn ngữ, phân hoạch mờ và quá trình mờ hóa

Biến ngôn ngữ (linguistic variable) là một biến mà các "giá trị" của nó không phải chỉ là số, mà là những nhãn ngôn ngữ như *thấp*, *trung bình*, *cao*. Mỗi nhãn ấy được gắn với một tập mờ trên miền giá trị của biến. Ví dụ, với biến "mức biến động ngày", ta có thể tạo ba nhãn

$$\text{THAP}, \qquad \text{TRUNG BINH}, \qquad \text{CAO}.$$

Mỗi nhãn là một hàm thuộc khác nhau trên miền giá trị của tỷ suất biến động.

Quá trình biến đổi một giá trị số sắc nét x thành các mức độ thuộc $\mu_{\text{THAP}}(x)$, $\mu_{\text{TRUNG BINH}}(x)$, $\mu_{\text{CAO}}(x)$ được gọi là *mờ hóa* (fuzzification). Ta không mất dữ liệu gốc; ta chỉ biểu diễn lại dữ liệu trong một ngôn ngữ khác, gần với cách con người suy nghĩ hơn.

Một khái niệm đặc biệt quan trọng là *phân hoạch mờ* (fuzzy partition). Trên trực giác, đó là một họ nhãn mờ phủ lên miền giá trị sao cho mọi điểm đều được mô tả bởi một hoặc nhiều nhãn. Trong nhiều tài liệu, người ta thích một tính chất mạnh hơn, gọi là *partition of unity* (phân hoạch có tổng bằng 1):

$$\sum_{k=1}^{m} \mu_{A_k}(x) = 1, \qquad \forall x \in X.$$

Tính chất này không bắt buộc trong mọi hệ mờ, nhưng nó rất hữu ích vì giúp các nhãn hoạt động như một bộ tọa độ mềm trên miền giá trị.

Người mới thường nghĩ rằng chỉ cần đặt vài hàm thuộc lên trục số là xong. Trên thực tế, một phân hoạch mờ tốt phải cân bằng ít nhất ba điều:

1. **độ phủ**: miền giá trị quan trọng phải thật sự được các nhãn chạm tới;
2. **độ chồng lấp**: các nhãn lân cận phải giao thoa vừa đủ để suy luận mượt, nhưng không được chồng quá mức đến mức mất phân biệt;
3. **tính diễn giải**: vị trí và độ rộng của các nhãn phải còn gắn với ngôn ngữ mà ta dùng.

Nếu hàm thuộc quá hẹp, hệ thống gần như quay về phân loại cứng. Nếu quá rộng, mọi luật đều kích hoạt một chút và mô hình mất khả năng phân biệt.

Trong các mô hình học được tham số, như ANFIS, vấn đề còn sâu hơn. Ban đầu ta có thể gọi hai nhãn là *LOW* và *HIGH*, nhưng nếu tâm của chúng bị đổi chỗ trong quá trình tối ưu mà không có ràng buộc thứ tự, thì nhãn ngôn ngữ sẽ mất nghĩa. Điều này sẽ trở lại như một điểm then chốt khi ta phân tích script ở chương sau.

8.6 Quan hệ mờ, phép hợp thành và suy luận nếu–thì

Cho hai miền X và Y. Một *quan hệ mờ* R từ X sang Y là một tập mờ trên tích Descartes $X \times Y$, tức là một hàm

$$\mu_R : X \times Y \to [0, 1].$$

Nếu R mã hóa mệnh đề "nếu x lớn thì y cũng lớn", thì $\mu_R(x, y)$ đo mức độ mà cặp (x, y) phù hợp với mệnh đề đó.

Khi có hai quan hệ mờ $R \subseteq X \times Y$ và $S \subseteq Y \times Z$, ta có thể ghép chúng bằng *phép hợp thành cực đại–cực tiểu* (max–min composition):

$$\mu_{R \circ S}(x, z) = \sup_{y \in Y} \min\{\mu_R(x, y), \mu_S(y, z)\}.$$

Nếu thay min bằng một T-norm khác, ta thu được các họ hợp thành khác. Đây là khung nhìn quan hệ học của suy luận mờ.

Ở tầng trực giác, công thức trên nói rằng: mức độ mà x liên hệ với z thông qua trung gian y được đo bằng cách, với từng y, lấy mắt xích yếu hơn trong hai quan hệ; sau đó chọn con đường mạnh nhất trong số mọi y. Điều này rất gần với trực giác "suy luận dây chuyền".

Trong nhiều hệ thực hành, người ta không triển khai suy luận quan hệ một cách tổng quát như vậy, mà làm việc trực tiếp với các luật mờ dạng nếu–thì:

$$\text{IF } x_1 \text{ is } A_1 \text{ AND } \cdots \text{ AND } x_d \text{ is } A_d \text{ THEN } y \text{ is } B.$$

Hoặc trong hệ Sugeno:

$$\text{IF } x_1 \text{ is } A_1 \text{ AND } \cdots \text{ AND } x_d \text{ is } A_d \text{ THEN } y = a_0 + a_1 x_1 + \cdots + a_d x_d.$$

Khác biệt này rất quan trọng. Luật Mamdani có hệ quả là một tập mờ ngôn ngữ; luật Sugeno có hệ quả là một hàm số, thường tuyến tính. Vì thế Sugeno dễ ghép với tối ưu tham số hơn, còn Mamdani tự nhiên hơn về mặt diễn đạt ngôn ngữ.

8.7 Nguyên lý mở rộng và cách lan truyền độ mờ qua phép biến đổi

Nếu $f : X \to Y$ là một ánh xạ kinh điển và A là một tập mờ trên X, thì *nguyên lý mở rộng* (extension principle) của Zadeh cho phép định nghĩa ảnh mờ của A qua f:

$$\mu_{f(A)}(y) = \sup_{x:f(x)=y} \mu_A(x).$$

Đây là một công thức nền tảng. Nó nói rằng mức độ mà y thuộc ảnh của A được lấy bằng mức phù hợp lớn nhất trong số các điểm x ánh xạ tới y.

Tại sao công thức này quan trọng? Vì nó cho phép ta đưa các phép biến đổi thông thường của toán học sang thế giới mờ mà không phải định nghĩa lại từng phép một từ đầu. Nếu $f(x) = 2x + 1$, hoặc $f(x_1, x_2) = x_1 + x_2$, hoặc $f(x) = x^2$, nguyên lý mở rộng cho ta một cách có hệ thống để lan truyền độ mờ qua f.

Nhược điểm là công thức tổng quát này có thể khó tính toán trực tiếp. Đó là lý do α-cut trở nên hữu ích. Với nhiều lớp số mờ trên $\mathbb{R}$, đặc biệt là số mờ lồi và chuẩn hóa, ta có thể thực hiện phép toán bằng cách thao tác trên các khoảng của α-cut. Chẳng hạn, nếu $A_\alpha = [a_\alpha^-, a_\alpha^+]$ và $B_\alpha = [b_\alpha^-, b_\alpha^+]$, thì

$$(A + B)_\alpha = [a_\alpha^- + b_\alpha^-,\, a_\alpha^+ + b_\alpha^+].$$

Kết quả này rất đẹp vì nó biến số mờ thành một họ khoảng và biến phép cộng mờ thành phép cộng khoảng.

Nhận xét 8.9. Từ góc nhìn tính toán, α-cut giống như một ngôn ngữ trung gian giữa tập mờ và giải tích khoảng. Đây là một trong những lý do khiến nhiều chứng minh trong lý thuyết số mờ được trình bày qua các mặt cắt α thay vì trực tiếp qua hàm thuộc.

8.8 Số mờ, tính lồi và số học mờ

Định nghĩa 8.10. Một *số mờ* trên $\mathbb{R}$ là một tập mờ A sao cho:

1. A chuẩn hóa, tức $\sup_x \mu_A(x) = 1$;
2. A lồi theo nghĩa mờ, tức

$$\mu_A(\lambda x + (1 - \lambda)y) \geq \min\{\mu_A(x), \mu_A(y)\}, \qquad \forall x, y \in \mathbb{R},\ \lambda \in [0, 1];$$

3. μ_A nửa liên tục trên;
4. miền đỡ của A là compact.

Điều kiện lồi theo nghĩa mờ bảo đảm rằng mọi α-cut của số mờ là một khoảng đóng. Đây là điểm làm số mờ trở nên tính toán được. Nếu các α-cut là những tập rời rạc hoặc quá dị thường, việc tính toán sẽ rất khó kiểm soát.

Hai ví dụ kinh điển là số mờ tam giác và số mờ hình thang. Chúng được ưa dùng trong các hệ chuyên gia và điều khiển cổ điển vì dễ khai báo bằng tay. Trong khi đó, số mờ Gaussian không phải lúc nào cũng có miền đỡ compact theo nghĩa chặt, nhưng lại rất hấp dẫn về mặt khả vi. Vì vậy, trong học máy, ta thường chấp nhận đánh đổi: bỏ một chút sạch sẽ của định nghĩa cổ điển để đổi lấy thuận lợi tối ưu.

Số học mờ cho phép cộng, trừ, nhân, chia các đại lượng không sắc nét. Tuy nhiên, người học cần tránh một ngộ nhận phổ biến: số mờ không phải là "số có sai số ngẫu nhiên". Nó gần hơn với việc "một đại lượng chưa được chốt sắc nét về khái niệm hoặc về nhận thức". Có những ngữ cảnh mà xác suất thích hợp hơn; có những ngữ cảnh mà logic mờ thích hợp hơn; và cũng có những ngữ cảnh cần cả hai.

8.9 Logic mờ và xác suất: hai loại bất định khác nhau

Đây là chỗ người mới nhầm nhiều nhất, và nếu nhầm ở đây thì về sau rất dễ dùng sai mô hình. Xác suất mô tả mức độ tin vào một biến cố trong một không gian ngẫu nhiên. Logic mờ mô tả mức độ phù hợp của một đối tượng với một khái niệm có biên giới mềm. Một bên nói về *liệu điều gì sẽ xảy ra*; bên kia nói về *đối tượng hiện tại giống khái niệm đến mức nào*.

Ví dụ, câu "khả năng ngày mai mưa là 0.8" là phát biểu xác suất. Câu "nhiệt độ 31 độ C thuộc khái niệm 'nóng' ở mức 0.8" là phát biểu mờ. Hai câu đều dùng số 0.8, nhưng ý nghĩa của con số hoàn toàn khác. Nếu ta trộn chúng vào nhau chỉ vì cùng nằm trong đoạn $[0, 1]$, ta sẽ suy luận sai.

Sự khác nhau còn thể hiện ở câu hỏi nền. Xác suất hỏi: *biến cố nào xảy ra trong thế giới ngẫu nhiên?* Logic mờ hỏi: *khi một đối tượng đã quan sát được, nó phù hợp với nhãn ngôn ngữ đến mức nào?* Vì vậy, một đại lượng có thể rất mờ nhưng không ngẫu nhiên, và một biến cố có thể rất ngẫu nhiên nhưng không hề mờ.

Điều đó không có nghĩa hai lý thuyết tách biệt hoàn toàn. Có những khung như *possibility theory* (lý thuyết khả năng), có những mô hình xác suất–mờ, và có những hệ mờ xác suất. Nhưng để kết hợp được chúng một cách nghiêm túc, trước hết ta phải giữ ranh giới khái niệm rõ ràng. Nói "membership 0.8 tức là xác suất 80%" gần như luôn là sai.

8.10 Giải mờ và quyết định cuối cùng

Nhiều hệ suy diễn mờ, đặc biệt là hệ Mamdani, sau khi tổng hợp các luật sẽ cho ra một tập mờ trên miền đầu ra. Muốn ra quyết định cuối cùng, ta cần chuyển đối tượng mờ ấy thành

một giá trị sắc nét. Quá trình đó gọi là *giải mờ* (defuzzification).

Phương pháp nổi tiếng nhất là *trọng tâm* (centroid):

$$y^\star = \frac{\int y\,\mu_B(y)\,dy}{\int \mu_B(y)\,dy},$$

với B là tập mờ đầu ra sau khi đã gộp các luật. Công thức này có trực giác khá đẹp: nó lấy "trọng tâm khối lượng" của hình do hàm thuộc đầu ra tạo ra. Tuy nhiên, tính toán có thể đắt nếu miền đầu ra phức tạp hoặc liên tục nhiều chiều.

Các lựa chọn khác như *mean of maxima* (trung bình các điểm cực đại), *smallest of maxima*, *largest of maxima* đơn giản hơn, nhưng lại nhạy cảm hơn với hình dạng đầu ra. Không có một cách giải mờ nào luôn thắng tuyệt đối; mỗi cách phản ánh một triết lý quyết định khác nhau.

Trong hệ Sugeno, tình hình thuận lợi hơn. Nếu mỗi luật có hệ quả là một hằng số hoặc một hàm tuyến tính, đầu ra cuối thường có dạng trung bình có trọng số:

$$y(x) = \frac{\sum_{i=1}^{R} w_i(x) f_i(x)}{\sum_{i=1}^{R} w_i(x)}.$$

Vì vậy, bước giải mờ được tích hợp gần như ngay trong cấu trúc mô hình. Đây là lý do hệ Sugeno và ANFIS đặc biệt hợp với bài toán học tham số.

8.11 Tính diễn giải không tự sinh ra từ việc gắn nhãn LOW, HIGH

Trong văn liệu ứng dụng, người ta rất dễ dùng từ "interpretable" hay "giải thích được" khi thấy mô hình có luật mờ. Nhưng tính diễn giải là một thuộc tính khó hơn nhiều. Một hệ mờ chỉ thật sự dễ đọc khi ít nhất bốn điều kiện được giữ:

1. nhãn ngôn ngữ tương ứng với vị trí có ý nghĩa trên trục giá trị;
2. các hàm thuộc đủ ít để con người còn theo dõi được;
3. các luật không quá nhiều đến mức việc đọc luật trở thành hình thức;
4. hệ quả của luật cũng được diễn đạt minh bạch, không chỉ tiền đề.

Điều kiện thứ nhất đặc biệt dễ bị phá trong các mô hình học được tham số. Nếu hai Gaussian được khởi tạo là "thấp" và "cao" nhưng sau huấn luyện tâm bị đảo ngược, nhãn LOW/HIGH chỉ còn là nhãn chỉ số chứ không còn là nhãn ngôn ngữ. Khi ấy, in ra một luật như "IF return is LOW" có thể tạo ảo giác diễn giải trong khi ngữ nghĩa thật đã trượt khỏi chữ LOW.

Điều kiện thứ ba cũng đáng lưu ý. Thêm nhiều hàm thuộc tưởng như giúp mô tả tinh hơn, nhưng số luật trong hệ lưới tăng theo cấp số mũ. Với d đặc trưng và m hàm thuộc mỗi đặc trưng, số luật là m^d. Đây không phải chi tiết phụ; nó là ràng buộc cấu trúc của toàn bộ họ mô hình. Một mô hình có 6 đặc trưng và 3 hàm thuộc mỗi đặc trưng đã có $3^6 = 729$ luật nếu dựng lưới đầy đủ. Con số ấy đã vượt xa khả năng đọc luật thủ công trong hầu hết bài toán thực.

8.12 Bài học chuyển tiếp sang ANFIS

Sau chương này, có ba ý tưởng mà ta cần mang theo sang chương về hệ suy diễn mờ và ANFIS.

Thứ nhất, tập mờ là một ngôn ngữ cho *độ phù hợp khái niệm*, không phải cho xác suất. Nếu quên điều đó, ta sẽ hiểu sai ý nghĩa của mọi hàm thuộc.

Thứ hai, T-norm, S-norm, phân hoạch mờ, và α-cut không phải các trang trí lý thuyết. Chúng quyết định cả ngữ nghĩa lẫn tính toán của hệ mờ. Một mô hình mờ tốt không thể được xây chỉ bằng cách đặt bừa vài Gaussian lên trục số.

Thứ ba, tính diễn giải của một hệ mờ là một thành tựu mong manh. Nó cần được xây và được bảo vệ bằng cấu trúc của mô hình, chứ không tự sinh ra từ việc ta gắn cho hai đường cong cái tên *LOW* và *HIGH*. Bài học này sẽ trở nên đặc biệt quan trọng khi ta bước sang ANFIS và sau đó đọc một script cụ thể có quảng bá khả năng diễn giải.

Chuong 9

Hệ suy diễn mờ: từ Mamdani đến Takagi–Sugeno

9.1 Luật nếu–thì là cách biến trực giác thành cấu trúc toán học

Khi con người mô tả một hiện tượng phức tạp, họ thường không bắt đầu bằng hệ phương trình. Họ bắt đầu bằng những câu kiểu:

> Nếu biên độ biến động cao và khoảng nhảy mở cửa lớn, thì thị trường ở trạng thái bất ổn.

Những câu như vậy có giá trị vì chúng gắn mô hình với ngôn ngữ chuyên gia. Nhưng để dùng trong tính toán, ta cần biến chúng thành một cơ chế hình thức. Hệ suy diễn mờ làm đúng việc đó: biến các luật nếu–thì có chứa khái niệm mềm thành một bản đồ toán học từ đầu vào sang đầu ra.

Một luật mờ tổng quát có dạng

$$R_r: \quad \text{Nếu } x_1 \text{ là } A_1^r, \ldots, x_d \text{ là } A_d^r \text{ thì } y \text{ là } B^r.$$

Ở đây:

1. A_j^r là các tập mờ ở phần tiền đề;
2. B^r là đối tượng ở phần hệ quả;
3. chỉ số r đánh số luật.

Toàn bộ hệ suy diễn mờ phải trả lời bốn câu hỏi:

1. làm mờ hóa đầu vào thế nào;

2. kết hợp mức độ đúng của các tiền đề ra sao;
3. ánh xạ từ tiền đề sang hệ quả bằng phép suy diễn nào;
4. giải mờ thế nào để ra đầu ra sắc nét.

9.2 Làm mờ hóa như một phép trích xuất đặc trưng

Giả sử đầu vào là $\boldsymbol{x} = (x_1, \ldots, x_d)$. Với mỗi biến x_j, ta gán một họ nhãn ngôn ngữ như $\{\text{LOW}, \text{MEDIUM}, \text{HIGH}\}$, và mỗi nhãn tương ứng với một hàm thuộc:

$$\mu_{A_{j1}}(x_j),\ \mu_{A_{j2}}(x_j),\ \ldots.$$

Về mặt tính toán, đây là một phép biến đổi phi tuyến từng tọa độ. Về mặt ngữ nghĩa, đây là phép trả lời câu hỏi: "giá trị quan sát hiện tại thuộc vào khái niệm này ở mức độ nào?"

Đây là điểm rất đáng lưu ý: phần làm mờ hóa của hệ mờ có thể được xem như một tầng biểu diễn. Nó không khác hoàn toàn tư duy của học sâu; chỉ khác ở chỗ biểu diễn được buộc phải đi qua những khái niệm ngôn ngữ mà con người đặt tên.

9.3 Độ kích hoạt của luật và ý nghĩa của phép "và"

Khi một luật có nhiều điều kiện tiền đề, ta phải quyết định cách kết hợp chúng. Nếu luật nói

$$\text{Nếu } x_1 \text{ là } A_1 \text{ và } x_2 \text{ là } A_2,$$

thì từ hai mức độ thuộc $\mu_{A_1}(x_1)$ và $\mu_{A_2}(x_2)$, ta cần tạo một độ kích hoạt duy nhất w.

9.3.1 Hai lựa chọn kinh điển

Hai cách phổ biến nhất là:

$$w = \min\big(\mu_{A_1}(x_1), \mu_{A_2}(x_2)\big),$$

hoặc

$$w = \mu_{A_1}(x_1)\mu_{A_2}(x_2).$$

Phép min diễn tả trực giác "mắt xích yếu nhất quyết định độ đúng của mệnh đề hợp". Phép tích diễn tả trực giác "mỗi điều kiện đều làm co mức độ kích hoạt theo kiểu nhân". Trong ANFIS, phép tích đặc biệt hấp dẫn vì nó trơn, khả vi, và phù hợp với huấn luyện bằng gradient.

9.3.2 Một ví dụ so sánh

Giả sử

$$\mu_{A_1}(x_1) = 0.8, \qquad \mu_{A_2}(x_2) = 0.6.$$

Nếu dùng min, ta được $w = 0.6$. Nếu dùng tích, ta được $w = 0.48$. Phép tích trừng phạt mạnh hơn việc nhiều điều kiện chỉ ở mức trung bình. Điều này cho thấy lựa chọn T-norm không phải một tiểu tiết kỹ thuật; nó thay đổi tính khí của cả hệ suy diễn.

9.4 Phép kéo theo mờ và vai trò của nó trong suy diễn

Trong logic kinh điển, mệnh đề "nếu P thì Q" có thể được biểu diễn qua bảng chân trị. Trong logic mờ, câu chuyện phức tạp hơn. Ta cần một *fuzzy implication* (phép kéo theo mờ) để kết nối mức độ đúng của tiền đề với mức độ đúng của hệ quả.

Có nhiều họ phép kéo theo mờ đã được đề xuất. Tuy nhiên, trong các hệ suy diễn ứng dụng, người ta thường không triển khai ở mức trừu tượng của logic kéo theo, mà đi trực tiếp vào cơ chế tính toán của Mamdani hoặc Takagi–Sugeno. Dù vậy, việc biết rằng đằng sau hệ suy diễn có một lớp logic như vậy giúp ta tránh nhầm tưởng rằng mọi cách "gắn hệ quả" đều là như nhau.

9.5 Hệ Mamdani: khi vế hệ quả vẫn là ngôn ngữ

Trong hệ Mamdani, mỗi luật có dạng

$$\text{Nếu } x_1 \text{ là } A_1^r, \ldots, x_d \text{ là } A_d^r \text{ thì } y \text{ là } B^r,$$

trong đó B^r là một tập mờ trên miền đầu ra.

9.5.1 Ba bước của Mamdani

Sau khi tính độ kích hoạt w_r, hệ Mamdani thường thực hiện:

1. **kéo theo cục bộ**: cắt hoặc co giãn tập mờ hệ quả B^r theo w_r;
2. **tổng hợp toàn cục**: hợp tất cả các hệ quả đã được kích hoạt;
3. **giải mờ**: chuyển tập mờ đầu ra thành một số sắc nét.

Nếu dùng phép cắt kiểu Mamdani, ta có

$$\mu_{B'_r}(y) = \min(w_r, \mu_{B^r}(y)).$$

Sau đó, nếu dùng hợp kiểu cực đại,

$$\mu_B(y) = \max_r \mu_{B'_r}(y).$$

Cuối cùng, một cách giải mờ kinh điển là trọng tâm:

$$y^\star = \frac{\int y\, \mu_B(y)\, dy}{\int \mu_B(y)\, dy}.$$

9.5.2 Vì sao Mamdani hấp dẫn

Điểm mạnh lớn nhất của Mamdani là tính ngôn ngữ. Cả đầu vào lẫn đầu ra đều có thể được diễn tả bằng nhãn mờ. Với các bài toán điều khiển hoặc chuyên gia, điều này giúp hệ thống rất gần với cách con người mô tả tri thức.

9.5.3 Giới hạn của Mamdani

Chính sự giàu ngôn ngữ ấy làm cho Mamdani kém thuận lợi hơn trong học tham số. Đầu ra là một tập mờ, nên quá trình giải mờ và tối ưu hóa trở nên nặng hơn. Đây là lý do các hệ neuro-fuzzy học bằng dữ liệu thường ưu tiên Sugeno hơn Mamdani.

9.6 Hệ Takagi–Sugeno–Kang: khi hệ quả là mô hình cục bộ sắc nét

Trong hệ Takagi–Sugeno–Kang, một luật có dạng

$$R_r: \quad \text{Nếu } x_1 \text{ là } A_1^r, \ldots, x_d \text{ là } A_d^r \text{ thì } y_r = p_{r0} + \sum_{j=1}^{d} p_{rj} x_j.$$

Hệ quả không còn là tập mờ trên miền đầu ra, mà là một hàm sắc nét của đầu vào. Điều này thay đổi bản chất của hệ suy diễn theo hướng cực kỳ quan trọng.

9.6.1 Đầu ra như tổ hợp của các mô hình cục bộ

Nếu w_r là độ kích hoạt của luật thứ r, đầu ra cuối là

$$y(\boldsymbol{x}) = \frac{\sum_{r=1}^{R} w_r(\boldsymbol{x})\, y_r(\boldsymbol{x})}{\sum_{r=1}^{R} w_r(\boldsymbol{x})}.$$

Đặt

$$\bar{w}_r(\boldsymbol{x}) = \frac{w_r(\boldsymbol{x})}{\sum_{s=1}^{R} w_s(\boldsymbol{x})},$$

ta viết được

$$y(\boldsymbol{x}) = \sum_{r=1}^{R} \bar{w}_r(\boldsymbol{x})\, y_r(\boldsymbol{x}).$$

Đây là một công thức rất sâu. Nó nói rằng hệ Sugeno là một tổ hợp có trọng số của nhiều mô hình cục bộ, trong đó trọng số phụ thuộc vào vị trí hiện tại của đầu vào trong không gian đặc trưng.

9.6.2 Hệ Sugeno như một phân hoạch đơn vị

Vì

$$\sum_{r=1}^{R} \bar{w}_r(\boldsymbol{x}) = 1, \qquad \bar{w}_r(\boldsymbol{x}) \geq 0,$$

các $\bar{w}_r$ tạo thành một *partition of unity* (phân hoạch đơn vị) trên miền đầu vào. Điều này cho phép ta nhìn Sugeno như một cơ chế ghép mềm các mô hình cục bộ affine. Đây là cầu nối rất đẹp giữa logic mờ và mô hình hóa cục bộ trong giải tích số, tối ưu và điều khiển.

9.7 TSK bậc 0 và bậc 1: khác nhau không chỉ ở số tham số

Nếu hệ quả chỉ là hằng số

$$y_r = c_r,$$

ta có TSK bậc 0. Nếu hệ quả là affine,

$$y_r = p_{r0} + \sum_{j} p_{rj} x_j,$$

ta có TSK bậc 1.

9.7.1 TSK bậc 0

TSK bậc 0 gần với tinh thần của các luật mờ cổ điển hơn. Mỗi luật đóng góp một giá trị đại diện. Hệ này thường dễ diễn giải hơn, nhưng khả năng xấp xỉ có thể kém hơn khi quan hệ đầu vào–đầu ra biến đổi mượt trong từng vùng.

9.7.2 TSK bậc 1

TSK bậc 1 cho phép mỗi vùng mờ có một mô hình tuyến tính cục bộ riêng. Điều này làm hệ linh hoạt hơn rất nhiều. Chính vì vậy, ANFIS kinh điển của Jang [6] sử dụng hệ Sugeno bậc 1.

9.8 Cơ sở luật được sinh ra như thế nào

Muốn xây một hệ suy diễn mờ, ta cần một cơ sở luật. Đây là điểm mà nhiều người mới thường lướt qua quá nhanh. Nhưng trong thực tế, cách sinh luật quyết định mạnh đến chất lượng và tính diễn giải của hệ.

9.8.1 Phân hoạch lưới toàn phần

Nếu mỗi đặc trưng có m nhãn mờ và có d đặc trưng, phân hoạch lưới sinh ra tối đa m^d luật. Cách này có một ưu điểm lớn: rõ ràng và có tính hệ thống. Tuy nhiên, nó bị bùng nổ số luật rất nhanh.

Ví dụ, chỉ với $d = 6$ và $m = 2$, ta đã có

$$2^6 = 64$$

luật. Với $m = 3$, con số thành

$$3^6 = 729.$$

Đây là lý do không thể nói về hệ mờ mà không đồng thời nói về lời nguyền số chiều.

9.8.2 Sinh luật dựa trên phân cụm

Một cách khác là dùng cấu trúc dữ liệu để sinh luật. Ta phân cụm dữ liệu, rồi mỗi cụm tạo ra một luật hoặc một trung tâm của vùng mờ. Cách này giảm số luật và làm cơ sở luật bám dữ liệu hơn. Đổi lại, luật thu được có thể kém "ngôn ngữ" hơn, vì chúng không còn đến từ một lưới khái niệm đều đặn.

9.8.3 Luật do chuyên gia cung cấp

Trong các hệ thống điều khiển hoặc chuyên gia, luật có thể đến trực tiếp từ người thiết kế. Đây là lựa chọn có giá trị lớn về mặt diễn giải, nhưng chất lượng dự báo phụ thuộc mạnh vào chất lượng tri thức chuyên gia và mức độ bao phủ của bộ luật.

9.9 Sự đánh đổi giữa độ phủ và tính phân biệt

Một cơ sở luật tốt phải cùng lúc thỏa hai yêu cầu:

1. mọi quan sát hợp lệ đều được ít nhất một số luật phủ tới;
2. các luật phải phân biệt đủ mạnh để không ai cũng kích hoạt giống nhau.

Nếu các hàm thuộc quá rộng, mọi luật đều kích hoạt một ít, và hệ mất khả năng phân biệt. Nếu các hàm thuộc quá hẹp, nhiều quan sát rơi vào vùng độ kích hoạt rất nhỏ, làm hệ thiếu ổn định và khó học. Đây là một biểu hiện khác của bài toán thiên kiến quy nạp trong hệ mờ.

9.10 Hệ mờ như một mô hình chuyên gia cục bộ

Có một cách nhìn rất hữu ích về hệ Sugeno: đó là một mô hình *mixture-of-experts* theo nghĩa mềm. Các luật đóng vai trò chuyên gia cục bộ; các hàm thuộc và độ kích hoạt đóng vai trò cơ chế cổng. Điều này cho phép ta nối tư duy fuzzy với nhiều ý tưởng hiện đại trong học máy.

Nhưng ta cũng phải thấy điểm khác. Trong một *mixture-of-experts* thông thường, cổng điều phối không nhất thiết phải mang nghĩa ngôn ngữ. Trong hệ mờ, điều phối được mong đợi là có ý nghĩa khái niệm như LOW, HIGH, MEDIUM. Chính kỳ vọng này làm cho bài toán diễn giải của hệ mờ vừa hấp dẫn vừa dễ bị tổn thương.

9.11 Tính nhận dạng và sự mong manh của nhãn ngôn ngữ

Một hệ mờ có thể có nhiều cấu hình tham số khác nhau tạo ra hành vi gần giống nhau. Hai hàm thuộc có thể đổi chỗ, độ rộng có thể nở ra để bù cho hệ số hệ quả, hoặc hai luật có thể gần như trùng lặp. Vì vậy, việc đọc nhãn LOW/HIGH từ một mô hình đã huấn luyện không tự động bảo đảm ta đang đọc đúng nghĩa khái niệm ban đầu.

Đây là điểm cực kỳ quan trọng cho ANFIS. Nếu tâm của hàm LOW sau huấn luyện vượt qua tâm của hàm HIGH, thì việc vẫn gọi nó là LOW là một ngộ nhận. Nói cách khác, hệ mờ chỉ giữ được sức mạnh diễn giải nếu ta quản trị được tính nhận dạng của các hàm thuộc.

9.12 Diễn giải và cái giá của diễn giải

Người ta hay nói rằng hệ mờ “dễ giải thích”. Câu này đúng, nhưng thiếu điều kiện đi kèm. Một hệ mờ chỉ thật sự dễ giải thích khi:

1. số luật đủ ít để con người có thể đọc;
2. mỗi luật có tiền đề mang nghĩa ngôn ngữ ổn định;
3. hệ quả của luật cũng có thể đọc và hiểu;
4. đầu ra cuối không bị pha trộn quá mạnh bởi một cấu trúc bên ngoài hệ luật.

Nếu chỉ thỏa điều kiện thứ nhất mà không thỏa các điều kiện còn lại, ta chỉ có một hệ trông giống dễ giải thích. Đây là khác biệt giữa *appearance of interpretability* (bề ngoài của tính diễn giải) và *substantive interpretability* (tính diễn giải có thực chất).

9.13 Lỗi mà các chỉ số đánh giá không nhìn thấy

Một hệ suy diễn mờ có thể đạt MSE thấp nhưng vẫn là một mô hình xấu theo nhiều nghĩa:

1. các luật không còn mang nghĩa ngôn ngữ;
2. các vùng mờ chồng lấp vô tổ chức;
3. đầu ra vi phạm các ràng buộc logic của bài toán;
4. hệ chỉ học tốt trên một giao thức đánh giá có rò rỉ thông tin.

Điều này dạy ta một nguyên lý lớn: chỉ số dự báo tốt là cần, nhưng không đủ để biện minh cho chất lượng của một hệ mờ.

9.14 Từ hệ suy diễn mờ đến ANFIS

Khi hệ Sugeno được viết dưới dạng công thức trơn, khả vi, và có hệ quả affine, ta có thể đưa nó vào một khung học tham số bằng dữ liệu. Chính tại điểm này, ANFIS xuất hiện như một cây cầu nối: giữ cấu trúc của hệ mờ, nhưng học tham số bằng các công cụ của tối ưu hiện đại. Chương tiếp theo sẽ phân tích kỹ cây cầu đó, cùng những gì nó giữ được và những gì nó có thể đánh mất.

Chuong 10

ANFIS: kiến trúc, học tham số và những giới hạn

10.1 ANFIS là gì nếu nói thật chặt

Tên đầy đủ của ANFIS là *Adaptive-Network-Based Fuzzy Inference System*, nghĩa là một hệ suy diễn mờ được biểu diễn dưới dạng mạng thích nghi. Cách nói này nghe khá rộng, nhưng với ANFIS kinh điển của Jang [6], ta có thể phát biểu chặt hơn:

> ANFIS là một cách tham số hóa hệ Sugeno bậc 1 sao cho các tham số tiền đề và tham số hệ quả có thể được học từ dữ liệu bằng sự kết hợp giữa bình phương tối thiểu và hạ dốc theo gradient.

Phát biểu này quan trọng vì nó bóc bỏ hai ngộ nhận phổ biến.

1. ANFIS không đơn giản chỉ là “fuzzy logic cộng với neural network” như một khẩu hiệu.

2. ANFIS không phải bất kỳ mô hình nào có một lớp mờ ở đâu đó trong mạng.

Nếu một mô hình chỉ dùng các hàm thuộc để tạo đặc trưng rồi đẩy qua một khối học sâu không còn giữ cấu trúc Sugeno, thì đó có thể là một mô hình lai fuzzy–deep, nhưng không còn là ANFIS theo nghĩa cổ điển nữa.

10.2 Mẫu hai đầu vào, hai luật: nơi mọi công thức của ANFIS lộ ra rõ nhất

Hãy xét trường hợp đơn giản nhất với hai đầu vào x, y và hai luật:

$$\begin{aligned} R_1: &\quad \text{Nếu } x \text{ là } A_1 \text{ và } y \text{ là } B_1 \text{ thì } f_1 = p_1 x + q_1 y + r_1, \\ R_2: &\quad \text{Nếu } x \text{ là } A_2 \text{ và } y \text{ là } B_2 \text{ thì } f_2 = p_2 x + q_2 y + r_2. \end{aligned}$$

Đặt

$$w_1 = \mu_{A_1}(x)\mu_{B_1}(y), \qquad w_2 = \mu_{A_2}(x)\mu_{B_2}(y).$$

Sau chuẩn hóa:

$$\bar{w}_1 = \frac{w_1}{w_1 + w_2}, \qquad \bar{w}_2 = \frac{w_2}{w_1 + w_2}.$$

Đầu ra cuối là

$$f = \bar{w}_1 f_1 + \bar{w}_2 f_2.$$

Từ ví dụ nhỏ này, toàn bộ bản chất của ANFIS hiện ra:

1. phần mờ nằm trong các μ và w_r;
2. phần tuyến tính cục bộ nằm trong f_r;
3. phần phi tuyến toàn cục xuất hiện do trọng số $\bar{w}_r$ phụ thuộc vào đầu vào.

10.3 Năm lớp của ANFIS và ý nghĩa thật của từng lớp

ANFIS kinh điển được mô tả bằng năm lớp. Nhưng mô tả hình thức chỉ có giá trị nếu ta hiểu mỗi lớp đang làm gì về mặt toán học.

10.3.1 Lớp 1: làm mờ hóa

Mỗi nút tính độ thuộc của một đầu vào với một nhãn mờ. Nếu dùng Gaussian:

$$O_{1,i} = \mu_i(x) = \exp\left(-\frac{(x - c_i)^2}{2\sigma_i^2}\right).$$

Các tham số (c_i, σ_i) là tham số tiền đề. Về bản chất, lớp này quyết định cách ta phân vùng mềm miền đầu vào.

10.3.2 Lớp 2: kích hoạt luật

Mỗi nút tương ứng với một luật:

$$O_{2,r} = w_r = \prod_j \mu_{A_j^r}(x_j).$$

Đây là nơi các khái niệm mờ riêng lẻ được kết hợp thành một chế độ cục bộ có ý nghĩa tổng hợp.

10.3.3 Lớp 3: chuẩn hóa

$$O_{3,r} = \bar{w}_r = \frac{w_r}{\sum_s w_s}.$$

Lớp này biến độ kích hoạt thành các trọng số chuẩn hóa. Nhờ đó, đầu ra cuối mang cấu trúc của một tổ hợp mềm giữa các mô hình cục bộ.

10.3.4 Lớp 4: hệ quả

Mỗi nút tính

$$O_{4,r} = \bar{w}_r f_r = \bar{w}_r \left(p_{r0} + \sum_j p_{rj} x_j \right).$$

Đây là nơi tri thức mờ gặp mô hình tuyến tính cục bộ.

10.3.5 Lớp 5: tổng hợp

$$O_{5,1} = \sum_r O_{4,r}.$$

Lớp cuối cộng tất cả đóng góp cục bộ để tạo đầu ra toàn cục.

10.4 Điều gì làm ANFIS đẹp về mặt toán học

Có ba lý do khiến ANFIS đẹp một cách khác thường.

10.4.1 Lý do thứ nhất: nó là mô hình cục bộ hóa có cấu trúc

ANFIS không cố học một hàm phi tuyến toàn cục một cách trực tiếp. Nó nói: hãy chia không gian đầu vào thành các vùng mờ, rồi trong mỗi vùng học một mô hình tuyến tính cục

bộ. Điều này vừa linh hoạt hơn một mô hình tuyến tính toàn cục, vừa dễ kiểm soát hơn một mạng sâu hoàn toàn không cấu trúc.

10.4.2 Lý do thứ hai: một nửa bài toán có dạng tuyến tính

Khi các tham số tiền đề được cố định, mô hình trở thành tuyến tính theo các hệ số hệ quả. Đây là chìa khóa mở ra thuật toán học lai.

10.4.3 Lý do thứ ba: nó giữ được cầu nối diễn giải

Nếu các hàm thuộc và luật còn giữ nghĩa, ta có thể đọc ra các vùng hoạt động của mô hình. Đây là điều nhiều kiến trúc học sâu thuần túy không có.

10.5 Học lai của ANFIS: suy ra từ dạng tuyến tính của hệ quả

Giả sử dữ liệu huấn luyện là $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$. Với mỗi mẫu $\boldsymbol{x}_i$, nếu phần tiền đề được cố định, ta tính được các $\bar{w}_{ir}$. Khi đó

$$\hat{y}_i = \sum_{r=1}^{R} \bar{w}_{ir} \left(p_{r0} + \sum_{j=1}^{d} p_{rj} x_{ij} \right).$$

Gộp tất cả các hệ số $p_{r0}, p_{r1}, \ldots, p_{rd}$ thành một vectơ lớn $\boldsymbol{\beta}$, ta viết được

$$\hat{\boldsymbol{y}} = A\boldsymbol{\beta}.$$

Trong đó ma trận A được tạo bằng cách xếp các đặc trưng đã nhân với trọng số chuẩn hóa:

$$A_i = \left[\bar{w}_{i1}, \bar{w}_{i1}x_{i1}, \ldots, \bar{w}_{i1}x_{id}, \bar{w}_{i2}, \ldots, \bar{w}_{iR}x_{id}\right].$$

Khi đó bài toán học hệ quả là

$$\min_{\boldsymbol{\beta}} \|A\boldsymbol{\beta} - \boldsymbol{y}\|_2^2.$$

Đây là một bài toán bình phương tối thiểu thuần túy.

10.5.1 Ý nghĩa của điều này

ANFIS không cần dùng gradient cho mọi thứ. Nó chỉ cần gradient cho phần tiền đề. Phần hệ quả có thể được cập nhật bằng một lời giải đóng hoặc gần đóng của bình phương tối thiểu.

Điều này làm cho ANFIS khác nhiều so với các lớp tùy biến fuzzy viết trong các framework hiện đại nhưng huấn luyện hoàn toàn bằng lan truyền ngược.

10.6 Ma trận hóa của phần hệ quả và mối liên hệ với hồi quy tuyến tính

Để thấy ANFIS không thần bí, ta hãy so sánh nó với hồi quy tuyến tính. Trong hồi quy tuyến tính, mỗi mẫu $\boldsymbol{x}_i$ sinh ra một hàng của ma trận thiết kế:

$$[1,\ x_{i1}, \dots, x_{id}].$$

Trong ANFIS, mỗi mẫu sinh ra một hàng lớn hơn nhiều:

$$[\bar{w}_{i1},\ \bar{w}_{i1}x_{i1}, \dots, \bar{w}_{i1}x_{id},\ \bar{w}_{i2}, \dots].$$

Về bản chất, ANFIS đang thực hiện một dạng mở rộng đặc trưng phụ thuộc dữ liệu, trong đó các trọng số mờ đóng vai trò "bật tắt mềm" những đặc trưng cục bộ. Đây là một cách hiểu cực kỳ hữu ích, vì nó đặt ANFIS vào cùng dòng tư duy với biểu diễn đặc trưng và mô hình cục bộ trong học máy.

10.7 Đạo hàm của phần tiền đề và động học học

Nếu dùng Gaussian

$$\mu(x; c, \sigma) = \exp\left(-\frac{(x-c)^2}{2\sigma^2}\right),$$

ta có

$$\frac{\partial \mu}{\partial c} = \mu(x; c, \sigma)\frac{x-c}{\sigma^2}, \qquad \frac{\partial \mu}{\partial \sigma} = \mu(x; c, \sigma)\frac{(x-c)^2}{\sigma^3}.$$

Hai công thức này cho thấy ba điều.

1. Nếu μ quá nhỏ, gradient cũng rất nhỏ.
2. Nếu σ quá lớn, hàm thuộc gần phẳng và gradient không còn định vị tốt.
3. Nếu σ quá nhỏ, hệ quá nhọn, dễ sinh vùng chết hoặc bất ổn.

Đây là lý do tại sao việc khởi tạo và điều chuẩn các hàm thuộc không thể xem là chi tiết phụ.

10.8 Khởi tạo các hàm thuộc: vì sao phân cụm thường được dùng

Nếu khởi tạo tâm và độ rộng hoàn toàn ngẫu nhiên, ANFIS có thể bắt đầu từ một phân hoạch rất nghèo nàn của không gian đầu vào. Một số luật gần như không bao giờ kích hoạt; một số khác chồng lấp quá mạnh; tổng độ kích hoạt ở nhiều vùng rất nhỏ. Điều này gây hại cả cho tối ưu lẫn cho diễn giải.

Phân cụm, đặc biệt là K-Means một chiều theo từng đặc trưng, được dùng vì nó cho ta các tâm ban đầu gắn với mật độ dữ liệu. Tuy nhiên, có hai điều phải nói thật:

1. phân cụm không tạo ra "nghĩa ngôn ngữ" một cách tự động; nó chỉ tạo ra cấu trúc dữ liệu ban đầu;
2. nếu các tâm được phép học tự do sau đó, nghĩa LOW/HIGH vẫn có thể bị đảo.

10.9 Bùng nổ số luật: bài toán trung tâm của ANFIS

Nếu có d đặc trưng và mỗi đặc trưng có m hàm thuộc, số luật kiểu lưới là

$$R = m^d.$$

Với $d = 10$ và $m = 3$, ta có

$$R = 59049.$$

Không chỉ số luật tăng nhanh, số tham số hệ quả cũng tăng theo $R(d + 1)$. Đây là lý do ANFIS nguyên thủy khó mở rộng lên không gian đặc trưng lớn.

10.9.1 Các chiến lược giảm

Những chiến lược phổ biến gồm:

1. giảm số đặc trưng vào khối mờ;
2. chia thành nhiều ANFIS nhỏ theo nhóm đặc trưng;
3. sinh luật bằng phân cụm thay vì lưới toàn phần;
4. dùng hệ mờ phân cấp;
5. dùng điều chuẩn thưa để triệt bớt luật.

Đây là lý do phương án *feature-group ANFIS* trong script có một ý tưởng đáng ghi nhận: nó đối mặt trực diện với bùng nổ số luật.

10.10 ANFIS nhiều đầu ra: tiện nhưng làm diễn giải khó hơn

Trong ANFIS gốc, ta thường mô tả một đầu ra duy nhất. Trong thực hành, đặc biệt với thư viện học sâu, người ta có thể cho mỗi luật sinh ra một vectơ đầu ra:

$$f_r(\boldsymbol{x}) \in \mathbb{R}^k.$$

Điều này tiện cho dự báo nhiều biến đồng thời. Nhưng nó cũng làm phần hệ quả khó đọc hơn, vì mỗi luật không còn cho một phương trình cục bộ duy nhất, mà cho một bó phương trình cục bộ. Nếu không thiết kế công cụ trích xuất cẩn thận, khả năng diễn giải suy giảm rất nhanh.

10.11 ANFIS và tính diễn giải: cần những điều kiện nào

Ta có thể gọi ANFIS là dễ diễn giải chỉ khi ít nhất các điều kiện sau được giữ:

1. nhãn của các hàm thuộc còn khớp với vị trí thật của các hàm thuộc;
2. số luật không vượt quá ngưỡng mà con người có thể đọc;
3. phần hệ quả được trích xuất và mô tả đầy đủ;
4. đầu ra cuối của mô hình vẫn chủ yếu đi qua cơ chế luật đó.

Nếu một mô hình chỉ giữ được điều kiện thứ nhất và thứ hai, nhưng hệ quả không được đọc ra, hoặc đầu ra cuối bị một khối khác pha trộn mạnh, thì việc gọi cả mô hình là "giải thích được" là một cường điệu.

10.12 Tính nhận dạng, hoán vị luật và sự mong manh của giải thích

Một khó khăn bản chất của ANFIS là nhiều cấu hình tham số khác nhau có thể mô tả hàm gần giống nhau. Ta có thể hoán vị thứ tự các luật mà hàm đầu ra không đổi. Ta cũng có thể đổi chỗ các hàm thuộc kèm theo hoán vị hệ quả. Do đó, ngay cả khi ANFIS học tốt về dự báo, tham số học được không phải lúc nào cũng có một cách đọc duy nhất.

Điều này có hai hệ quả sư phạm quan trọng:

1. không nên thần thoại hóa việc "đọc tham số" như thể đó là sự thật duy nhất về mô hình;

2. muốn diễn giải đáng tin, ta phải thêm các ràng buộc nhận dạng, ví dụ giữ thứ tự tâm, giữ cấu trúc phân hoạch, hoặc áp đặt các điều kiện trơn phù hợp.

10.13 ANFIS trong framework học sâu hiện đại: lợi và hại

Khi triển khai ANFIS trong TensorFlow hay PyTorch, ta thường viết một lớp tùy biến, cho phép ghép ANFIS với CNN, LSTM, hay *transformer*. Điều này mở ra rất nhiều khả năng kiến trúc, nhưng đồng thời tạo ra ba rủi ro.

10.13.1 Rủi ro thứ nhất: mất học lai

Vì tiện lập trình, người ta dễ huấn luyện toàn bộ bằng lan truyền ngược. Khi đó ANFIS không còn tận dụng lợi thế cấu trúc của bình phương tối thiểu ở phần hệ quả.

10.13.2 Rủi ro thứ hai: giữ bề ngoài mà mất bản chất

Một khối fuzzy nhỏ được gắn vào đâu đó trong mạng có thể làm mô hình trông "có mờ". Nhưng nếu đầu ra cuối không còn đi qua phép tổng hợp Sugeno một cách chi phối, thì mô hình đã rời xa bản chất ANFIS.

10.13.3 Rủi ro thứ ba: nhầm diễn giải cục bộ với diễn giải toàn cục

Ta có thể trích xuất luật của một nhánh, nhưng đầu ra cuối lại do cả một kiến trúc sâu pha trộn. Khi đó, luật mờ chỉ giải thích được một phần biểu diễn trung gian, không giải thích được toàn bộ hệ thống.

10.14 Khi nào không nên dùng ANFIS

ANFIS không phải công cụ cho mọi tình huống. Có ít nhất bốn bối cảnh mà ta nên thận trọng:

1. số đặc trưng quá lớn và không thể giảm hợp lý;
2. dữ liệu quá ít, không đủ để ước lượng nhiều mô hình cục bộ;
3. miền bài toán không có ý nghĩa ngôn ngữ rõ ràng cho các đặc trưng;
4. đầu ra cuối cùng đòi hỏi cấu trúc rất phức tạp mà vài chục hay vài trăm luật khó bao quát.

Trong các bối cảnh này, hoặc ta phải đơn giản hóa đầu vào, hoặc ta nên chọn một kiến trúc khác phù hợp hơn.

10.15 Những lỗi lập trình ANFIS hay gặp và vì sao chúng không nhỏ

Ta có thể liệt kê một số lỗi rất hay gặp:

1. không ép độ rộng dương;
2. không bảo vệ phép chia khi tổng độ kích hoạt quá nhỏ;
3. gán nhãn LOW/HIGH theo chỉ số cố định nhưng không giữ thứ tự tâm;
4. chỉ trích xuất tiền đề của luật mà bỏ qua hệ quả;
5. lưu mô hình nhưng không chuẩn bị cho việc tải lại lớp tùy biến;
6. đánh đồng việc có hàm thuộc với việc mô hình đầu ra giải thích được.

Những lỗi này không phải chuyện trình bày. Chúng chạm vào khả năng học, tính ổn định, khả năng lặp lại, và giá trị diễn giải của cả hệ thống.

10.16 ANFIS trong bài báo và ANFIS trong ứng dụng thực tế

Trong bài báo gốc, ANFIS thường được xét như một hệ mờ Sugeno khá thuần. Trong thực tế, nhất là trong mã nguồn ứng dụng, ANFIS dễ bị nhúng vào những kiến trúc lớn hơn. Điều này không sai. Vấn đề là ta phải trung thực trong cách gọi tên.

Nếu một mô hình lấy đầu ra của ANFIS rồi tiếp tục pha trộn với LSTM và nhiều lớp *Dense*, thì mô hình đó tốt nhất nên được gọi là *hybrid model with ANFIS components* (mô hình lai có thành phần ANFIS), thay vì được mô tả như thể toàn bộ đầu ra cuối vẫn là kết quả trực tiếp của hệ ANFIS nguyên nghĩa.

10.17 Kết luận chương

ANFIS đẹp vì nó nằm giữa ba thế giới: logic mờ, mô hình cục bộ tuyến tính, và tối ưu học từ dữ liệu. Nhưng chính vì đứng giữa ba thế giới ấy, nó cũng dễ bị hiểu sai ở cả ba phía. Nếu chỉ nhìn nó như một mạng, ta bỏ mất cấu trúc mờ. Nếu chỉ nhìn nó như một hệ mờ, ta bỏ

mất hình học của bài toán học. Nếu chỉ nhìn nó như công cụ diễn giải, ta dễ quên các điều kiện cần để diễn giải còn có nghĩa. Đây là lý do chương tiếp theo phải trở về một mô hình cụ thể trong code để xem những điều kiện ấy còn giữ được đến đâu.

Chuong 11

Phân tích lý thuyết và kiểm toán kỹ thuật file `run_feature_group_anfis.py`

11.1 Phải đọc chương này như thế nào để không rơi vào suy diễn cảm tính

Khi phân tích một script học máy có tên gọi hấp dẫn như "Feature-Group ANFIS + BiLSTM", người đọc rất dễ trượt vào một trong hai cực đoan. Cực đoan thứ nhất là tin ngay vào tên gọi và các chỉ số được in ra, từ đó xem mô hình như một thành công đã hoàn tất. Cực đoan thứ hai là thấy vài chỗ lạ rồi kết luận toàn bộ mô hình vô nghĩa. Cả hai đều là cách đọc hời hợt. Mục tiêu của chương này là dựng lại một kỷ luật đọc nghiêm túc hơn.

Kỷ luật đó có ba tầng. Tầng thứ nhất là *tầng xác nhận trực tiếp*: điều gì ta đọc được thẳng từ mã nguồn, không cần chạy huấn luyện, không cần suy đoán thêm. Tầng thứ hai là *tầng hệ quả logic*: từ những gì code làm, điều gì kéo theo một cách tất yếu về mặt toán học hoặc phương pháp luận. Tầng thứ ba là *tầng rủi ro hợp lý*: những điểm chưa thể kết luận cứng nếu không có thực nghiệm, nhưng có lý do vững để cảnh báo. Nếu không phân biệt ba tầng này, việc "phân tích" sẽ nhanh chóng trở thành một bài văn kết tội hoặc ca ngợi theo cảm xúc.

Từ việc đọc trực tiếp file `run_feature_group_anfis.py`, ta có thể xác nhận ít nhất các sự kiện sau đây.

1. Script tạo 6 đặc trưng đầu vào: `Close_ret`, `Open_ret`, `High_ret`, `Low_ret`, `range_pct`, `gap`.

2. Script tạo 4 biến mục tiêu: `target_Close_ret`, `target_Open_ret`, `target_High_ret`, `target_Low_ret`.

3. Hai nhánh ANFIS chỉ nhìn *last timestep* của cửa sổ, còn nhánh BiLSTM nhìn toàn bộ chuỗi con dài 60 bước.

4. `StandardScaler` được `fit_transform` trên toàn bộ `features` và `targets` trước khi tách train/test.
5. Trong lúc huấn luyện, `validation_data=(d['X_test'], d['y_test'])` được dùng trực tiếp trong `model.fit`.
6. Nếu có nhiều lần chạy, script chọn mô hình tốt nhất bằng chỉ số trên chính tập test.
7. Hàm `extract_rules` chỉ in phần tiền đề của luật, không in hệ số hệ quả.
8. Script mã hóa cứng `base_dir = '/Users/bao/Documents/tsa_paper_1'`, khác với workspace hiện tại.

Những mệnh đề trên không còn là chuyện quan điểm. Chúng là dữ kiện. Toàn bộ phần còn lại của chương này sẽ dựa trên các dữ kiện đó, chứ không xây trên tưởng tượng.

11.2 Bài toán toán học mà script thực sự đang giải là gì

Muốn đánh giá một mô hình, trước hết ta phải dựng lại chính xác bài toán mà nó đang học. Chỉ nhìn tên biến là chưa đủ, vì tên biến có thể gợi ý một ý nghĩa khác với công thức thật. Trong `prepare_data`, script tạo các đặc trưng đầu vào

$$\text{Close_ret}_t = \frac{C_t}{C_{t-1}} - 1, \qquad \text{Open_ret}_t = \frac{O_t}{O_{t-1}} - 1,$$

$$\text{High_ret}_t = \frac{H_t}{H_{t-1}} - 1, \qquad \text{Low_ret}_t = \frac{L_t}{L_{t-1}} - 1,$$

$$\text{range_pct}_t = \frac{H_t - L_t}{C_t}, \qquad \text{gap}_t = \frac{O_t - C_{t-1}}{C_{t-1}}.$$

Các đặc trưng này, xét riêng từng công thức, đều là đại lượng có thể biết được tại thời điểm cuối ngày t. Vì vậy, nếu chỉ xét phần tạo đặc trưng, ta chưa có rò rỉ trực tiếp từ tương lai.

Điểm tinh tế hơn nằm ở phần mục tiêu. Script định nghĩa

$$\text{target_Close_ret}_t = \frac{C_{t+1}}{C_t} - 1,$$

$$\text{target_Open_ret}_t = \frac{O_{t+1}}{C_t} - 1, \qquad \text{target_High_ret}_t = \frac{H_{t+1}}{C_t} - 1, \qquad \text{target_Low_ret}_t = \frac{L_{t+1}}{C_t} - 1.$$

Nói bằng lời, mô hình không học "return của Open ngày mai so với Open hôm nay" và cũng không học "return của High ngày mai so với High hôm nay". Nó học các giá trị OHLC của ngày mai nhưng tất cả đều được chuẩn hóa tương đối theo C_t, tức giá đóng cửa hôm nay.

Chi tiết này dẫn tới một kiểm tra rất nhỏ nhưng cực kỳ quan trọng. Nếu $\widehat{r}_t^{(O)}$ là dự báo

cho target_Open_ret$_t$, thì

$$\widehat{O}_{t+1} = C_t(1 + \widehat{r}_t^{(O)}).$$

Hoàn toàn tương tự,

$$\widehat{H}_{t+1} = C_t(1 + \widehat{r}_t^{(H)}), \qquad \widehat{L}_{t+1} = C_t(1 + \widehat{r}_t^{(L)}).$$

Vì vậy, việc script dùng `current_close` để biến toàn bộ bốn đầu ra từ return về price là *nhất quán với chính định nghĩa mục tiêu mà script đã chọn*. Đây là chỗ rất dễ bị cáo buộc oan. Nếu không đọc kỹ công thức, người ta sẽ thấy dòng

```
pred_px = current_close * (1 + pred_ret)
```

rồi kết luận ngay "sai vì Open/High/Low phải dùng giá riêng của chúng". Kết luận ấy nghe hợp trực giác, nhưng lại không đúng với bài toán mà code thực sự đang tối ưu.

Điều cần nói công bằng hơn là: định nghĩa mục tiêu này hơi lạ và làm ngữ nghĩa đầu vào–đầu ra kém trong suốt hơn, nhưng bản thân phép quy đổi về price không phải là bug tính toán. Sự phân biệt giữa "ngữ nghĩa hơi lạ" và "tính toán sai" không phải chuyện chữ nghĩa; nó là tiêu chuẩn trung thực khi đọc code.

Một điểm kỹ thuật khác cũng nên được dựng lại cẩn thận là cấu trúc cửa sổ thời gian. Script dùng cửa sổ dài $L = 60$ và, với mỗi chỉ số i, lấy đoạn $[i - L + 1, \ldots, i]$ để dự báo mục tiêu tại chính chỉ số i, tức dự báo từ thời điểm $t = i$ sang $t + 1$. Đây là một lựa chọn hợp lý: mọi đặc trưng trong cửa sổ đều đã biết tại lúc ta muốn dự báo ngày tiếp theo. Tuy nhiên, vòng lặp lại bắt đầu từ `i = look_back` thay vì chỉ số đầu tiên hợp lệ `look_back` $- 1$. Hệ quả là mô hình bỏ phí đúng một mẫu hợp lệ đầu tiên. Đây chỉ là lỗi nhỏ về chỉ số, không làm hỏng thí nghiệm, nhưng nó là ví dụ tốt cho thấy chuỗi thời gian đòi hỏi sự cẩn thận đến từng chỉ số.

11.3 Kiến trúc thực sự của mô hình và những ý tưởng đáng ghi nhận

Sau khi dựng lại bài toán, ta chuyển sang kiến trúc. Gọi

$$X_t \in \mathbb{R}^{60\times 6}$$

là cửa sổ 60 bước của 6 đặc trưng. Mô hình tách thành ba nhánh. Nhánh thứ nhất lấy bước cuối $x_t \in \mathbb{R}^6$, cắt 4 đặc trưng đầu và đưa vào `FeatureGroupANFIS` cho nhóm return. Nhánh thứ hai cũng lấy x_t, cắt 2 đặc trưng cuối và đưa vào một `FeatureGroupANFIS` khác cho nhóm chỉ báo. Nhánh thứ ba đưa toàn bộ X_t qua hai lớp BiLSTM nối tiếp. Cuối cùng, các biểu diễn này được ghép lại, đi qua một lớp `Dense(32, relu)` rồi sinh 4 đầu

ra tuyến tính.

Viết trừu tượng, mô hình có dạng

$$\widehat{\mathbf{y}}_{t+1} = g_\psi\big(h_{\text{ret}}(x_t), h_{\text{ind}}(x_t), h_{\text{seq}}(X_t)\big),$$

trong đó h_{ret} và h_{ind} là hai khối ANFIS, còn h_{seq} là khối BiLSTM. Biểu thức này cho ta thấy ngay một sự thật nền tảng: đầu ra cuối *không phải* đầu ra trực tiếp của một hệ Sugeno duy nhất. Nó là đầu ra của một mô hình lai, nơi các thành phần mờ chỉ là các bộ biến đổi trung gian ở đầu vào.

Tuy vậy, sẽ không công bằng nếu chỉ nhìn mô hình dưới lăng kính phê phán. Script này có ít nhất ba ý tưởng kiến trúc đáng ghi nhận.

Ý tưởng thứ nhất là *chia nhóm đặc trưng để giảm bùng nổ số luật*. Nếu dùng một ANFIS duy nhất cho cả 6 đặc trưng với 2 hàm thuộc mỗi đặc trưng, số luật của lưới đầy đủ là

$$2^6 = 64.$$

Script chia thành một ANFIS 4 đặc trưng và một ANFIS 2 đặc trưng, nên tổng số luật là

$$2^4 + 2^2 = 16 + 4 = 20.$$

Đây là một quyết định có cơ sở. Nó nhắm đúng một điểm yếu cấu trúc của ANFIS: số luật tăng theo cấp số mũ theo số đầu vào.

Ý tưởng thứ hai là *khởi tạo tâm của hàm thuộc bằng K-Means* thay vì thuần ngẫu nhiên. K-Means một chiều trên từng đặc trưng không tạo "nghĩa ngôn ngữ" một cách thần kỳ, nhưng nó thường cho một vị trí khởi tạo có trật tự hơn so với lấy ngẫu nhiên hoàn toàn. Nếu dữ liệu được tiền xử lý đúng, đây là một ý tưởng hợp lý và thậm chí khá thực dụng.

Ý tưởng thứ ba là *tách vai trò của nhánh mờ và nhánh thời gian*. Hai nhánh ANFIS nhìn vào bước cuối để mã hóa trạng thái cục bộ gần nhất; BiLSTM nhìn cả cửa sổ để mã hóa động học thời gian. Đây là một trực giác có thể bảo vệ được. Nó không đảm bảo mô hình sẽ tốt, nhưng ít nhất nó cho thấy tác giả có ý thức rằng "khái niệm ngôn ngữ cục bộ" và "mẫu động học trên chuỗi" là hai loại tín hiệu khác nhau.

Chính vì có những ý tưởng tốt này mà việc kiểm toán phần còn lại càng đáng làm. Khi một mô hình có vài trực giác hay, người ta càng dễ bỏ qua lỗi giao thức và càng dễ tin các tuyên bố diễn giải mạnh hơn mức mà code bảo đảm.

11.4 Ba sai phạm phương pháp luận làm hỏng giá trị của chỉ số đánh giá

Phần nghiêm trọng nhất của script không nằm ở chỗ nó dùng ANFIS, BiLSTM hay K-Means, mà nằm ở chỗ nó đánh giá mô hình như thế nào. Trong khoa học dữ liệu, một kiến trúc tầm thường nhưng giao thức sạch vẫn có thể cho kết luận hữu ích. Ngược lại, một kiến trúc thông minh nhưng giao thức sai có thể cho ra bảng chỉ số rất đẹp mà giá trị khoa học rất thấp. Script này rơi vào trường hợp thứ hai.

Sai phạm thứ nhất là chuẩn hóa trên toàn bộ dữ liệu trước khi tách train/test. Trong `prepare_data`, script thực hiện

```
scaled_feat = feat_scaler.fit_transform(features),    scaled_tgt = tgt_sc
```

rồi mới chia tập theo thời gian. Để thấy đây là rò rỉ thông tin theo nghĩa chặt, gọi μ_{all} và σ_{all} là trung bình và độ lệch chuẩn của toàn bộ đặc trưng. Khi đó một mẫu train tại thời điểm t sau chuẩn hóa có dạng

$$\widetilde{x}_t = \frac{x_t - \mu_{\text{all}}}{\sigma_{\text{all}}}.$$

Nhưng μ_{all} và σ_{all} phụ thuộc vào cả phần cuối chuỗi, bao gồm cả vùng về sau thuộc tập test. Như vậy $\widetilde{x}_t$ không chỉ là hàm của thông tin khả dụng tại thời điểm t cộng với thống kê huấn luyện; nó đã bị đặt trong một hệ quy chiếu có hấp thụ thông tin tương lai. Đó chính là rò rỉ thông tin.

Lỗi này còn nặng hơn vẻ bề ngoài của nó. Không chỉ phần huấn luyện mạng bị nhiễm thông tin tương lai; cả bước K-Means khởi tạo tâm hàm thuộc cũng bị nhiễm, vì K-Means được áp trên `d['X_train']` đã được chuẩn hóa bằng thống kê toàn chuỗi. Như vậy, ngay từ tầng tiền đề của các luật mờ, mô hình đã không còn được xây trên một điều kiện thông tin sạch.

Ví dụ 11.1. Xét một đặc trưng một chiều với bốn quan sát theo thời gian:

$$1, \quad 2, \quad 3, \quad 100.$$

Giả sử ba điểm đầu là train, điểm cuối là test.

Nếu chuẩn hóa đúng trên train, trung bình train là 2, nên ba điểm train được đặt quanh 0 một cách tự nhiên. Nếu chuẩn hóa trên toàn bộ chuỗi, trung bình toàn bộ là 26.5, khi đó cả ba điểm train bị đẩy mạnh sang phía âm chỉ vì sự hiện diện của giá trị 100 ở tương lai. Không có nhãn tương lai nào bị chép vào đầu vào, nhưng hình học của dữ liệu train đã bị uốn bởi test. Đó chính là leakage.

Sai phạm thứ hai là dùng tập test làm tập validation trong quá trình huấn luyện.

Câu lệnh

```
validation_data=(d['X_test'], d['y_test'])
```

được đưa trực tiếp vào `model.fit`. Khi đó, *EarlyStopping* theo dõi tập test, *ReduceLROnPlateau* theo dõi tập test, và trọng số tốt nhất được khôi phục cũng dựa trên tập test. Nói cách khác, tập test đã tham gia vào quá trình ra quyết định huấn luyện. Một tập như vậy không còn là test theo đúng nghĩa khoa học.

Lập luận này có thể viết rất gọn nhưng rất chặt. Giả sử một lần huấn luyện sinh ra dãy mô hình

$$f_{\theta^{(1)}}, f_{\theta^{(2)}}, \ldots, f_{\theta^{(E)}}.$$

Nếu ta dùng một tập dữ liệu để chọn epoch nào là tốt nhất, tập dữ liệu đó đã tham gia vào thủ tục chọn mô hình. Một tập vừa tham gia chọn mô hình vừa được dùng để báo cáo kết quả cuối không còn là đánh giá ngoài mẫu độc lập nữa. Nó là một phần của tối ưu.

Sai phạm thứ ba là chọn lần chạy tốt nhất theo chỉ số trên tập test. Khi `n_runs>1`, script lặp nhiều seed, tính Avg R^2 trên tập test cho từng lần chạy, rồi giữ lần tốt nhất. Đây là một tầng tối ưu hóa thứ hai trên test. Nó tương đương với việc dò nhiều kết quả ngẫu nhiên rồi chỉ công bố kết quả đẹp nhất trên cùng một bộ kiểm tra. Về bản chất, đây là một dạng *multiple testing* không hiệu chỉnh.

Ba sai phạm này không đứng riêng lẻ. Chúng chồng lên nhau theo một cách đặc biệt nguy hiểm. Trước hết, dữ liệu train đã được chuẩn hóa bằng thông tin của toàn bộ chuỗi. Sau đó, quá trình huấn luyện và dừng sớm nhìn trực tiếp vào test. Cuối cùng, nếu có nhiều lần chạy, test lại tiếp tục bị dùng để chọn seed đẹp nhất. Khi một chỉ số cuối cùng được in ra sau ba lớp can thiệp như vậy, giá trị của nó như một ước lượng ngoài mẫu độc lập gần như không còn.

Đây là lý do chương này phải nói rất thẳng: những lỗi nghiêm trọng nhất của script không phải là lỗi biểu thức hay lỗi API. Chúng là lỗi phương pháp luận. Và trong nghiên cứu, lỗi phương pháp luận mới là loại lỗi phá giá trị của toàn bộ kết quả.

11.5 Tại sao tuyên bố "giải thích được" của mô hình hiện tại bị thổi phồng

Phần thứ hai cần kiểm toán là câu chuyện diễn giải. Script được đặt tên và mô tả theo hướng nhấn mạnh "fuzzy rules", "semantic meaning" và "explainable predictions". Những cụm này không sai hoàn toàn, nhưng ở trạng thái hiện tại chúng mạnh hơn mức mà code thực sự bảo đảm.

Vấn đề đầu tiên nằm ở việc `get_rule_descriptions` chỉ sinh ra các chuỗi dạng

```
Rule 1: IF Close_ret is LOW AND Open_ret is HIGH ...
```

tức là chỉ có *vế tiền đề*. Trong ANFIS Sugeno bậc một, một luật đầy đủ phải có dạng

$$\text{IF antecedent THEN } y = a_0 + a_1 x_1 + \cdots + a_d x_d.$$

Chính phần hệ quả mới cho biết khi luật được kích hoạt thì đầu ra bị kéo về đâu. Nếu chỉ in tiền đề mà không in các hệ số hệ quả, ta chưa hề có "luật đầy đủ" theo nghĩa giải thích dự báo. Ta chỉ mới biết vùng nào của không gian đầu vào đang sáng lên.

Vấn đề thứ hai là nhãn LOW/HIGH không được bảo toàn về ngữ nghĩa sau huấn luyện. Script khởi tạo tâm các hàm thuộc bằng K-Means và sắp tăng dần, nên tại thời điểm khởi tạo ta có thể nói tạm rằng chỉ số 0 gần với "thấp" hơn chỉ số 1. Nhưng sau đó các tâm `mf_centers` là tham số được học tự do, không có ràng buộc thứ tự hay khoảng cách. Không có gì ngăn

$$c_{\text{LOW}} > c_{\text{HIGH}}$$

sau một giai đoạn tối ưu. Nếu điều đó xảy ra mà ta vẫn giữ nguyên nhãn LOW/HIGH trong báo cáo, thì chuỗi chữ "feature is LOW" chỉ còn là một nhãn chỉ số, không còn là một mệnh đề ngôn ngữ trung thực.

Vấn đề thứ ba nằm ở chính kiến trúc cuối của mô hình. Như đã thấy,

$$\widehat{\mathbf{y}}_{t+1} = g_\psi\big(h_{\text{ret}}(x_t), h_{\text{ind}}(x_t), h_{\text{seq}}(X_t)\big).$$

Đầu ra cuối không phải là đầu ra trực tiếp của các luật mờ. Nó là đầu ra của một khối hợp nhất biểu diễn ANFIS với biểu diễn BiLSTM rồi đi qua thêm các lớp `Dense`. Điều này không có nghĩa nhánh mờ vô giá trị. Nó chỉ có nghĩa rằng câu "mô hình dự báo cuối là giải thích được bằng luật mờ" là quá mạnh. Mức mô tả trung thực hơn phải là: *mô hình có các thành phần mờ, và ta có thể phân tích một phần cấu trúc quy tắc cũng như mức kích hoạt của chúng*.

Vấn đề thứ tư xuất hiện trong `analyze_prediction`. Hàm này trả ra các luật có *firing strength* lớn nhất, tức các luật đang kích hoạt mạnh. Đây là một thông tin hữu ích, nhưng nó chưa phải là giải thích đóng góp cuối cùng. Một luật có thể kích hoạt mạnh nhưng hệ quả của nó trung tính; một luật khác có thể kích hoạt vừa phải nhưng hệ quả kéo đầu ra rất mạnh theo một hướng nào đó. Không có phần hệ quả, và không có phân rã đóng góp sau khối trộn cuối, ta không thể nói rằng "top active rules" đã giải thích đầy đủ dự báo.

Ví dụ 11.2. Giả sử có hai luật với độ kích hoạt chuẩn hóa lần lượt là

$$w_1 = 0.9, \qquad w_2 = 0.1.$$

Nếu hệ quả của luật thứ nhất là $f_1(x) = 0$ còn hệ quả của luật thứ hai là $f_2(x) = 100$, thì đầu ra Sugeno là

$$y = w_1 f_1(x) + w_2 f_2(x) = 0.9 \cdot 0 + 0.1 \cdot 100 = 10.$$

Lúc này, luật 1 là luật kích hoạt mạnh nhất, nhưng chính luật 2 mới tạo phần lớn độ lệch khỏi 0. Vì vậy, chỉ báo "top active rules" không đủ để thay cho phân tích đóng góp thật của luật.

Tóm lại, vấn đề diễn giải của script không nằm ở chỗ "không có chút nào để đọc". Vẫn có thứ để đọc: số luật, tâm và độ rộng của hàm thuộc, mức kích hoạt tương đối giữa các luật. Nhưng phần có thể đọc ấy chỉ là một *lớp cục bộ* của mô hình, không phải lời giải thích trọn vẹn cho đầu ra cuối. Nếu không giữ đúng phạm vi phát biểu, ta sẽ biến một ưu điểm có thật nhưng hạn chế thành một tuyên bố quá mức.

11.6 Những bất nhất mô hình hóa khác: không phải lỗi nào cũng cùng mức độ, nhưng đều đáng nói

Ngoài các sai phạm giao thức và câu chuyện diễn giải, script còn chứa một cụm vấn đề khác thuộc tầng mô hình hóa và triển khai. Chúng không phải tất cả đều nghiêm trọng ngang nhau, nhưng trong một giáo trình nghiêm túc ta cần phân loại đúng mức thay vì bỏ qua hoặc phóng đại.

Điểm thứ nhất là *đầu ra OHLC không bị ép tuân thủ hình học của nến giá*. Mô hình sinh bốn số liên tục độc lập rồi tính RMSE, MAE, MAPE, R^2 cho từng thành phần. Nhưng dữ liệu OHLC không phải bốn số hoàn toàn độc lập. Một bộ dự báo hợp lệ cần thỏa

$$\widehat{H}_{t+1} \geq \max\{\widehat{O}_{t+1}, \widehat{C}_{t+1}\}, \qquad \widehat{L}_{t+1} \leq \min\{\widehat{O}_{t+1}, \widehat{C}_{t+1}\}.$$

Kiến trúc hiện tại không có ràng buộc nào trong hàm mất mát hoặc trong tham số hóa đầu ra để ép các bất đẳng thức đó. Vì vậy, ta có thể kết luận chắc chắn rằng mô hình *cho phép* sinh ra các bộ OHLC không hợp lệ. Điều ta chưa thể kết luận nếu không chạy thực nghiệm là tần suất vi phạm thực tế là bao nhiêu.

Điểm thứ hai là *sự không nhất quán ngữ nghĩa giữa đầu vào và đầu ra*. Đầu vào `Open_ret_t` được hiểu là thay đổi của O_t so với O_{t-1}, còn đầu ra `target_Open_ret_t` lại là thay đổi của O_{t+1} so với C_t. Cùng một hậu tố `_ret` nhưng hai biến mang hai ý nghĩa chuẩn hóa khác nhau. Điều này không làm phép tính sai, nhưng nó làm cho câu chuyện khái niệm trở nên mờ đục hơn. Với một mô hình muốn nhấn mạnh tính diễn giải, sự mờ đục ngữ nghĩa như vậy không nên bị xem nhẹ.

Điểm thứ ba liên quan đến *Directional Accuracy*. Hàm `calc_directional_accuracy` so sánh hướng của dự báo giá với giá hiện tại của từng thành phần O_t, H_t, L_t, C_t. Với *Close*, cách đo này khá tự nhiên vì mục tiêu cũng neo trực tiếp vào C_t. Nhưng với Open, High và Low, mô hình đã được huấn luyện trên các mục tiêu chuẩn hóa theo C_t, trong khi chỉ số hướng lại được tính so với O_t, H_t, L_t. Nói "Directional Accuracy hoàn toàn sai" có thể là quá mạnh; nhưng nói rằng nó không hoàn toàn đồng bộ với mục tiêu tối ưu trong huấn luyện là một kết luận cẩn trọng và đúng hơn nhiều.

Điểm thứ tư là *khả năng tái lập của script trong workspace hiện tại*. Hàm `main` mã hóa cứng đường dẫn

`/Users/bao/Documents/tsa_paper_1`,

trong khi workspace hiện tại là một cây thư mục khác. Điều này làm script chính không chạy được ngay trong ngữ cảnh repo đang mở. Đây không phải lỗi toán học của mô hình, nhưng lại là lỗi học thuật theo nghĩa thực dụng: một hiện vật nghiên cứu không thể gọi là tái lập tốt nếu chính đường dẫn dữ liệu của nó bị buộc chặt vào một máy cụ thể.

Điểm thứ năm là *việc tuần tự hóa và tải lại mô hình chưa được chuẩn bị đủ chắc*. Lớp `FeatureGroupANFIS` có `get_config`, cho thấy ý định hỗ trợ lưu mô hình là có. Nhưng không có đăng ký kiểu kiểu `@register_keras_serializable`, nên việc tải lại trong môi trường khác sẽ phụ thuộc vào việc người dùng có truyền `custom_objects` đúng hay không. Đây là một điểm yếu về đóng gói và tính lặp lại, không phải lỗi phá huấn luyện ngay lập tức.

Điều quan trọng ở đây là phân biệt mức độ nghiêm trọng. Các lỗi giao thức ở mục trước phá hỏng trực tiếp giá trị của đánh giá. Các điểm ở mục này chủ yếu làm suy giảm tính hợp lệ cấu trúc, tính trong suốt hoặc tính tái lập. Chúng vẫn đáng sửa, nhưng không nên trộn lẫn mọi thứ thành một mức "sai như nhau".

Ví dụ 11.3. Nếu mô hình dự báo

$$\widehat{O} = 100, \qquad \widehat{C} = 102, \qquad \widehat{H} = 101, \qquad \widehat{L} = 99,$$

thì

$$\widehat{H} = 101 < \max\{\widehat{O}, \widehat{C}\} = 102.$$

Mọi chỉ số RMSE trên từng tọa độ có thể vẫn thấp, nhưng bộ OHLC này không còn là một cây nến hợp lệ. Ví dụ toy này cho thấy vì sao lỗi cấu trúc đầu ra không thể bị xem là chuyện thẩm mỹ.

11.7 Điều gì nên được nói công bằng về BiLSTM, K-Means và bài báo tham chiếu

Một bài phân tích nghiêm túc không chỉ biết tìm lỗi. Nó còn phải biết dừng ở đúng chỗ, để không biến thận trọng thành thiên kiến phủ định.

Trước hết, *BiLSTM trong cửa sổ quá khứ không tự động gây rò rỉ thông tin*. Đây là nơi người đọc hay phản ứng theo thói quen. Nếu mô hình dự báo tại thời điểm t dùng toàn bộ đoạn lịch sử $[t-L+1, \ldots, t]$, thì BiLSTM chỉ đi xuôi và ngược trên cùng một đoạn đã quan sát được. Nó không nhìn vào $t+1, t+2, \ldots$. Vì vậy, cáo buộc "BiLSTM là nhìn vào tương

lai" trong ngữ cảnh này là không chính xác. Điều đáng phê bình hơn là BiLSTM làm việc giải thích động học khó hơn, chứ không phải nó mặc nhiên gây leakage.

Thứ hai, *K-Means để khởi tạo tâm hàm thuộc là một ý tưởng hợp lý nếu dữ liệu đầu vào sạch*. Vấn đề của script không nằm ở bản thân K-Means. Vấn đề nằm ở chỗ dữ liệu được đưa vào K-Means đã bị chuẩn hóa bằng thống kê của toàn bộ chuỗi. Nếu sửa đúng quy trình tách dữ liệu và chuẩn hóa, việc dùng K-Means một chiều để lấy tâm ban đầu cho hai Gaussian vẫn là một lựa chọn có thể bảo vệ được.

Thứ ba, *đối chiếu với Boyacioglu–Avci phải được làm rất tiết chế*. Script nhắc đến bài báo của Boyacioglu và Avci cùng với con số R^2 rất cao. Tuy nhiên, một tài liệu tham khảo chỉ có giá trị trực tiếp khi bài toán, dữ liệu, tần suất, giao thức và cách đo tương đối đồng dạng. Ở đây, script hiện tại sử dụng một thiết lập khác đáng kể. Vì vậy, điều trung thực có thể nói là: kiến trúc này lấy cảm hứng từ việc dùng hai Gaussian membership functions và tinh thần ANFIS trong tài liệu tham khảo. Điều không nên gợi ý là: con số của bài báo kia là hậu thuẫn thực nghiệm trực tiếp cho chất lượng của mô hình hiện tại.

Chính sự công bằng trong ba điểm trên làm cho phần phê bình còn lại có trọng lượng hơn. Ta không kết luận mọi thứ đều sai. Ta chỉ tách cái gì đáng giữ khỏi cái gì phải sửa.

11.8 Nếu muốn biến mô hình này thành một nghiên cứu đáng tin hơn, phải sửa theo thứ tự nào

Khi một script có nhiều vấn đề, phản xạ phổ biến là sửa từ chỗ dễ nhìn thấy nhất: thêm lớp, thay optimizer, tăng số luật, đổi activation, hay làm cho biểu đồ đẹp hơn. Đó gần như luôn là thứ tự sai. Với mô hình này, nếu mục tiêu là tăng giá trị khoa học chứ không chỉ tăng vẻ phức tạp, thứ tự sửa phải đảo lại hoàn toàn.

Ưu tiên đầu tiên phải là **sửa giao thức đánh giá**. Ta cần tạo train/validation/test theo thứ tự thời gian ngay sau khi tạo cửa sổ; fit các bộ chuẩn hóa chỉ trên train; dùng validation cho dừng sớm, giảm tốc độ học và chọn seed; giữ test đúng nghĩa để chỉ đánh giá một lần sau cùng. Chỉ sau khi tầng này sạch, mọi chỉ số RMSE, MAE, MAPE, R^2, Directional Accuracy mới bắt đầu có tư cách được đọc như bằng chứng.

Ưu tiên thứ hai là **sửa câu chuyện diễn giải**. Nếu muốn giữ nhãn LOW/HIGH, ta cần ràng buộc thứ tự của tâm các hàm thuộc, chẳng hạn bằng một tham số hóa kiểu

$$c_2 = c_1 + \text{softplus}(\delta),$$

để bảo đảm $c_2 \geq c_1$. Ta cũng cần trích xuất đầy đủ hệ quả của từng luật Sugeno, thay vì chỉ in tiền đề. Và quan trọng hơn cả, phần mô tả mô hình phải đổi giọng: đây là một *hybrid model with fuzzy components*, không phải một bộ dự báo cuối "giải thích được hoàn toàn".

Ưu tiên thứ ba là **sửa cấu trúc đầu ra**. Nếu bài toán thật sự là dự báo OHLC, ta nên

tham số hóa đầu ra theo cách tôn trọng hình học nến giá, hoặc ít nhất thêm số hạng phạt cho các vi phạm

$$\widehat{H} < \max(\widehat{O}, \widehat{C}), \qquad \widehat{L} > \min(\widehat{O}, \widehat{C}).$$

Một cách khác là đổi sang tham số hóa bằng $(\widehat{C}, \widehat{r}_{\text{up}}, \widehat{r}_{\text{down}}, \widehat{g})$ rồi suy ra O, H, L theo các biến luôn không âm thích hợp. Khi đó, cấu trúc hợp lệ được bảo vệ ngay trong hình học của đầu ra.

Ưu tiên thứ tư mới là **tối ưu kiến trúc**. Chỉ sau khi quy trình sạch và tuyên bố diễn giải trung thực, ta mới nên bàn đến việc BiLSTM có cần hay không, số units bao nhiêu, số hàm thuộc có nên tăng, hay có nên thay Dense cuối bằng một cơ chế trộn có cấu trúc hơn. Sửa kiến trúc trước khi sửa giao thức gần như chỉ làm tăng tốc độ tự đánh lừa mình.

11.9 Kết luận của chương và bài học phương pháp lớn hơn chính script này

`run_feature_group_anfis.py` không phải là một script vô nghĩa. Nó có vài trực giác kiến trúc đáng tôn trọng: chia nhóm đặc trưng để tránh bùng nổ số luật, dùng K-Means để khởi tạo tâm, và cố gắng ghép tri thức mờ với động học thời gian. Nhưng chính vì có những trực giác tốt ấy, nó càng cần được đọc với chuẩn mực cao hơn.

Kết luận quan trọng nhất của chương là thế này: các lỗi nặng nhất của script không nằm ở "mô hình có dùng ANFIS đúng nghĩa thuần túy hay không", mà nằm ở chỗ các chỉ số cuối được sản sinh ra bởi một giao thức đã dùng thông tin của test ở nhiều tầng. Sau đó, trên nền các chỉ số ấy, script lại kể một câu chuyện diễn giải mạnh hơn mức mà cấu trúc mô hình thực sự cho phép. Nếu không tách hai lớp sai này ra, người đọc rất dễ hoặc bị mê hoặc bởi phần fuzzy, hoặc phản ứng ngược lại bằng cách phủ định sạch mọi ý tưởng trong mô hình.

Giá trị sư phạm lớn nhất của chương này vì vậy không chỉ là ở việc nó chỉ ra vài lỗi của một file Python cụ thể. Giá trị lớn hơn là ở một thói quen trí tuệ: khi đọc bất kỳ mô hình nào, hãy luôn hỏi đồng thời ba câu. Bài toán toán học thực sự là gì. Code có thực hiện đúng bài toán ấy không. Và giao thức đánh giá có đủ sạch để các chỉ số cuối còn đáng tin hay không. Nếu giữ được ba câu hỏi đó trong đầu, người học sẽ tránh được cả hai cạm bẫy phổ biến nhất của thời đại học máy: tin quá nhanh vào những mô hình trông ấn tượng, và phủ định quá nhanh những ý tưởng còn có thể cứu được bằng một quy trình đúng.

Chuong 12

Tổng kết, định hướng tự học và bài tập

12.1 Sau khi đi hết giáo trình này, điều gì thật sự cần ở lại trong đầu người học

Nếu chỉ nhớ lại tên các mô hình đã đi qua, người học mới chỉ mang được phần vỏ của giáo trình. Điều đáng giữ lại sâu hơn là một *chuẩn mực nội tại* khi nhìn vào dữ liệu, mô hình và kết quả. Chuẩn mực đó có thể tóm vào ba trục mà ta đã đi qua từ đầu đến cuối cuốn sách.

Trục thứ nhất là trục *thống kê và suy luận ngoài mẫu*. Từ hồi quy tuyến tính, điều chuẩn, xác suất có điều kiện, cho đến validation, test, leakage và backtesting, bài học xuyên suốt là: mô hình không thể được đánh giá tách rời khỏi giao thức sinh ra chỉ số. Một bảng R^2 cao không tự nói lên điều gì nếu ta không biết cách dữ liệu được chia, cách siêu tham số được chọn, và test có bị lạm dụng hay không. Người học trưởng thành phải có phản xạ xem phương pháp trước khi xem thành tích.

Trục thứ hai là trục *biểu diễn và tối ưu*. Từ các mô hình tuyến tính, cây, hạt nhân, mạng nơ-ron, đến LSTM và BiLSTM, ta đã thấy rằng mọi mô hình học máy đều là một cách xấp xỉ hàm với một thiên kiến quy nạp riêng. Không có kiến trúc nào thần kỳ ngoài ngữ cảnh. Kiến trúc càng phức tạp thì năng lực biểu diễn càng mạnh, nhưng cùng lúc số cách thất bại cũng nhiều hơn: tối ưu khó hơn, quá khớp dễ hơn, diễn giải mờ hơn. Bài học lớn ở đây là không được nhầm năng lực biểu diễn với giá trị khoa học.

Trục thứ ba là trục *logic mờ và ngôn ngữ khái niệm*. Tập mờ, hàm thuộc, T-norm, hệ Sugeno, ANFIS cho ta một cách mô hình hóa những khái niệm mà con người dùng hằng ngày nhưng logic hai giá trị mô tả quá thô. Tuy nhiên, chương về ANFIS và chương phân tích code cũng chỉ ra rằng tính diễn giải không tự sinh ra chỉ vì mô hình có luật. Muốn giữ được diễn giải, ta phải giữ được nghĩa của nhãn mờ, số lượng luật ở mức đọc được, và phạm vi phát biểu trung thực về phần nào của mô hình thật sự được luật chi phối.

Ba trục đó giao nhau ở một năng lực quan trọng hơn mọi tên gọi mô hình: năng lực đọc một hệ thống dự báo như một công trình khoa học. Khi cầm một script, một bài báo, hay một

notebook, ta không chỉ hỏi "nó có hiện đại không" mà phải hỏi "nó đúng bài toán không, đúng giao thức không, đúng ngôn ngữ khái niệm không". Nếu sau giáo trình này người học hình thành được thói quen ấy, thì phần lớn mục tiêu của sách đã đạt.

12.2 Tự học tiếp như thế nào để không sa vào việc chỉ sưu tập tên mô hình

Một nguy cơ rất phổ biến sau khi học một lượng lớn kiến thức ML/DL là người học bước sang giai đoạn "sưu tập kiến trúc". Cứ mỗi tuần họ biết thêm một tên mới, một sơ đồ mới, một bài báo mới, nhưng năng lực đặt bài toán và kiểm tra giao thức thì không tăng tương xứng. Cách tự học như vậy thường tạo cảm giác tiến bộ nhanh nhưng nền móng rất rỗng.

Muốn tránh điều đó, thứ tự tự học nên đi từ *chuẩn mực khoa học* sang *kiến trúc*, chứ không phải ngược lại. Nghĩa là, trước mỗi kiến trúc mới, ta tự hỏi năm câu. Bài toán đầu vào–đầu ra là gì. Tín hiệu nào có thể biết tại thời điểm dự báo. Mất mát nào phản ánh đúng mục tiêu ra quyết định. Giao thức tách dữ liệu nào phù hợp. Và cuối cùng, kiến trúc mới này thêm thiên kiến quy nạp gì so với các mô hình mình đã biết. Nếu không đi qua năm câu này, việc học thêm mô hình mới chỉ làm bộ sưu tập thuật ngữ của ta dày lên.

Về lộ trình đọc sâu hơn, một cách bền vững là chia theo ba tuyến song song. Tuyến thứ nhất là tuyến nền tảng toán học và thống kê: đọc lại xác suất có điều kiện, suy luận Bayes, tối ưu lồi, bất đẳng thức, và các lập luận về tổng quát hóa. Đây là tuyến giúp người học không bị ngợp trước ký hiệu trong các tài liệu khó hơn. Tuyến thứ hai là tuyến mô hình: mỗi giai đoạn chỉ chọn một họ mô hình để đào sâu, ví dụ một tháng cho mô hình tuyến tính và điều chuẩn, một tháng cho mạng hồi tiếp, một tháng cho mô hình mờ. Tuyến thứ ba là tuyến thực nghiệm: luôn gắn việc đọc với một codebase thật, vì chỉ khi đọc code thật người học mới thấy lý thuyết nào thật sự kiểm soát được sai sót.

Trong danh mục tài liệu, các cuốn của Bishop, Hastie–Tibshirani–Friedman, Murphy, Goodfellow và Hyndman là các mốc rất tốt, nhưng cách đọc quan trọng hơn tên sách. Không nên đọc với mục tiêu "đi cho hết". Nên đọc với mục tiêu "mỗi chương phải để lại cho mình một tiêu chuẩn kiểm tra mô hình". Chẳng hạn, sau chương về *ridge*, ta phải có khả năng tự chứng minh vì sao ma trận $X^\top X + \lambda I$ khả nghịch khi $\lambda > 0$. Sau chương về validation, ta phải nhìn một script và nhận ra ngay chỗ nào test đang bị dùng sai. Sau chương về ANFIS, ta phải hiểu vì sao nhãn LOW/HIGH không thể mặc định giữ nguyên nghĩa sau khi tham số được học tự do.

Một nguyên tắc tự học nữa là luôn xen kẽ *đọc tiến* với *đọc lùi*. Đọc tiến là học thêm một ý tưởng mới. Đọc lùi là quay lại một code cũ, một mô hình cũ, hoặc một bài báo cũ rồi xem mình đọc nó sâu hơn được bao nhiêu sau khi đã có kiến thức mới. Người học đi nhanh nhưng chắc là người có thể đọc lùi tốt. Chính chương phân tích `run_feature_group_anfis.py` là một ví dụ của đọc lùi: ta dùng toàn bộ lý thuyết trước đó để soi lại một hệ thống cụ thể.

12.3 Nếu viết lại mô hình trong repo như một nghiên cứu nghiêm túc, ta phải bắt đầu từ đâu

Phần này có giá trị đặc biệt vì nó biến toàn bộ giáo trình từ "kiến thức để biết" thành "kiến thức để làm". Giả sử mục tiêu của ta không chỉ là sửa một script cho chạy được, mà là nâng nó thành một thí nghiệm có giá trị khoa học hơn. Câu hỏi đầu tiên không phải là "thêm lớp gì", mà là "định nghĩa lại bài toán ra sao cho sáng sủa và kiểm chứng được".

Trước hết, ta phải chốt rõ bài toán dự báo. Có ít nhất ba lựa chọn khác nhau và không nên trộn lẫn chúng: dự báo trực tiếp mức giá OHLC của ngày mai; dự báo các tỷ suất thay đổi của OHLC theo một mốc chuẩn cố định; hoặc dự báo một bộ tham số cấu trúc của nến giá, chẳng hạn giá đóng cửa tương lai cộng với biên độ trên/dưới tương đối. Mỗi cách đặt bài toán kéo theo một hình học đầu ra, một hàm mất mát, và một câu chuyện diễn giải khác nhau. Nếu chưa chốt điều này mà đã bàn kiến trúc, thì kiến trúc mới chỉ là trang trí trên một bài toán mơ hồ.

Sau khi chốt bài toán, bước thứ hai là thiết kế *giao thức thời gian*. Dữ liệu chuỗi không thể bị cắt như dữ liệu độc lập cùng phân phối. Ta cần ít nhất ba đoạn theo thứ tự thời gian: train, validation, test. Trong một bài toán tài chính, nhiều nhóm còn dùng *walk-forward validation* hay *rolling origin evaluation* để phản ánh sự thay đổi chế độ. Điều cốt lõi là mọi thống kê học được trong pipeline, từ chuẩn hóa đến chọn siêu tham số, đều phải được xây từ vùng thời gian thích hợp và không được hút thông tin từ tương lai.

Bước thứ ba là thiết kế lại pipeline theo đúng nghĩa "học từ train". Nếu ta chuẩn hóa, bộ chuẩn hóa phải được *fit* trên train rồi *transform* sang validation và test. Nếu ta khởi tạo tâm hàm thuộc bằng K-Means, K-Means cũng phải nhìn dữ liệu train sau chuẩn hóa train mà thôi. Nếu ta chọn số luật, số units hay seed, quyết định đó phải dựa trên validation. Test chỉ được phép xuất hiện như một nghi lễ cuối cùng, một lần, sau khi mọi quyết định đã khóa.

Bước thứ tư là sửa mô hình để tôn trọng cấu trúc đầu ra. Với dữ liệu OHLC, đây không phải một chi tiết đẹp đẽ tùy chọn. Đầu ra không hợp lệ về hình học có thể khiến cả một dự báo "đẹp chỉ số" trở nên khó dùng. Một cách sửa là thêm số hạng phạt

$$\mathcal{L}_{\text{ohlc}} = \lambda_1 \,\text{ReLU}\big(\max(\widehat{O}, \widehat{C}) - \widehat{H}\big) \;+\; \lambda_2 \,\text{ReLU}\big(\widehat{L} - \min(\widehat{O}, \widehat{C})\big),$$

vào hàm mất mát tổng. Một cách mạnh hơn là chọn tham số hóa đầu ra sao cho bất đẳng thức hợp lệ được thỏa ngay theo định nghĩa.

Bước thứ năm là làm rõ mục tiêu diễn giải. Nếu vẫn muốn dùng nhánh mờ như một phần "đọc được" của mô hình, ta cần quyết định mình muốn giải thích cái gì. Muốn giải thích biểu diễn cục bộ ở bước cuối, hay muốn giải thích dự báo cuối. Hai mục tiêu ấy khác nhau. Nếu chỉ giải thích biểu diễn cục bộ, ta nên nói thẳng điều đó và không gán cho toàn bộ hệ thống danh hiệu "fully explainable". Nếu muốn tiến gần hơn tới giải thích đầu ra cuối, ta phải hoặc làm vai trò của nhánh mờ chi phối hơn, hoặc thiết kế cơ chế trộn sao cho đóng góp

của từng nhánh còn truy vết được.

Bước thứ sáu là nâng tiêu chuẩn báo cáo. Một báo cáo tốt không chỉ nêu chỉ số cuối cùng mà phải trả lời các câu hỏi: kết quả ổn định đến đâu theo seed; có vi phạm OHLC hợp lệ không; nhánh mờ đóng góp gì so với mô hình không có nhánh mờ; nếu bỏ BiLSTM thì mất gì; nếu bỏ K-Means khởi tạo thì có thay đổi lớn không. Đây chính là tinh thần của *ablation study* (nghiên cứu triệt giảm thành phần): thay vì chỉ khoe mô hình đầy đủ, ta phải chứng minh từng mảnh của mô hình có giá trị gì.

12.4 Bài tập tự luyện: từ toán nền đến đọc code thật

Phần bài tập dưới đây không được thiết kế như một danh sách để đủ mục. Mỗi bài nhắm vào một kỹ năng mà người học cần thật sự có sau giáo trình: kỹ năng chứng minh, kỹ năng diễn giải mô hình, kỹ năng thiết kế giao thức, và kỹ năng đọc code với kỷ luật khoa học.

Nhóm 1: bài tập nền tảng toán học và thống kê. Đây là nhóm giúp người học không chỉ nhớ kết luận, mà còn nắm được cấu trúc lập luận phía sau kết luận.

Bài tập 12.1. Chứng minh rằng trong hồi quy *ridge*, ma trận $X^\top X + \lambda I$ luôn khả nghịch với mọi $\lambda > 0$, ngay cả khi $X^\top X$ suy biến. Sau đó giải thích bằng ngôn ngữ hình học vì sao điều này có nghĩa là bài toán tối ưu trở nên "được khóa chặt" hơn so với bình phương tối thiểu thuần túy.

Bài tập 12.2. Chứng minh rằng hàm dự báo tối ưu theo mất mát MSE là kỳ vọng có điều kiện, còn theo mất mát MAE là trung vị có điều kiện. Không chỉ viết công thức kết quả; hãy chỉ ra chính xác ở bước nào tính lồi của trị tuyệt đối và bình phương được dùng.

Bài tập 12.3. Cho một tập mờ A trên $\mathbb{R}$. Chứng minh lại định lý biểu diễn qua α-cut:

$$\mu_A(x) = \sup\{\alpha \in (0, 1] : x \in A_\alpha\}.$$

Sau đó giải thích vì sao kết quả này làm cho lý thuyết số mờ tính toán được hơn nhiều.

Nhóm 2: bài tập về logic mờ và ANFIS. Mục tiêu của nhóm này là buộc người học nối định nghĩa với cấu trúc mô hình cụ thể.

Bài tập 12.4. Xét một hệ Sugeno có hai luật, một đầu vào x, và hai hàm thuộc Gaussian. Hãy viết công thức đầu ra cuối cùng dưới dạng trung bình có trọng số, rồi tính đạo hàm của đầu ra theo tâm của hàm thuộc thứ nhất. Sau đó bình luận xem ở đâu trong công thức này xuất hiện nguy cơ gradient rất nhỏ.

Bài tập 12.5. Thiết kế một phân hoạch mờ ba nhãn *THAP, TRUNG BINH, CAO* cho đại lượng tỷ suất thay đổi ngày trên miền $[-0.1, 0.1]$. Hãy làm hai phiên bản: một bằng tam giác, một bằng Gaussian. So sánh hai phiên bản theo ba tiêu chí: tính diễn giải, độ trơn để tối ưu, và độ nhạy khi dữ liệu lệch phân phối.

Bài tập 12.6. Giả sử bạn có hai hàm thuộc Gaussian được gắn nhãn LOW và HIGH. Hãy đề xuất một cách tham số hóa để luôn bảo đảm tâm của HIGH không nhỏ hơn tâm của LOW trong suốt quá trình học. Sau đó phân tích xem ràng buộc đó giúp gì cho diễn giải và có thể làm hại gì cho độ tự do biểu diễn.

Nhóm 3: bài tập về thiết kế mô hình và đánh giá chuỗi thời gian. Đây là nhóm bài tập biến kiến thức lý thuyết thành khả năng dựng thí nghiệm đúng.

Bài tập 12.7. Viết lại một pipeline tạo cửa sổ chuỗi thời gian gồm train, validation, test theo thứ tự thời gian. Chỉ ra thật rõ các bước nào được phép *fit* trên train và các bước nào chỉ được *transform*. Sau đó thử cố ý làm sai bằng cách fit bộ chuẩn hóa trên toàn bộ dữ liệu và mô tả sự khác biệt về logic đánh giá.

Bài tập 12.8. Đề xuất một tham số hóa đầu ra cho bài toán OHLC sao cho các ràng buộc

$$\widehat{H} \geq \max(\widehat{O}, \widehat{C}), \qquad \widehat{L} \leq \min(\widehat{O}, \widehat{C})$$

được bảo đảm theo định nghĩa, không cần thêm penalty. Gợi ý: dùng giá đóng cửa dự báo và hai biên độ không âm.

Bài tập 12.9. Thiết kế một *ablation study* tối thiểu cho mô hình lai mờ + BiLSTM. Bạn sẽ so sánh những biến thể nào để trả lời ba câu hỏi: nhánh mờ có ích không, BiLSTM có ích không, và khởi tạo K-Means có ích không. Đừng chỉ liệt kê mô hình; hãy nêu rõ tiêu chí đọc kết quả.

Nhóm 4: bài tập đọc code và kiểm toán kỹ thuật. Đây là nhóm gần nhất với công việc của một nhà nghiên cứu hoặc kỹ sư phải đánh giá một repo thật.

Bài tập 12.10. Mở lại file `run_feature_group_anfis.py` và viết toàn bộ hàm `prepare_data` bằng ký hiệu toán học. Hãy định nghĩa chính xác từng đặc trưng, từng mục tiêu, từng tensor đầu ra, và chỉ ra tại bước nào điều kiện thông tin của chuỗi thời gian bị vi phạm.

Bài tập 12.11. Trong `run_feature_group_anfis.py`, hãy chỉ ra mọi nơi mà tập test được dùng trước khi báo cáo kết quả cuối. Sau đó viết lại quy trình đúng bằng mã giả. Bài này chỉ hoàn thành khi bạn phân biệt được rõ vai trò của train, validation và test bằng lời của chính mình.

Bài tập 12.12. Viết một hàm trích xuất đầy đủ hệ quả của từng luật trong lớp `FeatureGroupANFIS`. Mục tiêu cuối là in được luật dưới dạng:

$$\text{IF } \cdots \text{ THEN } y_k = a_0 + a_1 x_1 + \cdots + a_d x_d.$$

Sau đó, bình luận xem việc có luật đầy đủ đã đủ để gọi đầu ra cuối của mô hình là giải thích được hay chưa.

Bài tập 12.13. Hãy thử thiết kế một phiên bản không dùng BiLSTM, chỉ dùng các mô hình Sugeno cục bộ hoặc ANFIS nhiều đầu ra. So sánh trung thực với phiên bản lai hiện tại về ba mặt: độ chính xác kỳ vọng, chi phí huấn luyện, và mức độ diễn giải có thể bảo vệ được.

12.5 Kết lời

Kết thúc một giáo trình dài, điều người học thường cần nhất không phải là thêm một danh sách thuật ngữ, mà là một cảm giác định hướng rõ ràng: từ đây mình nên nhìn mô hình như thế nào, nên học tiếp theo nhịp nào, và nên tự kiểm tra mình bằng loại bài tập nào. Nếu chương cuối này làm tốt nhiệm vụ của nó, thì người đọc sẽ rời cuốn sách không chỉ với một túi kiến thức, mà với một chuẩn mực làm việc: tôn trọng bài toán, tôn trọng giao thức, tôn trọng ngữ nghĩa, và đủ trung thực để không gọi một mô hình là "giải thích được" khi phần có thể giải thích thật ra chỉ là một mảnh nhỏ của toàn hệ thống.

Giữ được chuẩn mực ấy, bạn sẽ học nhanh hơn, nhưng quan trọng hơn, bạn sẽ học chắc hơn. Và trong một lĩnh vực đang thay đổi rất nhanh như ML, DL và logic mờ ứng dụng, học chắc luôn có giá trị lâu hơn học ào ạt.

Chuong 13

Phụ lục: ví dụ tính tay, quy trình chuẩn và khung sửa mô hình

13.1 Mục đích của phụ lục

Phần phụ lục này không phải nơi gom những mẩu vụn còn sót lại, mà là nơi làm ba việc quan trọng. Thứ nhất, ta giải một vài ví dụ tính tay để nối lại trực giác toán học với công thức. Thứ hai, ta viết ra một quy trình thí nghiệm chuẩn cho bài toán dự báo chuỗi thời gian có ANFIS, nhằm tránh các lỗi phương pháp luận đã chỉ ra ở Chương 11. Thứ ba, ta phác họa một phiên bản sửa của mô hình trong `run_feature_group_anfis.py` theo tinh thần vừa chặt chẽ thống kê, vừa trung thực về diễn giải.

13.2 Ví dụ tính tay 1: hồi quy tuyến tính, đạo hàm và nghiệm đóng

Xét ba điểm dữ liệu một chiều:

$$(x_1, y_1) = (1, 2), \qquad (x_2, y_2) = (2, 3), \qquad (x_3, y_3) = (3, 5).$$

Ta muốn khớp mô hình

$$\hat{y} = wx + b.$$

Đặt ma trận thiết kế mở rộng

$$X = \begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{bmatrix}, \qquad \boldsymbol{y} = \begin{bmatrix} 2 \\ 3 \\ 5 \end{bmatrix}, \qquad \boldsymbol{\theta} = \begin{bmatrix} w \\ b \end{bmatrix}.$$

Hàm mất mát bình phương là

$$L(\boldsymbol{\theta}) = \|X\boldsymbol{\theta} - \boldsymbol{y}\|_2^2.$$

Phương trình chuẩn cho ta

$$X^\top X\boldsymbol{\theta} = X^\top \boldsymbol{y}.$$

Ta tính được

$$X^\top X = \begin{bmatrix} 14 & 6 \\ 6 & 3 \end{bmatrix}, \qquad X^\top \boldsymbol{y} = \begin{bmatrix} 23 \\ 10 \end{bmatrix}.$$

Giải hệ:

$$\begin{bmatrix} 14 & 6 \\ 6 & 3 \end{bmatrix} \begin{bmatrix} w \\ b \end{bmatrix} = \begin{bmatrix} 23 \\ 10 \end{bmatrix}.$$

Từ phương trình thứ hai,

$$6w + 3b = 10 \Longrightarrow 2w + b = \frac{10}{3}.$$

Từ đó $b = \frac{10}{3} - 2w$. Thay vào phương trình đầu:

$$14w + 6\left(\frac{10}{3} - 2w\right) = 23 \Longrightarrow 14w + 20 - 12w = 23 \Longrightarrow 2w = 3 \Longrightarrow w = 1.5.$$

Suy ra

$$b = \frac{10}{3} - 3 = \frac{1}{3}.$$

Vậy mô hình tốt nhất theo bình phương tối thiểu là

$$\hat{y} = 1.5x + \frac{1}{3}.$$

Ví dụ này nhỏ, nhưng nó làm rõ một điều rất lớn: nghiệm đóng không phải điều gì huyền bí, mà là hệ quả của một cấu trúc lồi rất mạnh. Khi ta rời thế giới này để sang ANFIS hay học sâu, điều ta đánh mất trước hết là sự đơn giản của bề mặt tối ưu.

13.3 Ví dụ tính tay 2: suy diễn mờ Sugeno với hai luật

Xét một đầu vào $x \in \mathbb{R}$ và hai luật:

Rule 1: Nếu x là LOW thì $y_1 = 1.0x + 0.5$,

Rule 2: Nếu x là HIGH thì $y_2 = 2.0x - 0.2$.

Giả sử hai hàm thuộc Gaussian:

$$\mu_{\text{LOW}}(x) = \exp\left(-\frac{(x-0)^2}{2(1)^2}\right), \qquad \mu_{\text{HIGH}}(x) = \exp\left(-\frac{(x-2)^2}{2(1)^2}\right).$$

Lấy $x = 1$. Khi đó

$$w_1 = \mu_{\text{LOW}}(1) = e^{-1/2}, \qquad w_2 = \mu_{\text{HIGH}}(1) = e^{-1/2}.$$

Do hai độ kích hoạt bằng nhau, sau chuẩn hóa ta có

$$\bar{w}_1 = \bar{w}_2 = \frac{1}{2}.$$

Phần hệ quả:

$$y_1 = 1.0(1) + 0.5 = 1.5, \qquad y_2 = 2.0(1) - 0.2 = 1.8.$$

Đầu ra cuối:

$$y = \bar{w}_1 y_1 + \bar{w}_2 y_2 = \frac{1}{2}(1.5) + \frac{1}{2}(1.8) = 1.65.$$

Ví dụ này cho thấy trực giác của Sugeno: đầu ra không phải chọn duy nhất một luật, mà là trung bình mềm của các luật đang hoạt động. Khi nhiều luật cùng kích hoạt, dự báo cuối là kết quả pha trộn cục bộ.

13.4 Ví dụ tính tay 3: ANFIS hai đầu vào, bốn luật

Xét hai đầu vào x_1, x_2, mỗi đầu vào có hai nhãn LOW và HIGH. Khi đó có bốn luật:

$$\begin{aligned}
R_1 &: (x_1 \text{ LOW}, x_2 \text{ LOW}),\\
R_2 &: (x_1 \text{ LOW}, x_2 \text{ HIGH}),\\
R_3 &: (x_1 \text{ HIGH}, x_2 \text{ LOW}),\\
R_4 &: (x_1 \text{ HIGH}, x_2 \text{ HIGH}).
\end{aligned}$$

Nếu dùng tích làm T-norm, độ kích hoạt là

$$w_1 = \mu_{L1}(x_1)\mu_{L2}(x_2), \quad w_2 = \mu_{L1}(x_1)\mu_{H2}(x_2), \quad w_3 = \mu_{H1}(x_1)\mu_{L2}(x_2), \quad w_4 = \mu_{H1}(x_1)\mu_{H2}(x_2).$$

Giả sử mỗi luật có hệ quả affine

$$f_r(x_1, x_2) = p_{r0} + p_{r1}x_1 + p_{r2}x_2.$$

Khi đó đầu ra cuối là

$$y = \sum_{r=1}^{4} \bar{w}_r f_r(x_1, x_2), \qquad \bar{w}_r = \frac{w_r}{\sum_{s=1}^{4} w_s}.$$

Điều đáng chú ý ở đây là phần phi tuyến của mô hình nằm trọn trong $\bar{w}_r$, còn phần hệ quả là tuyến tính. Đây chính là lý do ANFIS có thể tận dụng bình phương tối thiểu ở một nửa bài toán.

13.5 Quy trình chuẩn cho dữ liệu chuỗi thời gian có ANFIS và học sâu

Ta viết ra quy trình chuẩn dưới dạng tuần tự, rồi phân tích lý do.

13.5.1 Bước 1: xác định mục tiêu thật rõ

Ta phải chốt chính xác biến mục tiêu là gì. Là giá tuyệt đối? Là tỷ suất đơn giản? Là log-return? Là dự báo một bước hay nhiều bước? Là trực tiếp dự báo (O, H, L, C) hay dự báo một biểu diễn khác rồi quy đổi?

Nếu câu hỏi này mờ, toàn bộ những bước sau sẽ mang tính ứng biến. Trong script đã phân tích, biến mục tiêu thực tế là bốn tỷ lệ của $\text{Open}_{t+1}, \text{High}_{t+1}, \text{Low}_{t+1}, \text{Close}_{t+1}$ so với Close_t. Chỉ riêng việc nói thật rõ điều đó đã giúp ta nhìn ra những điểm không nhất quán trong diễn giải.

13.5.2 Bước 2: tách dữ liệu theo thời gian trước khi học bất kỳ biến đổi nào

Trật tự đúng là:

1. tạo các hàng dữ liệu thời gian hợp lệ;
2. tạo cửa sổ;
3. tách thành tập huấn luyện, tập thẩm định, tập kiểm tra theo thời gian;
4. chỉ sau đó mới fit bộ chuẩn hóa trên tập huấn luyện.

Điều này nghe có vẻ nhỏ, nhưng đây chính là đường ranh giữa một thí nghiệm có giá trị và một thí nghiệm bị rò rỉ thông tin.

13.5.3 Bước 3: khởi tạo khối mờ trên tập huấn luyện

Nếu dùng K-Means để khởi tạo tâm của hàm thuộc, K-Means phải được fit trên tập huấn luyện sau khi chuẩn hóa bằng thống kê của tập huấn luyện. Không được phép dùng tập thẩm định hay tập kiểm tra trong bước này.

13.5.4 Bước 4: huấn luyện với tập thẩm định đúng nghĩa

Mọi cơ chế ra quyết định trong quá trình học, bao gồm dừng sớm, giảm tốc độ học, chọn *seed*, chọn số lượng hàm thuộc, chọn số lớp LSTM, đều phải dựa trên tập thẩm định.

13.5.5 Bước 5: khóa mô hình rồi mới động đến tập kiểm tra

Khi mọi quyết định đã khóa, ta mới báo cáo kết quả trên tập kiểm tra. Tập kiểm tra không phải là nơi để "xem thử rồi chỉnh tiếp".

13.6 Mã giả cho quy trình sửa của script

```
1. load raw dataframe
2. build features and targets using only information up to time t
3. build rolling windows
4. split windows chronologically into train / val / test
5. fit feature_scaler on train only
6. fit target_scaler on train only
7. transform train / val / test using those fitted scalers
8. compute K-Means centers on train only
9. train model on train, monitor val
10. choose best checkpoint using val only
11. evaluate test once
12. export metrics + rule antecedents + rule consequents + error cases
```

Mã giả này trông thô, nhưng nó chính là xương sống của tính đáng tin trong thí nghiệm.

13.7 Khung sửa mô hình để giữ ràng buộc OHLC

Nếu mục tiêu là dự báo một bộ giá trị OHLC hợp lệ, ta cần biến đổi cách tham số hóa đầu ra.

13.7.1 Phương án 1: dự báo $(O, C, \Delta_H, \Delta_L)$

Ta dự báo

$$\hat{O}_{t+1}, \qquad \hat{C}_{t+1}, \qquad \widehat{\Delta_H}_{t+1} \geq 0, \qquad \widehat{\Delta_L}_{t+1} \geq 0,$$

rồi đặt

$$\hat{H}_{t+1} = \max(\hat{O}_{t+1}, \hat{C}_{t+1}) + \widehat{\Delta_H}_{t+1},$$
$$\hat{L}_{t+1} = \min(\hat{O}_{t+1}, \hat{C}_{t+1}) - \widehat{\Delta_L}_{t+1}.$$

Để ép không âm, có thể dùng softplus ở hai đầu ra Δ_H, Δ_L.

13.7.2 Phương án 2: thêm số hạng phạt vào hàm mất mát

Nếu vẫn muốn dự báo trực tiếp bốn giá trị, ta có thể thêm

$$\mathcal{L}_{\text{valid}} = \lambda_1 \,\text{ReLU}\big(\max(\hat{O}, \hat{C}) - \hat{H}\big) + \lambda_2 \,\text{ReLU}\big(\hat{L} - \min(\hat{O}, \hat{C})\big).$$

Hàm mất mát tổng sẽ là

$$\mathcal{L} = \mathcal{L}_{\text{forecast}} + \mathcal{L}_{\text{valid}}.$$

Phương án này mềm hơn nhưng không đảm bảo tuyệt đối nếu λ_1, λ_2 không đủ lớn.

13.8 Khung sửa mô hình để giữ nghĩa của nhãn LOW/HIGH

Một lỗi diễn giải quan trọng trong script là nhãn LOW/HIGH có thể đổi nghĩa sau huấn luyện. Có nhiều cách sửa.

13.8.1 Cách 1: tham số hóa có thứ tự

Thay vì học hai tâm c_1, c_2 độc lập, ta đặt

$$c_1 = a, \qquad c_2 = a + \text{softplus}(d),$$

trong đó a, d là tham số tự do. Khi đó luôn có $c_2 \geq c_1$. Nhãn LOW/HIGH được giữ ổn định về thứ tự.

13.8.2 Cách 2: sắp xếp lại sau mỗi vòng cập nhật

Cách này đơn giản hơn nhưng kém trơn hơn: sau mỗi bước hoặc mỗi epoch, sắp xếp lại các tâm theo thứ tự tăng dần và hoán vị các tham số liên quan. Tuy nhiên, việc hoán vị này cần làm rất cẩn thận để không phá vỡ liên kết với hệ quả và mô tả luật.

13.9 Khung sửa phần trích xuất luật

Muốn thật sự có giá trị diễn giải, phần trích xuất luật phải gồm cả tiền đề và hệ quả. Với mỗi luật r và mỗi đầu ra k, ta nên ghi rõ:

$$\text{IF } \cdots \text{ THEN } y_k = a_{r0}^{(k)} + a_{r1}^{(k)} x_1 + \cdots + a_{rd}^{(k)} x_d.$$

Ngoài ra, ta còn nên ghi:

1. tâm và độ rộng hiện tại của từng hàm thuộc;
2. mức kích hoạt trung bình của luật trên tập huấn luyện/thẩm định;
3. độ đóng góp trung bình của luật vào mỗi đầu ra.

Như vậy mới có thể nói đến phân tích luật một cách nghiêm túc.

13.10 Một ví dụ về báo cáo lỗi tốt hơn cho mô hình

Thay vì chỉ báo cáo "Close $R^2 = 0.98$", một báo cáo tử tế hơn nên có ít nhất các nhóm mục sau.

13.10.1 Nhóm 1: chỉ số dự báo

RMSE, MAE, MAPE, R^2, độ chính xác hướng, trên tập thẩm định và tập kiểm tra, với mô tả rõ ràng về biến mục tiêu.

13.10.2 Nhóm 2: chỉ số hợp lệ cấu trúc

Tỷ lệ mẫu vi phạm $\hat{H} < \max(\hat{O}, \hat{C})$, tỷ lệ mẫu vi phạm $\hat{L} > \min(\hat{O}, \hat{C})$, và phân bố độ vi phạm.

13.10.3 Nhóm 3: chỉ số diễn giải

Các luật kích hoạt nhiều nhất, hệ số hệ quả quan trọng nhất, mức ổn định của các tâm hàm thuộc qua nhiều lần huấn luyện.

13.10.4 Nhóm 4: chỉ số phương pháp luận

Tập nào được dùng để fit bộ chuẩn hóa, tập nào được dùng cho dừng sớm, số lần chạy, và cách chọn mô hình cuối.

Khi báo cáo được tổ chức như vậy, nhiều ảo giác thành tích sẽ tự biến mất.

13.11 Một lời nhắc cuối về tinh thần học

Phụ lục này khép lại tài liệu bằng một thái độ quan trọng hơn cả công thức: hãy luôn hỏi mô hình đang *học cái gì*, đang *được đánh giá như thế nào*, và đang *được kể lại trung thực đến đâu*. Người mới thường nghĩ học máy khó vì có nhiều công thức. Thực ra, phần khó hơn nhiều là giữ được tính trung thực trí tuệ khi một kết quả đẹp xuất hiện trước mắt.

Nếu bạn giữ được thái độ đó, bạn sẽ không chỉ học được nhiều mô hình hơn, mà còn tránh được rất nhiều sai lầm mà người khác có thể mất nhiều tháng mới nhìn ra.

Chuong 14

Những ngộ nhận thường gặp khi học ML, DL, logic mờ và ANFIS

14.1 Ngộ nhận không chỉ là câu sai, mà là một khung nhìn sai

Điều làm ngộ nhận nguy hiểm không phải chỉ là nó dẫn tới một kết luận sai ở một chỗ cụ thể. Điều nguy hiểm hơn là nó thay cả bộ câu hỏi mà người học dùng để đọc mô hình. Khi khung nhìn đã sai, người học có thể nhìn đúng công thức mà vẫn rút ra kết luận sai, đơn giản vì họ đang hỏi nhầm điều cần hỏi.

Ví dụ, nếu người học mang sẵn trong đầu mệnh đề "mô hình càng phức tạp thì càng mạnh", họ sẽ vô thức xem số tầng, số nhánh, số thành phần là bằng chứng thay cho giao thức đánh giá. Nếu người học tin rằng "có fuzzy thì chắc chắn dễ giải thích", họ sẽ dừng lại ở vài câu luật in ra, mà không kiểm tra xem đầu ra cuối có còn đi trực tiếp qua cơ chế luật đó hay không. Nếu người học cho rằng "test score cao là đủ", họ sẽ bỏ qua câu hỏi quan trọng hơn nhiều: test có còn là test theo đúng nghĩa phương pháp luận hay không.

Vì vậy, chương này không viết như một bài cảnh báo chung chung. Mỗi cụm ngộ nhận dưới đây sẽ được bóc tách bằng ví dụ, phản ví dụ, hoặc một mệnh đề ngắn có chứng minh. Mục tiêu là để người đọc không chỉ nghe "đừng nghĩ như thế", mà thật sự thấy *vì sao* cách nghĩ đó sai.

14.2 Cụm ngộ nhận thứ nhất: nhầm độ phức tạp kiến trúc và bảng chỉ số với sức mạnh khoa học

Ngộ nhận đầu tiên là xem độ phức tạp biểu diễn như bằng chứng đủ cho chất lượng khoa học. Điều đúng chỉ là: mô hình phức tạp hơn thường có không gian biểu diễn giàu hơn. Điều

sai là: giàu hơn về biểu diễn đồng nghĩa với đáng tin hơn về kết luận.

Ví dụ 14.1. Xét hai mô hình cho cùng một bài toán dự báo chuỗi thời gian.

Mô hình A là một hệ lai gồm BiLSTM, hai nhánh ANFIS, nhiều *callbacks*, và khoảng 50 000 tham số. Nhưng nó chuẩn hóa trên toàn bộ dữ liệu trước khi tách train/test và dùng test để dừng sớm.

Mô hình B chỉ là một hồi quy tuyến tính có điều chuẩn với khoảng 30 tham số. Nhưng nó tách train/validation/test đúng theo thời gian, chỉ fit chuẩn hóa trên train, chọn siêu tham số trên validation, và chỉ đo test đúng một lần sau cùng.

Nếu Mô hình A báo $R^2 = 0.98$ còn Mô hình B báo $R^2 = 0.91$, ta không được phép kết luận ngay rằng A "khoa học hơn" B. Con số 0.98 của A được sinh ra từ một giao thức đã hấp thụ thông tin từ test, nên ý nghĩa của nó không còn ngang với 0.91 của B. Đây là ví dụ điển hình cho việc độ phức tạp và chỉ số đẹp không tự động chuyển thành sức mạnh chứng cứ.

Ở đây người học thường phản biện: "nhưng nếu mô hình học được rất tốt thì chắc vẫn có điều gì đó đúng". Câu này chỉ đúng một phần nhỏ. Học được một hàm mất mát thấp chưa nói gì đủ mạnh về tính đúng đắn của bài toán, về tính sạch của giao thức, hay về khả năng dùng kết quả như bằng chứng ngoài mẫu.

Mệnh đề 14.2. *Giả sử $\widehat{s}_1, \ldots, \widehat{s}_m$ là các ước lượng ngẫu nhiên của cùng một chất lượng thật s trên cùng một tập test, và ta báo cáo*

$$\widehat{s}_{\text{max}} = \max\{\widehat{s}_1, \ldots, \widehat{s}_m\}.$$

Nếu các $\widehat{s}_i$ không gần như chắc chắn bằng nhau, thì

$$\mathbb{E}[\widehat{s}_{\text{max}}] \geq \mathbb{E}[\widehat{s}_i] = s,$$

và bất đẳng thức là nghiêm ngặt trong nhiều trường hợp thông thường.

Proof. Với mọi i, ta luôn có

$$\widehat{s}_{\text{max}} \geq \widehat{s}_i.$$

Lấy kỳ vọng hai vế cho từng i, suy ra

$$\mathbb{E}[\widehat{s}_{\text{max}}] \geq \mathbb{E}[\widehat{s}_i].$$

Nếu mỗi $\widehat{s}_i$ là ước lượng không chệch của cùng đại lượng s, thì $\mathbb{E}[\widehat{s}_i] = s$. Do đó

$$\mathbb{E}[\widehat{s}_{\text{max}}] \geq s.$$

Trong các trường hợp mà biến ngẫu nhiên $\widehat{s}_{\text{max}}$ có xác suất dương lớn hơn hẳn từng $\widehat{s}_i$, bất đẳng thức sẽ là nghiêm ngặt. □

Mệnh đề trên giải thích rất trực tiếp vì sao việc chạy nhiều seed rồi chọn kết quả đẹp nhất trên cùng một test làm nảy sinh thiên lệch lạc quan. Không cần đến một lý thuyết quá sâu; chỉ riêng phép lấy cực đại đã đủ tạo ra thiên lệch hướng lên.

Ví dụ 14.3. Giả sử cùng một kiến trúc được huấn luyện với ba seed, và trên cùng tập test ta thu được R^2 lần lượt là

$$0.90, \qquad 0.93, \qquad 0.97.$$

Nếu ta chỉ công bố 0.97 mà không nói nó là cực đại của ba lần thử, người đọc sẽ tưởng 0.97 là một đại diện trung thực cho hiệu năng điển hình. Trong khi đó, trung bình ba lần thử chỉ là

$$\frac{0.90 + 0.93 + 0.97}{3} = 0.933\overline{3}.$$

Sự khác biệt giữa 0.97 và 0.933 không phải chỉ là "một con số nhỏ". Nó là khác biệt giữa *kết quả đẹp nhất* và *kết quả điển hình*.

Một ngộ nhận hẹp hơn nhưng rất phổ biến là nghĩ rằng *data leakage* chỉ có nghĩa là đưa nhãn tương lai trực tiếp vào đầu vào. Với chuỗi thời gian, cách hiểu này quá hẹp.

Ví dụ 14.4. Xét chuỗi một chiều

$$x_1 = 1, \quad x_2 = 2, \quad x_3 = 3, \quad x_4 = 100.$$

Giả sử train là ba điểm đầu, test là điểm cuối.

Nếu chuẩn hóa đúng cách trên train, ta có

$$\mu_{\text{train}} = \frac{1+2+3}{3} = 2, \qquad \sigma_{\text{train}} = \sqrt{\frac{(1-2)^2 + (2-2)^2 + (3-2)^2}{3}} = \sqrt{\frac{2}{3}}.$$

Do đó

$$\widetilde{x}_1^{(\text{train})} = \frac{1-2}{\sqrt{2/3}}, \qquad \widetilde{x}_2^{(\text{train})} = 0, \qquad \widetilde{x}_3^{(\text{train})} = \frac{1}{\sqrt{2/3}}.$$

Nếu chuẩn hóa sai trên toàn bộ dữ liệu, ta có

$$\mu_{\text{all}} = \frac{1+2+3+100}{4} = 26.5.$$

Rõ ràng mọi điểm train lúc này bị đặt trong một hệ quy chiếu đã hấp thụ giá trị 100 của tương lai. Dù nhãn tương lai không bị đưa thẳng vào đầu vào, thông tin tương lai vẫn đã đi vào pipeline qua thống kê chuẩn hóa.

Ví dụ trên cho thấy leakage không nhất thiết xuất hiện dưới dạng một cột nhãn bị chép nhầm vào đặc trưng. Nó có thể đến từ các phép biến đổi tưởng như vô hại nhưng được fit sai thời điểm.

14.3 Cụm ngộ nhận thứ hai: nhầm logic mờ với xác suất, và nhầm sự hiện diện của luật với khả năng diễn giải

Ngộ nhận ở vùng này thường có hai tầng. Tầng thứ nhất là nhầm giá trị hàm thuộc với xác suất. Tầng thứ hai là nhầm việc "có luật mờ" với việc "đầu ra cuối giải thích được".

Ví dụ 14.5. Phát biểu "xác suất ngày mai mưa là 0.8" nói về một biến cố ngẫu nhiên chưa xảy ra. Phát biểu "nhiệt độ 31°C thuộc khái niệm nóng ở mức 0.8" nói về một giá trị đã quan sát, và đo mức độ phù hợp của nó với nhãn ngôn ngữ 'nóng'.

Hai phát biểu này cùng dùng số 0.8, nhưng một bên là xác suất của biến cố, một bên là độ thuộc của khái niệm. Nếu ta thay thế câu thứ hai bằng "xác suất 31 độ C là nóng bằng 0.8", thì câu nói đã đổi nghĩa hoàn toàn. Nó không còn là logic mờ nữa, mà là một thứ lai tạp mơ hồ giữa xác suất và ngôn ngữ.

Phân biệt này không phải chuyện triết học mơ hồ. Nó tác động trực tiếp đến cách ta đọc luật mờ. Khi một luật nói "IF return is HIGH", nó không nói "xác suất return nằm trong vùng HIGH là bao nhiêu". Nó nói giá trị return hiện tại phù hợp với khái niệm HIGH ở mức nào.

Sai lầm thứ hai còn phổ biến hơn: thấy luật thì tưởng đã có giải thích.

Ví dụ 14.6. Giả sử một luật Sugeno có dạng

$$\text{IF } x_1 \text{ is LOW AND } x_2 \text{ is HIGH THEN } y = 0.1 + 3x_1 - 2x_2.$$

Nếu ta chỉ in ra phần

$$\text{IF } x_1 \text{ is LOW AND } x_2 \text{ is HIGH},$$

thì ta mới biết vùng nào của không gian đầu vào đang được nói tới. Ta chưa biết khi luật kích hoạt thì đầu ra bị kéo lên hay kéo xuống, mạnh hay yếu, phụ thuộc tuyến tính ra sao vào x_1, x_2. Nói rằng "đã giải thích được luật" trong tình huống này là quá sớm.

Một lớp ngộ nhận sâu hơn nữa là nghĩ rằng chỉ cần gắn hai nhãn LOW/HIGH cho hai Gaussian thì nhãn ngôn ngữ ấy sẽ còn đúng sau khi huấn luyện.

Ví dụ 14.7. Giả sử ban đầu ta khởi tạo hai tâm

$$c_{\text{LOW}} = -1, \qquad c_{\text{HIGH}} = 1.$$

Ban đầu, cách đặt tên này có ý nghĩa.

Nhưng sau khi học tự do, mô hình có thể cập nhật thành

$$c_{\text{LOW}} = 0.8, \qquad c_{\text{HIGH}} = -0.2.$$

Nếu ta vẫn tiếp tục in luật "return is LOW" cho Gaussian đầu tiên chỉ vì nó mang chỉ số 0, thì chữ LOW đã không còn là 'thấp' theo nghĩa ngôn ngữ nữa. Nó chỉ còn là tên của một tham số.

Ví dụ này giải thích vì sao trong các hệ mờ học bằng gradient, tính diễn giải cần được *bảo vệ bằng ràng buộc cấu trúc*. Nếu không có ràng buộc, nhãn ngôn ngữ rất dễ trượt thành nhãn kỹ thuật.

Ngộ nhận về diễn giải còn gắn liền với bài toán đầu ra có cấu trúc, như OHLC.

Ví dụ 14.8. Giả sử mô hình dự báo

$$\widehat{O} = 100, \qquad \widehat{C} = 102, \qquad \widehat{H} = 101, \qquad \widehat{L} = 99.$$

Khi đó

$$\widehat{H} = 101 < \max\{\widehat{O}, \widehat{C}\} = 102.$$

Bộ dự báo này vi phạm hình học của một cây nến giá hợp lệ, mặc dù từng thành phần riêng lẻ có thể vẫn gần với giá thật theo RMSE hoặc MAE.

Ví dụ này cho thấy một bài học quan trọng: một mô hình có thể "đúng theo từng tọa độ" mà vẫn "sai theo cấu trúc". Nếu người học chỉ nhìn các chỉ số riêng lẻ mà không kiểm tra ràng buộc hình học của đầu ra, họ sẽ đánh giá quá cao chất lượng thực sự của mô hình.

Cuối cùng, ở vùng này còn một phản ứng quá tay khác: cứ thấy BiLSTM là kết luận ngay "nhìn vào tương lai".

Ví dụ 14.9. Giả sử tại thời điểm t, ta có trọn vẹn cửa sổ quá khứ

$$(x_{t-59}, x_{t-58}, \ldots, x_t).$$

Một BiLSTM xử lý chuỗi này theo hai hướng: từ x_{t-59} đến x_t, và từ x_t ngược về x_{t-59}. Cả hai hướng đều chỉ đi trong tập thông tin đã biết tại thời điểm dự báo. Vì vậy, bản thân BiLSTM trong tình huống này không tự động gây leakage.

Điều nên phê bình ở đây không phải là "nhìn vào tương lai thật", mà là việc diễn giải động học trở nên khó hơn và mô hình có thêm độ tự do biểu diễn. Phân biệt như vậy giúp người học không đi từ ngộ nhận này sang ngộ nhận ngược lại.

14.4 Cụm ngộ nhận thứ ba: nhầm dáng vẻ học thuật, trích dẫn và khả năng chạy một lần với chiều sâu thật sự

Ngộ nhận cuối cùng ít ồn ào hơn nhưng lại rất dai: cứ nhiều thuật ngữ tiếng Anh, nhiều trích dẫn, nhiều tên bài báo nổi tiếng thì nội dung tự khắc sâu. Sự thật là thuật ngữ và trích dẫn chỉ có giá trị khi chúng được gắn với một cơ chế giải thích cụ thể.

Ví dụ 14.10. Xét hai cách viết sau về cùng một ý tưởng.

Cách viết A:

> Mô hình dùng *gating*, *hybrid fuzzy representation*, *calibration* và *ablation study*-oriented design.

Cách viết B:

> Mô hình trộn đầu ra của nhánh mờ và nhánh thời gian bằng một lớp học được. Muốn biết nhánh nào thật sự có ích, ta phải làm nghiên cứu triệt giảm thành phần: bỏ từng nhánh rồi đo lại trên cùng giao thức.

Cách A nghe "chuyên môn" hơn, nhưng Cách B mới giải thích cơ chế. Nếu người học thấy Cách A sâu hơn chỉ vì có nhiều tiếng Anh hơn, thì đó là một ngộ nhận về phong cách học thuật.

Trích dẫn tài liệu cũng bị ngộ nhận theo cách tương tự.

Ví dụ 14.11. Giả sử một bài báo năm 2010 dùng ANFIS cho dữ liệu tháng của chỉ số vĩ mô và báo R^2 rất cao. Nếu một script hiện tại dùng dữ liệu ngày, dùng OHLC, thêm BiLSTM, thêm cách tạo đặc trưng mới, rồi gợi ý rằng kết quả của bài báo kia là hậu thuẫn trực tiếp cho mô hình mới, thì lập luận đã nhảy quá xa.

Điều trung thực hơn phải là: bài báo cũ cho một nguồn cảm hứng về việc dùng hai Gaussian membership functions hoặc về tinh thần ANFIS. Nó không thể được dùng như bằng chứng thực nghiệm trực tiếp cho một bối cảnh dữ liệu và giao thức đã thay đổi mạnh.

Một ngộ nhận nữa là coi *tái lập* như đồng nghĩa với "chạy được một lần và lưu được file trọng số".

Ví dụ 14.12. Giả sử một dự án có các yếu tố sau:

1. đường dẫn dữ liệu được mã hóa cứng theo máy tác giả;
2. mô hình có lớp tùy biến nhưng không đăng ký để tải lại thuận tiện;
3. không ghi rõ seed hay phiên bản thư viện;
4. kết quả được báo sau khi đã chọn run đẹp nhất.

Một dự án như vậy có thể vẫn chạy được trên máy tác giả và vẫn lưu được file `.keras`, nhưng rất khó gọi là tái lập tốt theo nghĩa nghiên cứu. Người khác chưa chắc dựng lại được cùng pipeline, cùng quyết định chọn mô hình, hay cùng kết quả cuối.

Điểm cốt lõi ở đây là: dáng vẻ học thuật không thể thay cho cơ chế học thuật. Một tài liệu thật sự sâu phải làm rõ giả thiết, mô hình, dữ liệu, giao thức và phạm vi phát biểu. Nếu chỉ có nhãn dán đẹp, thì cảm giác sâu rất có thể chỉ là cảm giác.

14.5 Một quy trình tự kiểm để chống ngộ nhận khi đọc mô hình

Ngộ nhận không biến mất chỉ bằng cách đọc xong một chương. Nó chỉ giảm khi người học tạo được một quy trình tự kiểm đủ cụ thể để dùng lặp đi lặp lại. Một quy trình tối thiểu có thể gồm năm bước sau.

1. Viết lại bài toán bằng công thức: đầu vào là gì, mục tiêu là gì, mốc thời gian của thông tin là gì.
2. Tách riêng ba lớp câu hỏi: biểu diễn, giao thức đánh giá, và khả năng diễn giải.
3. Kiểm tra từng phép tiền xử lý xem nó được fit trên tập nào.
4. Với các phát biểu mạnh như “explainable”, “state-of-the-art”, “better”, luôn hỏi: tốt hơn ở chỉ số nào, trên giao thức nào, và so với đường cơ sở nào.
5. Cố ý tạo một phản ví dụ nhỏ: một chuỗi toy, một bộ OHLC vô lý, hoặc một luật thiếu hệ quả, để xem mệnh đề của mình có còn đứng được không.

Nhận xét 14.13. Nếu một nhận định không sống sót qua một phản ví dụ toy rất nhỏ, thì nhận định đó thường đang quá mạnh so với chứng cứ. Đây là một nguyên tắc kiểm tra cực kỳ hữu ích khi đọc cả bài báo lẫn mã nguồn.

14.6 Kết luận chương

Các ngộ nhận trong chương này không phải những “lỗi của người mới” theo nghĩa tầm thường. Chúng là các lối tắt nhận thức rất hấp dẫn, đến mức ngay cả người đã học khá lâu vẫn có thể mắc nếu không tự kiểm thường xuyên. Vì vậy, mục tiêu cuối cùng không phải là học thuộc một danh sách cấm, mà là tạo ra một thói quen trí tuệ: khi đứng trước một mô hình, phải luôn có đủ bình tĩnh để hỏi xem cái gì là biểu diễn, cái gì là giao thức, cái gì là diễn giải, và cái gì mới chỉ là vẻ ngoài.

Nếu giữ được thói quen đó, người học sẽ không chỉ tránh được vài ngộ nhận về ML, DL hay logic mờ. Họ sẽ có một cách đọc mô hình trưởng thành hơn: ít bị choáng bởi sự hào nhoáng, ít kết luận oan, và khó tự đánh lừa mình hơn khi đi từ lý thuyết sang code thật.

Bibliography

[1] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006. ISBN 9780387310732.

[2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. ISBN 9780262035613.

[3] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer, 2 edition, 2009. ISBN 9780387848587.

[4] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.

[5] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, 3 edition, 2021. URL https://otexts.com/fpp3/.

[6] Jyh-Shing Roger Jang. Anfis: Adaptive-network-based fuzzy inference system. *IEEE Transactions on Systems, Man, and Cybernetics*, 23(3):665–685, 1993. doi: 10.1109/21.256541.

[7] George J. Klir and Bo Yuan. *Fuzzy Sets and Fuzzy Logic: Theory and Applications*. Prentice Hall, 1995. ISBN 9780131011717.

[8] Kevin P. Murphy. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022. ISBN 9780262046824.

[9] Timothy J. Ross. *Fuzzy Logic with Engineering Applications*. Wiley, 3 edition, 2010. ISBN 9780470743768.

[10] Mike Schuster and Kuldip K. Paliwal. Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11):2673–2681, 1997. doi: 10.1109/78.650093.

[11] Lotfi A. Zadeh. Fuzzy sets. *Information and Control*, 8(3):338–353, 1965. doi: 10.1016/S0019-9958(65)90241-X.